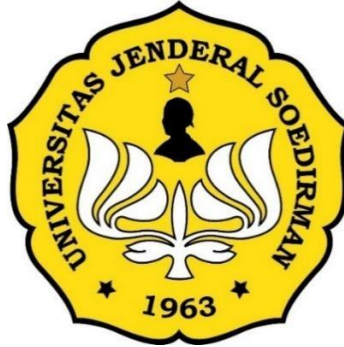


**PENERAPAN KONSEP ALGORITMA DAN PEMROGRAMAN DALAM
PENYELESAIAN ENAM STUDI KASUS**



Disusun untuk memenuhi tugas terstruktur:

Mata Kuliah : Algoritma dan Pemrograman

Dosen Pengampu : Lutfiah Maharani Siniwi, M.Stat.

Kelompok H - Statistika B

Penyusun:

Jihan Nur Khanifah	K1D025056
Fajar Kurniawa	K1D025060
Nisrinaya Aprilla	K1D025074
Winahyu Tri Sulistyo	K1D025076
Hilma Nur Wildatunnisa	K1D025084

**PROGRAM STUDI STATISTIKA
UNIVERSITAS JENDERAL SOEDIRMAN**

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi telah mendorong kebutuhan akan kemampuan berpikir komputasional (*computational thinking*) dan pemrograman dalam berbagai bidang ilmu. Pemrograman tidak hanya berfungsi sebagai sarana untuk menulis kode, tetapi juga sebagai alat untuk melatih kemampuan analisis, pemecahan masalah, serta penyusunan solusi yang sistematis. Kemampuan tersebut menjadi penting karena berbagai permasalahan di era digital memerlukan pendekatan logis dan terstruktur untuk memperoleh solusi yang efektif. Penelitian oleh Yuen, Liu, dan Leong (2023) menunjukkan bahwa pembelajaran pemrograman berkontribusi terhadap peningkatan kemampuan berpikir komputasional dan pemecahan masalah melalui proses identifikasi masalah, perancangan algoritma, hingga implementasi solusi dalam bentuk program.

Dalam pembelajaran Algoritma dan Pemrograman, mahasiswa tidak hanya dituntut memahami sintaks bahasa pemrograman, tetapi juga mampu menerjemahkan permasalahan nyata ke dalam langkah-langkah algoritmik yang dapat dijalankan komputer. Zhou dkk. (2023) menjelaskan bahwa kemampuan berpikir komputasional membantu seseorang dalam melakukan dekomposisi masalah, mengenali pola, melakukan abstraksi, dan menyusun algoritma untuk menyelesaikan persoalan secara sistematis.

Berdasarkan ketentuan project mata kuliah Algoritma dan Pemrograman, mahasiswa diminta menyelesaikan enam studi kasus yang mencakup analisis teks, manipulasi string, penyelesaian persamaan kuadrat, pengolahan data NIP ASN, klasifikasi cluster tiga dimensi, dan perhitungan interval konfidensi proporsi. Keenam kasus tersebut dipilih karena mewakili penerapan konsep dasar pemrograman seperti penggunaan variabel, tipe data, operator, percabangan, fungsi, struktur data, serta operasi matematika dalam penyelesaian masalah komputasional.

Melalui project ini, mahasiswa diharapkan mampu mengembangkan kemampuan analisis masalah, perancangan algoritma, implementasi program menggunakan Python dan R, serta melakukan pengujian program secara sistematis. Selain itu, project ini juga menjadi sarana untuk memahami bagaimana suatu permasalahan dapat diselesaikan melalui pendekatan algoritmik yang efektif dan efisien.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah dalam project ini adalah sebagai berikut:

1. Bagaimana menganalisis kebutuhan input, proses, output, dan batasan pada setiap studi kasus yang diberikan?
2. Bagaimana merancang algoritma dalam bentuk pseudocode dan flowchart yang sesuai dengan permasalahan?
3. Bagaimana mengimplementasikan algoritma ke dalam bahasa pemrograman Python dan R dengan logika yang setara?
4. Bagaimana melakukan validasi input dan penanganan kondisi khusus pada setiap program?
5. Bagaimana menguji program untuk memastikan hasil yang diperoleh sesuai dengan kebutuhan dan ketentuan soal?

1.3 Tujuan

Berdasarkan rumusan masalah tersebut, tujuan dalam project ini adalah sebagai berikut:

1. Menganalisis permasalahan komputasional berdasarkan kebutuhan input, proses, output, dan batasan masalah.
2. Merancang solusi dalam bentuk algoritma, pseudocode, dan flowchart yang sistematis.
3. Mengimplementasikan algoritma menggunakan bahasa pemrograman Python dan R.
4. Menerapkan konsep dasar pemrograman seperti variabel, tipe data, operator, string, percabangan, fungsi, dan struktur data.
5. Melakukan pengujian program untuk memastikan kebenaran hasil dan keandalan program.
6. Mengembangkan kemampuan berpikir komputasional dan pemecahan masalah melalui implementasi program komputer.

BAB II

ANALISIS MASALAH

2.1 Analisis Program 1: Menghitung Jumlah Kata dan Jumlah Kalimat

1. Deskripsi Masalah

Dalam pengolahan teks (*text processing*), informasi sederhana seperti jumlah kata dan jumlah kalimat sering digunakan sebagai dasar analisis dokumen. Program ini dirancang untuk membaca sebuah teks yang dimasukkan pengguna, kemudian menghitung jumlah kata dan jumlah kalimat yang terdapat pada teks tersebut. Kemampuan ini menjadi dasar berbagai aplikasi pengolahan bahasa alami (*Natural Language Processing*), seperti analisis dokumen, pemeriksaan keterbacaan teks, dan sistem pencarian informasi.

2. Analisis Penyelesaian

Program menerima masukan berupa sebuah teks. Untuk memperoleh jumlah kata, teks dipisahkan berdasarkan spasi sehingga setiap kata menjadi elemen yang dapat dihitung. Sementara itu, jumlah kalimat ditentukan berdasarkan jumlah tanda titik yang berfungsi sebagai penanda akhir kalimat.

Pendekatan ini dipilih karena lebih sederhana dan sesuai dengan batasan soal, yaitu tanda titik hanya digunakan sebagai akhir kalimat. Dengan demikian, program tidak perlu mempertimbangkan penggunaan titik pada singkatan atau angka desimal.

3. Input – Proses – Output

Komponen	Keterangan
Input	Teks dari pengguna
Proses	Memisahkan kata dan menghitung jumlah titik
Output	Jumlah kata dan jumlah kalimat

4. Batasan Masalah

- Tanda titik hanya digunakan untuk mengakhiri kalimat
- Tidak mempertimbangkan tanda baca lain seperti "?" atau "!"
- Teks dapat terdiri atas satu atau lebih kalimat

5. Kondisi Khusus

- Teks kosong
- Teks tanpa tanda titik
- Teks dengan banyak spasi berurutan

2.2 Analisis Program 2: Membuat String Kalimat

1. Deskripsi Masalah

String merupakan salah satu tipe data dasar yang paling sering digunakan dalam pemrograman. Program ini bertujuan membentuk beberapa objek string sehingga menghasilkan tampilan teks yang sama persis dengan format yang diberikan pada soal.

2. Analisis Penyelesaian

Tantangan utama dalam program ini bukan terletak pada perhitungan matematis, melainkan pada bagaimana programmer memahami representasi karakter dalam bahasa pemrograman. Beberapa karakter khusus seperti tanda petik tunggal, tanda petik ganda, atau karakter baris baru memerlukan penanganan khusus agar dapat ditampilkan dengan benar. Melalui soal ini, mahasiswa dilatih untuk memahami konsep penyimpanan data teks serta perbedaan penulisan string pada bahasa Python dan R.

3. Input – Proses – Output

Komponen	Keterangan
Input	Tidak ada
Proses	Membentuk string sesuai format
Output	Kalimat yang dicetak

4. Batasan Masalah

- Format hasil harus identik dengan soal
- Huruf besar, huruf kecil, spasi, dan tanda baca tidak boleh berubah

5. Kondisi Khusus

- Kesalahan penggunaan tanda kutip
- Kesalahan karakter escape

2.3 Analisis Program 3: Menentukan Akar Persamaan Kuadrat

1. Deskripsi Masalah

Persamaan kuadrat merupakan salah satu bentuk persamaan matematika yang banyak digunakan dalam berbagai bidang ilmu, seperti fisika, statistika, ekonomi, dan teknik. Program ini bertujuan menentukan akar-akar persamaan kuadrat berdasarkan nilai koefisien yang diberikan pengguna.

2. Analisis Penyelesaian

Penyelesaian diawali dengan membaca nilai koefisien a , b , dan c . Selanjutnya program menghitung diskriminan menggunakan rumus:

$$D = b^2 - 4ac$$

Nilai diskriminan digunakan untuk menentukan jenis akar persamaan.

- Jika $D > 0$, terdapat dua akar real berbeda
- Jika $D = 0$, terdapat satu akar real kembar
- Jika $D < 0$, akar bersifat imajiner

Penggunaan diskriminan penting karena memungkinkan program menentukan kondisi akar sebelum melakukan perhitungan lebih lanjut.

3. Input – Proses – Output

Komponen	Keterangan
Input	a, b, c
Proses	Menghitung diskriminan dan akar
Output	Nilai akar atau pesan akar imajiner

4. Batasan Masalah

- Nilai a tidak boleh sama dengan nol
- Hanya akar real yang ditampilkan

5. Kondisi Khusus

- $a = 0$
- Diskriminan bernilai negatif
- Diskriminan bernilai nol

2.4 Analisis Program 4: Menampilkan Tanggal Lahir ASN Berdasarkan NIP

• Deskripsi Masalah

Nomor Induk Pegawai (NIP) ASN mengandung informasi tertentu, termasuk tanggal lahir pegawai. Program ini bertujuan mengekstrak informasi tersebut dari NIP dan menampilkannya dalam format yang lebih mudah dipahami.

• Analisis Penyelesaian

Program mengambil delapan digit pertama NIP yang merepresentasikan tahun, bulan, dan tanggal lahir. Setelah data dipisahkan, program menggunakan percabangan untuk mengubah kode bulan numerik menjadi nama bulan. Penggunaan percabangan diperlukan karena setiap angka bulan memiliki pasangan nama bulan yang berbeda.

• Input – Proses – Output

Komponen	Keterangan
Input	NIP ASN

Proses	Memotong digit dan mengonversi bulan
Output	Tanggal lahir ASN

- Batasan Masalah
 - NIP harus memiliki format yang benar
 - Digit bulan harus berada pada rentang 01 – 12
- Kondisi Khusus
 - Panjang NIP kurang dari 8 digit
 - Bulan tidak valid
 - Input mengandung huruf

2.5 Analisis Program 5: Klasifikasi Cluster Tiga Dimensi

1. Deskripsi Masalah

Klasifikasi ialah proses pengelompokan data berdasarkan karakteristik tertentu. Pada soal ini, sebuah titik dalam ruang tiga dimensi akan diklasifikasikan ke dalam cluster A , B , atau C berdasarkan jarak terdekat terhadap pusat cluster yang telah ditentukan.

2. Analisis Penyelesaian

Program menerima koordinat titik $U(x_1, x_2, x_3)$. Selanjutnya jarak titik tersebut terhadap pusat cluster A , B , dan C dihitung menggunakan rumus jarak Euclidean. Setelah itu, program membandingkan seluruh nilai jarak. Cluster dengan jarak terkecil dianggap sebagai kelompok yang paling merepresentasikan posisi titik tersebut.

Secara konsep, metode ini merupakan bentuk sederhana dari algoritma klasifikasi berbasis kedekatan (distance-based classification) yang banyak digunakan dalam analisis data dan machine learning.

3. Input – Proses – Output

Komponen	Keterangan
Input	Koordinat titik U
Proses	Menghitung jarak ke tiga pusat cluster
Output	Cluster A , B , atau C

4. Batasan Masalah

- Titik harus memiliki tiga koordinat numerik
- Pusat cluster bersifat tetap

5. Kondisi Khusus

- Jarak ke dua cluster sama

- Input bukan angka
- Koordinat tidak lengkap

2.6 Analisis Program 6: Interval Konfidensi Proporsi

1. Deskripsi Masalah

Dalam statistika, interval konfidensi digunakan untuk memperkirakan rentang nilai parameter populasi berdasarkan data sampel. Program ini bertujuan menghitung interval konfidensi proporsi populasi menggunakan proporsi sampel yang diberikan pengguna.

2. Analisis Penyelesaian

Program menerima nilai proporsi sampel ($\hat{p}$), ukuran sampel (n), dan tingkat signifikansi (α). Berdasarkan nilai (α), program menentukan nilai z yang sesuai. Selanjutnya program menghitung margin of error menggunakan rumus:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Nilai tersebut digunakan untuk menentukan batas bawah dan batas atas interval konfidensi. Program juga harus melakukan validasi karena secara teori nilai proporsi hanya berada pada rentang 0 sampai 1. Jika pengguna memasukkan nilai di luar rentang tersebut, hasil perhitungan tidak memiliki makna statistik sehingga program harus menampilkan pesan kesalahan.

3. Input – Proses – Output

Komponen	Keterangan
Input	$\hat{p}$, n , α
Proses	Menghitung margin of error dan interval konfidensi
Output	Batas bawah dan batas atas interval

4. Batasan Masalah

- Nilai $\hat{p}$ harus berada pada rentang $0 \leq \hat{p} \leq 1$
- Ukuran sampel harus lebih besar dari nol
- α hanya 5% atau 10% sesuai soal

5. Kondisi Khusus

- $\hat{p} < 0$
- $\hat{p} > 1$
- $n \leq 0$
- α tidak sesuai ketentuan

BAB III

PSEUDOCODE DAN FLOWCHART

3.1 Pseudocode

1. Program 1: Menghitung Jumlah Kata dan Jumlah Kalimat

START

DEKLARASI:

teks: string

jumlah_kalimat, jumlah_kata: integer

INPUT:

Tampilkan "Masukkan teks: "

Baca teks dari pengguna

PROSES:

jumlah_kalimat $\leftarrow$ banyaknya karakter "." dalam teks

jumlah_kata $\leftarrow$ banyaknya pemisahan teks berdasarkan spasi

OUTPUT:

Tampilkan "Teks tersebut memuat ", jumlah_kalimat, " kalimat dan ",
jumlah_kata, " kata."

END

2. Program 2: Membuat String Kalimat

START

DEKLARASI:

K1, K2, K3, K4: string

INPUT:

Tidak ada input

PROSES:

Kata 1 $\leftarrow$ “Saya tak ‘kan menyerah.”

Kata 2 $\leftarrow$ “Ia berkata, “Aku menyayangimu.”

Kata 3 $\leftarrow$ “Coba jelaskan pengertian “cross-validation” dalam Machine Learning!”

Kata 4 $\leftarrow$ “Surat keputusan itu bernomor 62/UN.34/19/2023.”

OUTPUT:

Tampilkan Kata 1

Tampilkan Kata 2

Tampilkan Kata 3

Tampilkan Kata 4

END

3. Program 3: Menentukan Akar Persamaan Kuadrat

START

DEKLARASI:

a, b, c : real

diskriminan : real

x1, x2 : real

koefisien : vector

FUNGSI akar_kuadrat (koefisien)

a $\leftarrow$ koefisien [1]

b $\leftarrow$ koefisien [2]

c $\leftarrow$ koefisien [3]

diskriminan $\leftarrow b^2 - 4ac$

IF diskriminan ≥ 0 THEN

x1 $\leftarrow \frac{-b + \sqrt{\text{diskriminan}}}{2a}$

$$x_2 \leftarrow \frac{-b - \sqrt{\text{diskriminan}}}{2a}$$

Tampilkan “Akar pertama = “, x1

Tampilkan “Akar kedua = “, x2

ELSE

Tampilkan “Persamaan hanya memiliki akar imajiner”

END IF

END FUNGSI

PROGRAM UTAMA

INPUT:

INPUT a

INPUT b

INPUT c

koefisien $\leftarrow$ [a, b, c]

PANGGIL akar_kuadrat (koefisien)

END

4. Program 4: Menampilkan Tanggal Lahir ASN Berdasarkan NIP

START

DEKLARASI: nip : string

tahun, bulan, tanggal, namaBulan : string

DESKRIPSI:

Input NIP ASN

Ambil tahun lahir ASN, yaitu digit pertama hingga keempat NIP

Ambil bulan lahir ASN, yaitu digit kelima hingga keenam NIP

Ambil tanggal lahir ASN, yaitu digit ketujuh hingga kedelapan NIP

Jika bulan = "01" maka nama bulan "Januari"

Jika bulan = "02" maka nama bulan "Februari"

Jika bulan = "03" maka nama bulan "Maret"

Jika bulan = "04" maka nama bulan "April"

Jika bulan = "05" maka nama bulan "Mei"

Jika bulan = "06" maka nama bulan "Juni"

Jika bulan = "07" maka nama bulan "Juli"

Jika bulan = "08" maka nama bulan "Agustus"

Jika bulan = "09" maka nama bulan "September"

Jika bulan = "10" maka nama bulan "Oktober"

Jika bulan = "11" maka nama bulan "November"

Jika bulan = "12" maka nama bulan "Desember"

Jika bulan lebih dari 12 atau kurang dari 1, maka nama bulan = "Bulan Tidak Valid"

Tampilkan "Tanggal Lahir ASN : " tanggal, nama bulan, tahun"

END

5. Program 5: Klasifikasi Cluster Tiga Dimensi

START

INPUT x1, x2, x3

SET A = (2, 1, 3)

SET B = (1, -4, 6)

SET C = (-2, 3, -2)

Hitung_jarak (x, y):

$$\text{RETURN } \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2}$$

Jarak_A ← hitung_jarak (U, A)

Jarak_B ← hitung_jarak (U, B)

Jarak_C ← hitung_jarak (U, C)

```

IF jarak_A < jarak_B AND jarak_A < jarak_C THEN
    cluster ← "A"
ELSE IF jarak_B < jarak_A AND jarak_B < jarak_C THEN
    cluster ← "B"
ELSE
    cluster ← "C"

```

```

OUTPUT jarak_A
OUTPUT jarak_B
OUTPUT jarak_C
OUTPUT cluster

```

END

6. Program 6: Interval Konfidensi Proporsi

START

INPUT p_hat

INPUT alpha

INPUT n

IF p_hat < 0 OR p_hat > 1 THEN

 OUTPUT "Input tidak valid"

ELSE

 IF alpha = 0.10 THEN

 z ← 1.645

 ELSE IF alpha = 0.05 THEN

 z ← 1.96

 ELSE

 OUTPUT "Alpha hanya boleh 0.10 atau 0.05"

 STOP

END IF

margin_error ← $z \times \sqrt{(p_hat \times (1 - p_hat))/n}$

$\text{batas_bawah} \leftarrow p_hat - \text{margin_error}$

$\text{batas_atas} \leftarrow p_hat + \text{margin_error}$

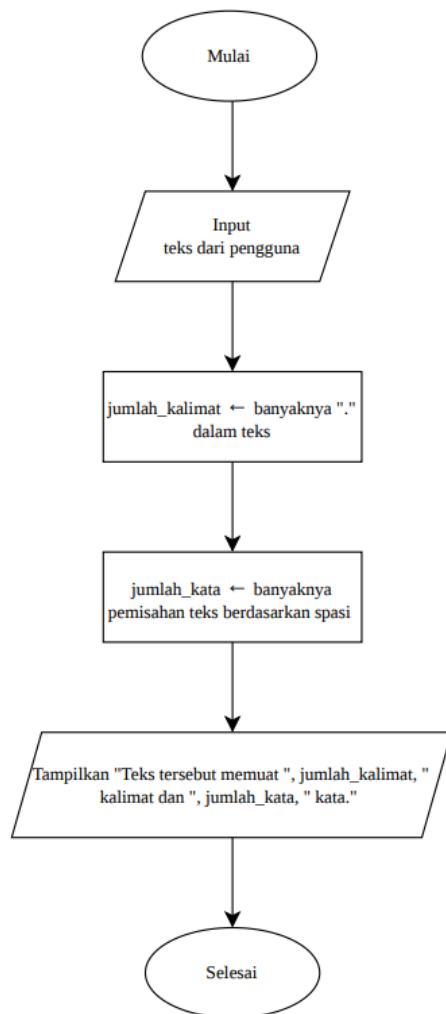
OUTPUT "Interval Konfidensi :"

OUTPUT batas_bawah

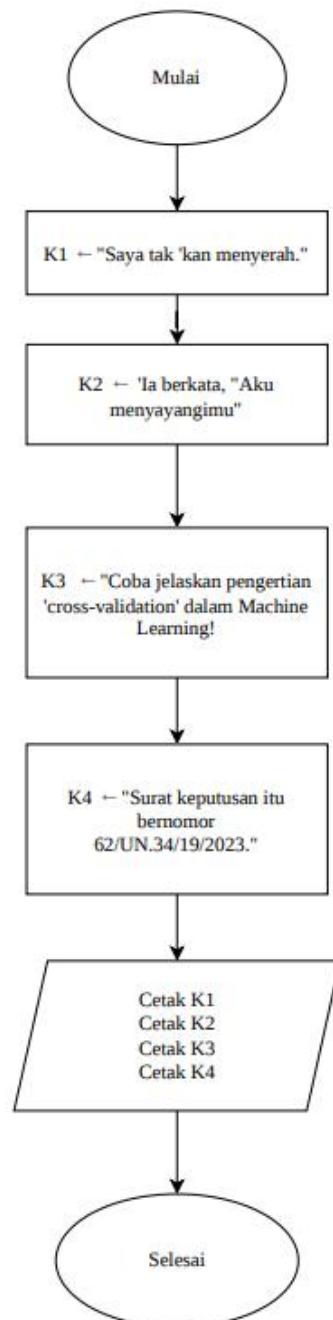
OUTPUT batas_atas

3.2 Flowchart

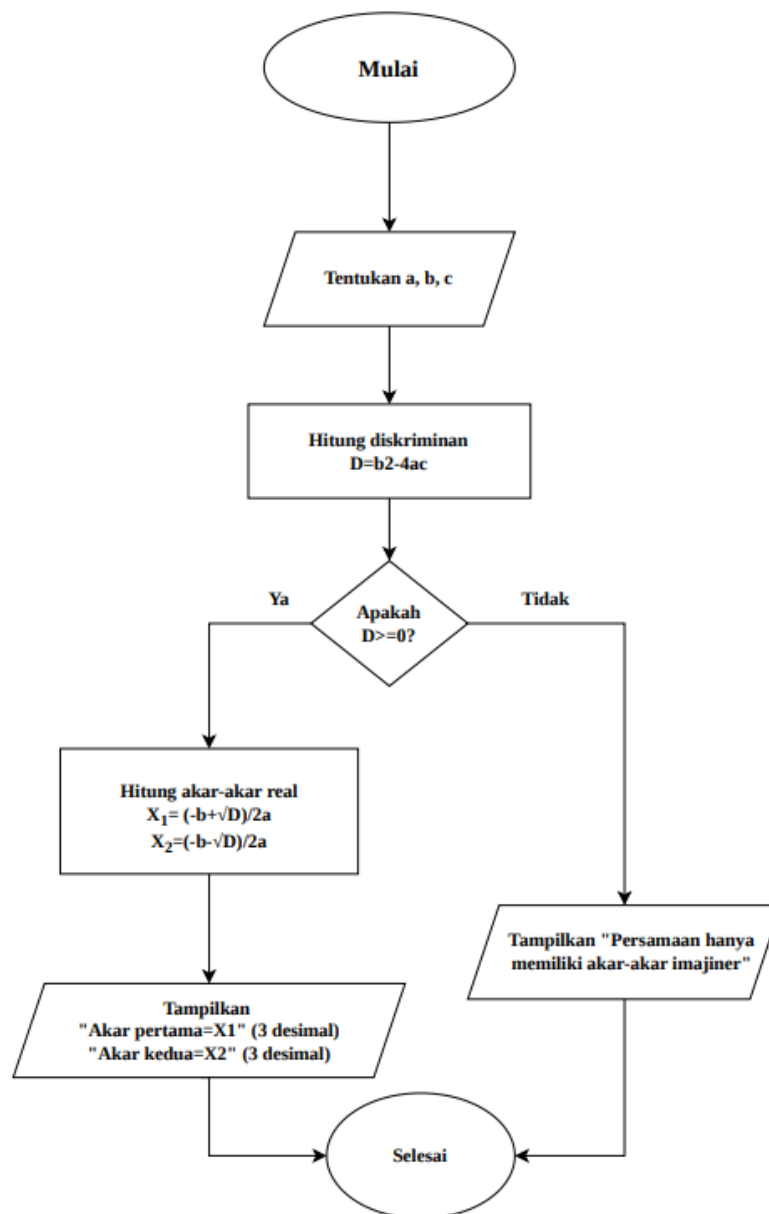
1. Program 1: Menghitung Jumlah Kata dan Jumlah Kalimat



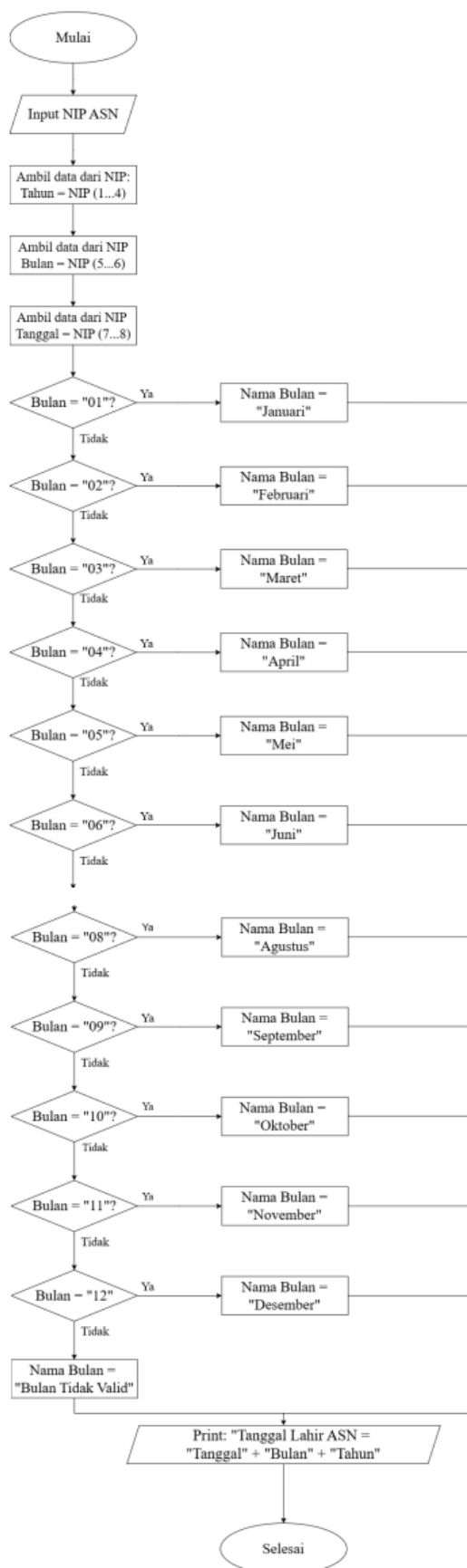
2. Program 2: Membuat String Kalimat



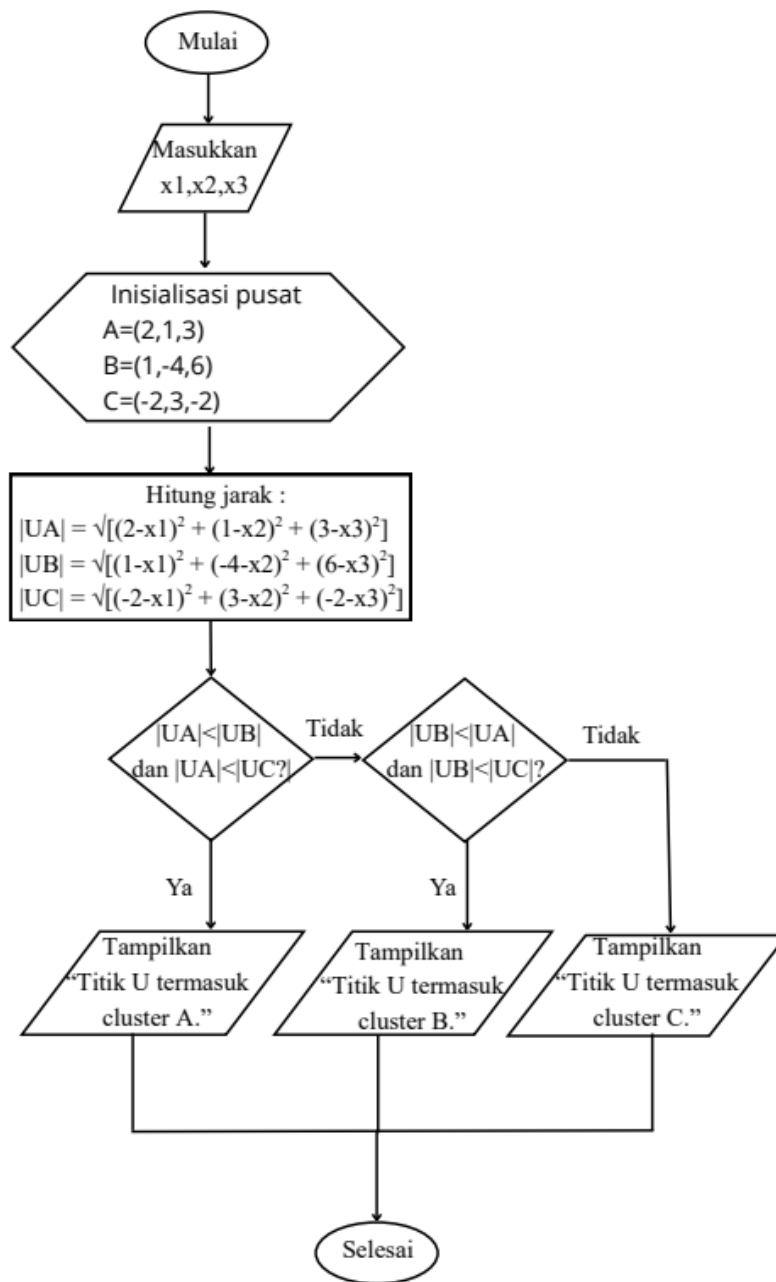
3. Program 3: Menentukan Akar Persamaan Kuadrat



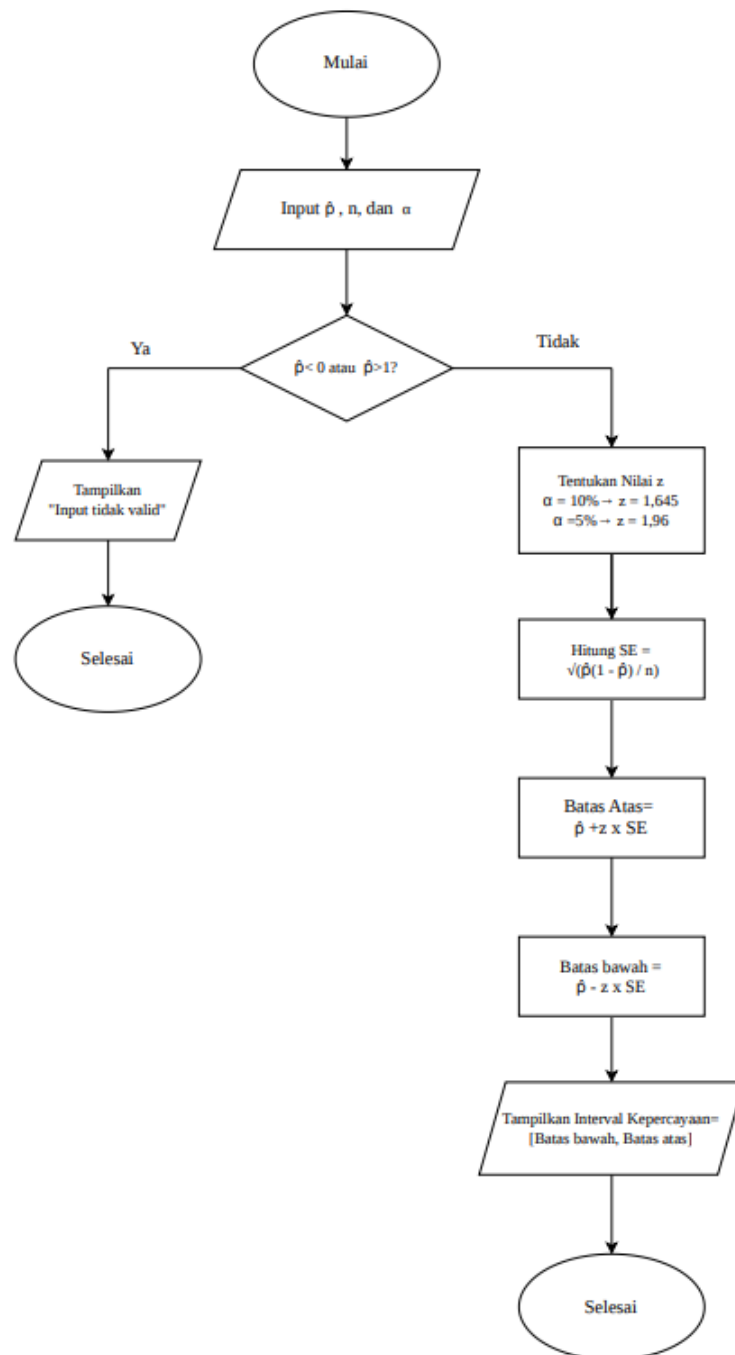
4. Program 4: Menampilkan Tanggal Lahir ASN Berdasarkan NIP



5. Program 5: Klasifikasi Cluster Tiga Dimensi



6. Program 6: Interval Konfidensi Proporsi



BAB IV

IMPLEMENTASI PROGRAM

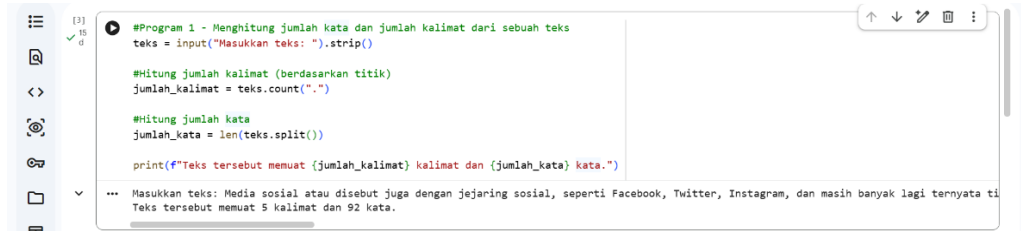
4.1 Source Code Python

1. Program 1: Menghitung Jumlah Kata dan Jumlah Kalimat

```
#Program 1 - Menghitung jumlah kata dan jumlah kalimat dari sebuah teks
#Menerima input teks dari pengguna dan menghapus spasi di awal maupun akhir teks
teks = input("Masukkan teks: ").strip()
#Menghitung jumlah tanda titik (.) dalam teks sebagai jumlah kalimat
jumlah_kalimat = teks.count(".")
#Memisahkan teks menjadi kata-kata berdasarkan spasi, lalu menghitung jumlah
katanya
jumlah_kata = len(teks.split())
#Menampilkan hasil jumlah kalimat dan jumlah kata
print(f"Teks tersebut memuat {jumlah_kalimat} kalimat dan {jumlah_kata} kata.")
```

Berikut adalah screenshot hasil skenario uji program 1 pada Python:

• Uji 1



```
#Program 1 - Menghitung jumlah kata dan jumlah kalimat dari sebuah teks
teks = input("Masukkan teks: ").strip()

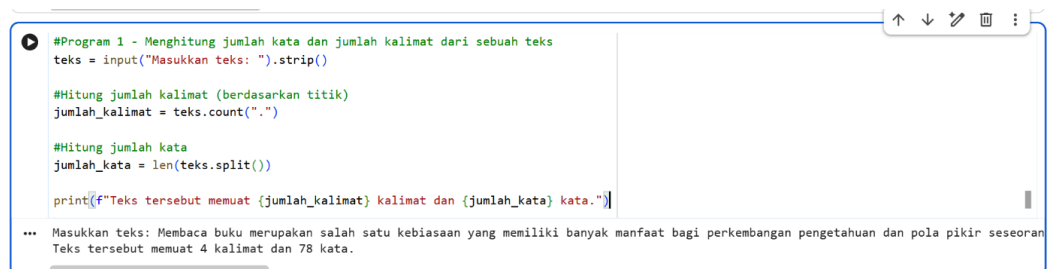
#Hitung jumlah kalimat (berdasarkan titik)
jumlah_kalimat = teks.count(".")

#Hitung jumlah kata
jumlah_kata = len(teks.split())

print(f"Teks tersebut memuat {jumlah_kalimat} kalimat dan {jumlah_kata} kata.")
```

... Masukkan teks: Media sosial atau disebut juga dengan jejaring sosial, seperti Facebook, Twitter, Instagram, dan masih banyak lagi ternyata ti
Teks tersebut memuat 5 kalimat dan 92 kata.

• Uji 2



```
#Program 1 - Menghitung jumlah kata dan jumlah kalimat dari sebuah teks
teks = input("Masukkan teks: ").strip()

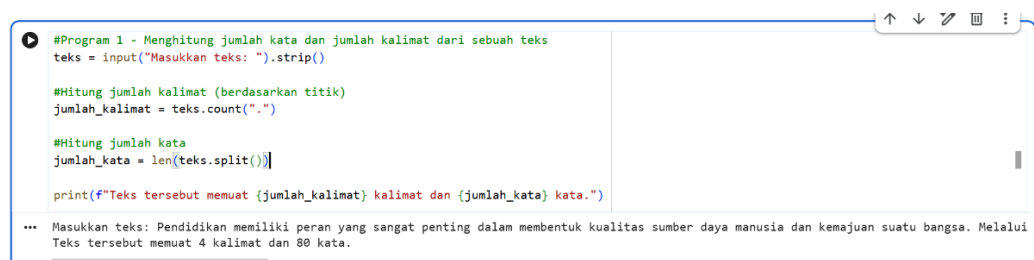
#Hitung jumlah kalimat (berdasarkan titik)
jumlah_kalimat = teks.count(".")

#Hitung jumlah kata
jumlah_kata = len(teks.split())

print(f"Teks tersebut memuat {jumlah_kalimat} kalimat dan {jumlah_kata} kata.")
```

... Masukkan teks: Membaca buku merupakan salah satu kebiasaan yang memiliki banyak manfaat bagi perkembangan pengetahuan dan pola pikir seseorang
Teks tersebut memuat 4 kalimat dan 78 kata.

• Uji 3



```
#Program 1 - Menghitung jumlah kata dan jumlah kalimat dari sebuah teks
teks = input("Masukkan teks: ").strip()

#Hitung jumlah kalimat (berdasarkan titik)
jumlah_kalimat = teks.count(".")

#Hitung jumlah kata
jumlah_kata = len(teks.split())

print(f"Teks tersebut memuat {jumlah_kalimat} kalimat dan {jumlah_kata} kata.")
```

... Masukkan teks: Pendidikan memiliki peran yang sangat penting dalam membentuk kualitas sumber daya manusia dan kemajuan suatu bangsa. Melalui
Teks tersebut memuat 4 kalimat dan 80 kata.

2. Program 2: Membuat String Kalimat

#Program 2 - Membuat objek string Kata1 sampai Kata4

Kata1 = "Saya tak 'kan menyerah."

Kata2 = 'Ia berkata, "Aku menyayangimu."'

Kata3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""

Kata4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."

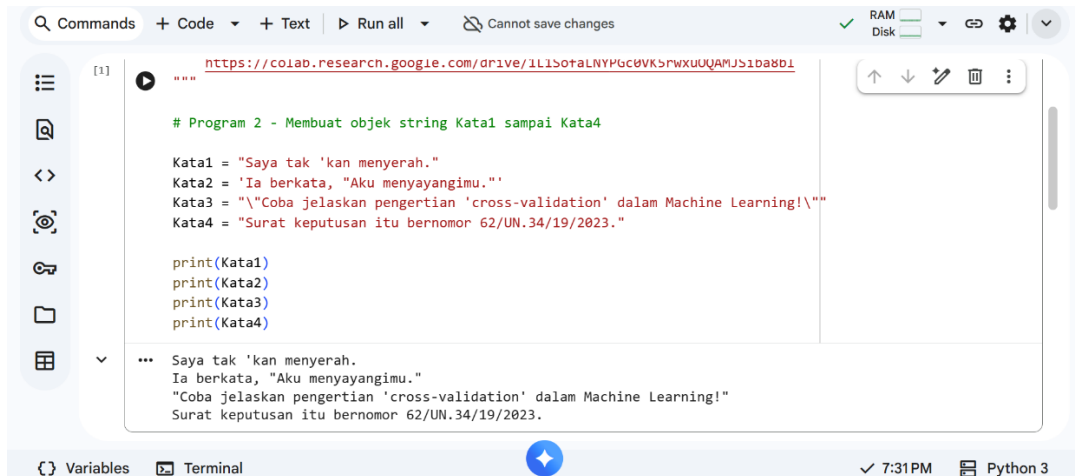
print(Kata1)

print(Kata2)

print(Kata3)

print(Kata4)

Berikut adalah screenshot hasil skenario uji program 2 pada Python:



```
https://colab.research.google.com/drive/1L15otaLNYPGc0VK5rwxU0UQAMJ51ba8D1

# Program 2 - Membuat objek string Kata1 sampai Kata4

Kata1 = "Saya tak 'kan menyerah."
Kata2 = 'Ia berkata, "Aku menyayangimu."'
Kata3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
Kata4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."

print(Kata1)
print(Kata2)
print(Kata3)
print(Kata4)
```

... Saya tak 'kan menyerah.
Ia berkata, "Aku menyayangimu."
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
Surat keputusan itu bernomor 62/UN.34/19/2023.

3. Program 3: Menentukan Akar Persamaan Kuadrat

import math

#Fungsi untuk menghitung akar persamaan kuadrat

def hitung_akar(data_koefisien):

 nilai_a, nilai_b, nilai_c = data_koefisien

 nilai_diskriminan = (nilai_b ** 2) - (4 * nilai_a * nilai_c)

 if nilai_diskriminan >= 0:

 akar_1 = (-nilai_b + math.sqrt(nilai_diskriminan)) / (2 * nilai_a)

 akar_2 = (-nilai_b - math.sqrt(nilai_diskriminan)) / (2 * nilai_a)

 print(f"Akar pertama = {akar_1:.3f}")

 print(f"Akar kedua = {akar_2:.3f}")

 else:

 print("Persamaan memiliki akar imajiner")

#Input pengguna

```

nilai_a = float(input("Masukkan koefisien a : "))
nilai_b = float(input("Masukkan koefisien b : "))
nilai_c = float(input("Masukkan koefisien c : "))

#List
data_koefisien = [nilai_a, nilai_b, nilai_c]

#Pemanggilan fungsi
hitung_akar(data_koefisien)

```

Berikut adalah screenshot hasil skenario uji program 3 pada Python:

- Uji 1

```

import math

# Fungsi untuk menghitung akar persamaan kuadrat
def hitung_akar(data_koefisien):

    nilai_a, nilai_b, nilai_c = data_koefisien

    nilai_diskriminan = (nilai_b ** 2) - (4 * nilai_a * nilai_c)

    if nilai_diskriminan >= 0:

        akar_1 = (-nilai_b + math.sqrt(nilai_diskriminan)) / (2 * nilai_a)
        akar_2 = (-nilai_b - math.sqrt(nilai_diskriminan)) / (2 * nilai_a)

        print(f"Akar pertama = {akar_1:.3f}")
        print(f"Akar kedua = {akar_2:.3f}")

    else:
        print("Persamaan memiliki akar imajiner")

# Input pengguna
nilai_a = float(input("Masukkan koefisien a : "))
nilai_b = float(input("Masukkan koefisien b : "))
nilai_c = float(input("Masukkan koefisien c : "))

# List
data_koefisien = [nilai_a, nilai_b, nilai_c]

# Pemanggilan fungsi
hitung_akar(data_koefisien)

```

Output:

```

Masukkan koefisien a : 1
Masukkan koefisien b : -5
Masukkan koefisien c : 6
Akar pertama = 3.000
Akar kedua = 2.000

```

- Uji 2

```

import math

# Fungsi untuk menghitung akar persamaan kuadrat
def hitung_akar(data_koefisien):

    nilai_a, nilai_b, nilai_c = data_koefisien

    nilai_diskriminan = (nilai_b ** 2) - (4 * nilai_a * nilai_c)

    if nilai_diskriminan >= 0:

        akar_1 = (-nilai_b + math.sqrt(nilai_diskriminan)) / (2 * nilai_a)
        akar_2 = (-nilai_b - math.sqrt(nilai_diskriminan)) / (2 * nilai_a)

        print(f"Akar pertama = {akar_1:.3f}")
        print(f"Akar kedua = {akar_2:.3f}")

    else:
        print("Persamaan memiliki akar imajiner")

# Input pengguna
nilai_a = float(input("Masukkan koefisien a : "))
nilai_b = float(input("Masukkan koefisien b : "))
nilai_c = float(input("Masukkan koefisien c : "))

# List
data_koefisien = [nilai_a, nilai_b, nilai_c]

# Pemanggilan fungsi
hitung_akar(data_koefisien)

```

Output:

```

Masukkan koefisien a : 2
Masukkan koefisien b : -2
Masukkan koefisien c : 4
Persamaan memiliki akar imajiner

```

- Uji 3

```
import math

# Fungsi untuk menghitung akar persamaan kuadrat
def hitung_akar(data_koefisien):

    nilai_a, nilai_b, nilai_c = data_koefisien

    nilai_diskriminan = (nilai_b ** 2) - (4 * nilai_a * nilai_c)

    if nilai_diskriminan >= 0:

        akar_1 = (-nilai_b + math.sqrt(nilai_diskriminan)) / (2 * nilai_a)
        akar_2 = (-nilai_b - math.sqrt(nilai_diskriminan)) / (2 * nilai_a)

        print(f"Akar pertama = {akar_1:.3f}")
        print(f"Akar kedua = {akar_2:.3f}")

    else:
        print("Persamaan memiliki akar imajiner")

# Input pengguna
nilai_a = float(input("Masukkan koefisien a : "))
nilai_b = float(input("Masukkan koefisien b : "))
nilai_c = float(input("Masukkan koefisien c : "))

# list
data_koefisien = [nilai_a, nilai_b, nilai_c]

# Pemanggilan fungsi
hitung_akar(data_koefisien)
```

Masukkan koefisien a : 0
Masukkan koefisien b : 3
Masukkan koefisien c : -2

```
ZeroDivisionError                                Traceback (most recent call last)
/tmp/ipykernel_1745/1217246129.py in <cell line: 0>()
     29
--> 30 # Pemanggilan fungsi
----> 31 hitung_akar(data_koefisien)

/tmp/ipykernel_1745/1217246129.py in hitung_akar(data_koefisien)
     10     if nilai_diskriminan >= 0:
     11
--> 12         akar_1 = (-nilai_b + math.sqrt(nilai_diskriminan)) / (2 * nilai_a)
     13         akar_2 = (-nilai_b - math.sqrt(nilai_diskriminan)) / (2 * nilai_a)
     14

ZeroDivisionError: float division by zero
```

Next steps: [Explain error](#)

4. Program 4: Menampilkan Tanggal Lahir ASN Berdasarkan NIP

#Input NIP ASN

NIP = input("Masukkan NIP ASN: ")

if not NIP.isdigit():

 raise ValueError("NIP Harus Berupa Angka")

#Mengambil tahun, bulan, dan tanggal lahir

Tahun = NIP[0:4]

Bulan = NIP[4:6]

Tanggal = NIP[6:8]

#Percabangan untuk nama bulan

if Bulan == "01":

 nama_bulan = "Januari"

elif Bulan == "02":

 nama_bulan = "Februari"

elif Bulan == "03":

 nama_bulan = "Maret"

elif Bulan == "04":

 nama_bulan = "April"

elif Bulan == "05":

```

        nama_bulan = "Mei"
elif Bulan == "06":
    nama_bulan = "Juni"
elif Bulan == "07":
    nama_bulan = "Juli"
elif Bulan == "08":
    nama_bulan = "Agustus"
elif Bulan == "09":
    nama_bulan = "September"
elif Bulan == "10":
    nama_bulan = "Oktober"
elif Bulan == "11":
    nama_bulan = "November"
elif Bulan == "12":
    nama_bulan = "Desember"
else:
    nama_bulan = "Bulan Tidak Valid"

```

#Output

```
print("Tanggal Lahir ASN:", Tanggal, nama_bulan, Tahun)
```

Berikut adalah screenshot hasil skenario uji program 4 pada Python:

- Uji 1

```

Program 4
File Edit View Insert Runtime Tools Help
Commands Code Text Run all
101
# Uji Pertama
# Input NIP ASN
NIP = input("Masukkan NIP ASN: ")
if not NIP.isdigit():
    raise ValueError("NIP Harus Berupa Angka")
# Mengambil tahun, bulan, dan tanggal lahir
Tahun = NIP[6:8]
Bulan = NIP[8:10]
Tanggal = NIP[10:12]
# Percabangan untuk nama bulan
if Bulan == "01":
    nama_bulan = "Januari"
elif Bulan == "02":
    nama_bulan = "Februari"
elif Bulan == "03":
    nama_bulan = "Maret"
elif Bulan == "04":
    nama_bulan = "April"
elif Bulan == "05":
    nama_bulan = "Mei"
elif Bulan == "06":
    nama_bulan = "Juni"
elif Bulan == "07":
    nama_bulan = "Juli"
elif Bulan == "08":
    nama_bulan = "Agustus"
elif Bulan == "09":
    nama_bulan = "September"
elif Bulan == "10":
    nama_bulan = "Oktober"
elif Bulan == "11":
    nama_bulan = "November"
elif Bulan == "12":
    nama_bulan = "Desember"
else:
    nama_bulan = "Bulan Tidak Valid"
# Output
print("Tanggal Lahir ASN:", Tanggal, nama_bulan, Tahun)

Masukkan NIP ASN: 19900122100010010
Tanggal Lahir ASN: 21 Januari 1993

```


- Uji 2

```

# Uji 2
# Input NTP ASN
NTP = input("Masukkan NTP ASN: ")
if not NTP.isdigit():
    raise ValueError("NTP Harus Berupa Angka")
# Mengambil tahun, bulan, dan tanggal lahir
Tahun = NTP[0:4]
Bulan = NTP[4:6]
Tanggal = NTP[6:8]
# Percabangan untuk nama bulan
if Bulan == "01":
    nama_bulan = "Januari"
elif Bulan == "02":
    nama_bulan = "Februari"
elif Bulan == "03":
    nama_bulan = "Maret"
elif Bulan == "04":
    nama_bulan = "April"
elif Bulan == "05":
    nama_bulan = "Mei"
elif Bulan == "06":
    nama_bulan = "Juni"
elif Bulan == "07":
    nama_bulan = "Juli"
elif Bulan == "08":
    nama_bulan = "Agustus"
elif Bulan == "09":
    nama_bulan = "September"
elif Bulan == "10":
    nama_bulan = "Oktober"
elif Bulan == "11":
    nama_bulan = "November"
elif Bulan == "12":
    nama_bulan = "Desember"
else:
    nama_bulan = "Bulan Tidak Valid"
# Output
print("Tanggal Lahir ASN:", Tanggal, nama_bulan, Tahun)

```

Masukkan NTP ASN: 200314092019010910
Tanggal Lahir ASN: 09 Bulan Tidak Valid 2006

- Uji 3

```

# Percabangan untuk nama bulan
if Bulan == "01":
    nama_bulan = "Januari"
elif Bulan == "02":
    nama_bulan = "Februari"
elif Bulan == "03":
    nama_bulan = "Maret"
elif Bulan == "04":
    nama_bulan = "April"
elif Bulan == "05":
    nama_bulan = "Mei"
elif Bulan == "06":
    nama_bulan = "Juni"
elif Bulan == "07":
    nama_bulan = "Juli"
elif Bulan == "08":
    nama_bulan = "Agustus"
elif Bulan == "09":
    nama_bulan = "September"
elif Bulan == "10":
    nama_bulan = "Oktober"
elif Bulan == "11":
    nama_bulan = "November"
elif Bulan == "12":
    nama_bulan = "Desember"
else:
    nama_bulan = "Bulan Tidak Valid"
# Output
print("Tanggal Lahir ASN:", Tanggal, nama_bulan, Tahun)

```

Masukkan NTP ASN: 200314092019010910
Tanggal Lahir ASN: 09 Bulan Tidak Valid 2006

5. Program 5: Klasifikasi Cluster Tiga Dimensi

```
import math
```

```
def hitung_jarak(x, y):
```

```
    return math.sqrt(
```

```
        (x[0] - y[0]) ** 2 + (x[1] - y[1]) ** 2 + (x[2] - y[2]) ** 2)
```

```
    a = (2, 1, 3)
```

```
    b = (1, -4, 6)
```

```
    c = (-2, 3, -2)
```

```
    try:
```

```
        x1 = float(input("Masukkan x1: "))
```

```
        x2 = float(input("Masukkan x2: "))
```

```
        x3 = float(input("Masukkan x3: "))
```

```
        titik_u = (x1, x2, x3)
```

```
        jarak_a = hitung_jarak(titik_u, a)
```

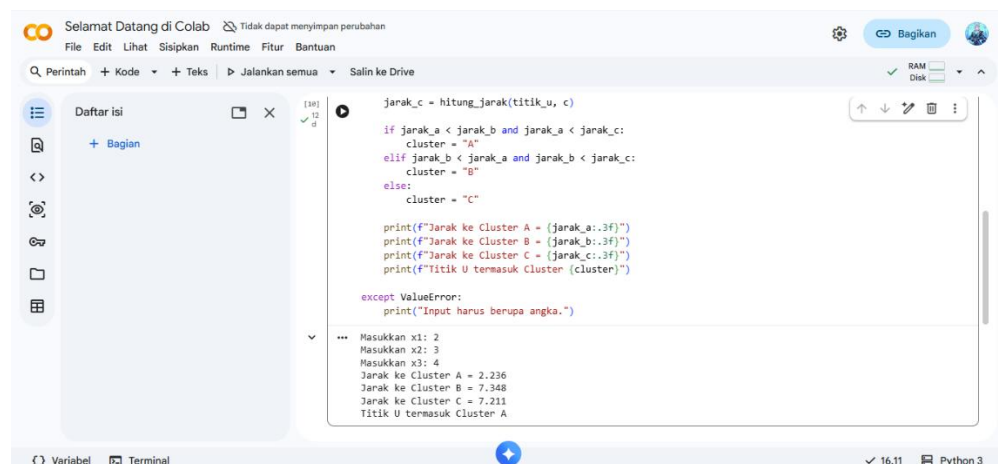
```

jarak_b = hitung_jarak(titik_u, b)
jarak_c = hitung_jarak(titik_u, c)
if jarak_a < jarak_b and jarak_a < jarak_c:
    cluster = "A"
elif jarak_b < jarak_a and jarak_b < jarak_c:
    cluster = "B"
else:
    cluster = "C"
print(f"Jarak ke Cluster A = {jarak_a:.3f}")
print(f"Jarak ke Cluster B = {jarak_b:.3f}")
print(f"Jarak ke Cluster C = {jarak_c:.3f}")
print(f"Titik U termasuk Cluster {cluster}")
except ValueError:
    print("Input harus berupa angka.")

```

Berikut adalah screenshot hasil skenario uji program 5 pada Python:

- Uji 1

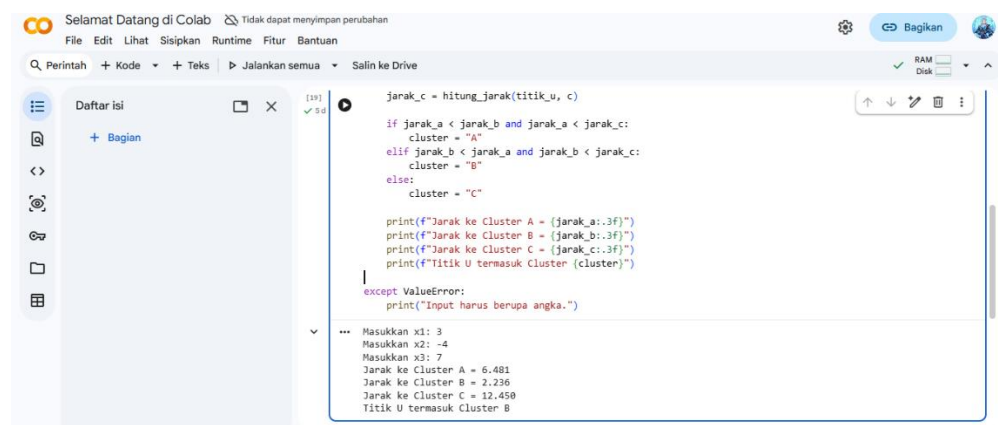


```

Selamat Datang di Colab
File Edit Lihat Sisipkan Runtime Fitur Bantuan
Perintah + Kode + Teks Jalankan semua Salin ke Drive
Daftar isi + Bagian
jarak_c = hitung_jarak(titik_u, c)
if jarak_a < jarak_b and jarak_a < jarak_c:
    cluster = "A"
elif jarak_b < jarak_a and jarak_b < jarak_c:
    cluster = "B"
else:
    cluster = "C"
print(f"Jarak ke Cluster A = {jarak_a:.3f}")
print(f"Jarak ke Cluster B = {jarak_b:.3f}")
print(f"Jarak ke Cluster C = {jarak_c:.3f}")
print(f"Titik U termasuk Cluster {cluster}")
except ValueError:
    print("Input harus berupa angka.")
Masukkan x1: 2
Masukkan x2: 3
Masukkan x3: 4
Jarak ke Cluster A = 2.236
Jarak ke Cluster B = 7.348
Jarak ke Cluster C = 7.211
Titik U termasuk Cluster A
16.11 Python 3

```

- Uji 2



```

Selamat Datang di Colab
File Edit Lihat Sisipkan Runtime Fitur Bantuan
Perintah + Kode + Teks Jalankan semua Salin ke Drive
Daftar isi + Bagian
jarak_c = hitung_jarak(titik_u, c)
if jarak_a < jarak_b and jarak_a < jarak_c:
    cluster = "A"
elif jarak_b < jarak_a and jarak_b < jarak_c:
    cluster = "B"
else:
    cluster = "C"
print(f"Jarak ke Cluster A = {jarak_a:.3f}")
print(f"Jarak ke Cluster B = {jarak_b:.3f}")
print(f"Jarak ke Cluster C = {jarak_c:.3f}")
print(f"Titik U termasuk Cluster {cluster}")
except ValueError:
    print("Input harus berupa angka.")
Masukkan x1: 3
Masukkan x2: -4
Masukkan x3: 7
Jarak ke Cluster A = 6.481
Jarak ke Cluster B = 2.236
Jarak ke Cluster C = 12.450
Titik U termasuk Cluster B

```

- Uji 3

```

x1 = float(input("Masukkan x1: "))
x2 = float(input("Masukkan x2: "))
x3 = float(input("Masukkan x3: "))

titik_u = (x1, x2, x3)

jarak_a = hitung_jarak(titik_u, a)
jarak_b = hitung_jarak(titik_u, b)
jarak_c = hitung_jarak(titik_u, c)

if jarak_a < jarak_b and jarak_a < jarak_c:
    cluster = "A"
elif jarak_b < jarak_a and jarak_b < jarak_c:
    cluster = "B"
else:
    cluster = "C"

print(f"Jarak ke Cluster A = {jarak_a:.3f}")
print(f"Jarak ke Cluster B = {jarak_b:.3f}")
print(f"Jarak ke Cluster C = {jarak_c:.3f}")
print(f"Titik U termasuk Cluster {cluster}")

except ValueError:
    print("Input harus berupa angka.")

```

Masukkan x1: 2
 Masukkan x2: b
 Input harus berupa angka.

6. Program 6: Interval Konfidensi Proporsi

```
import math
```

```
def cari_nilai_kritis(tingkat_signifikansi):
```

```
    if tingkat_signifikansi == 0.10:
```

```
        return 1.645
```

```
    elif tingkat_signifikansi == 0.05:
```

```
        return 1.96
```

```
    else:
```

```
        return None
```

```
def hitung_batas_kesalahan(proporsi_sampel, nilai_kritis, ukuran_sampel):
```

```
    return nilai_kritis * math.sqrt(
```

```
        (proporsi_sampel * (1 - proporsi_sampel)) / ukuran_sampel)
```

```
def hitung_interval_kepercayaan(proporsi_sampel, tingkat_signifikansi,
```

```
    ukuran_sampel):
```

```
    if proporsi_sampel < 0 or proporsi_sampel > 1:
```

```
        return "Error: Nilai proporsi harus antara 0 dan 1."
```

```
    nilai_kritis = cari_nilai_kritis(tingkat_signifikansi)
```

```
    if nilai_kritis is None:
```

```
        return "Error: Tingkat signifikansi hanya boleh 0.10 atau 0.05."
```

```
    batas_kesalahan = hitung_batas_kesalahan(
```

```
        proporsi_sampel,
```

```
        nilai_kritis,
```

```
        ukuran_sampel
```

```
)
```

```

    interval_minimum = proporsi_sampel - batas_kesalahan
    interval_maksimum = proporsi_sampel + batas_kesalahan
    return interval_minimum, interval_maksimum

#Program Utama
nilai_proporsi = float(input("Masukkan nilai proporsi sampel: "))
alpha_input = float(input("Masukkan tingkat signifikansi (0.10 atau 0.05): "))
jumlah_data = int(input("Masukkan ukuran sampel: "))
hasil_perhitungan = hitung_interval_kepercayaan(
    nilai_proporsi,
    alpha_input,
    jumlah_data
)
if isinstance(hasil_perhitungan, str):
    print(hasil_perhitungan)
else:
    batas_awal, batas_akhir = hasil_perhitungan
    print(f"Interval Kepercayaan = ({batas_awal:.4f}, {batas_akhir:.4f})")
)

```

Berikut adalah screenshot hasil skenario uji program 6 pada Python:

- Uji 1

```
[4]
✓ 16s
import math

def cari_nilai_kritis(tingkat_signifikansi):
    if tingkat_signifikansi == 0.10:
        return 1.645
    elif tingkat_signifikansi == 0.05:
        return 1.96
    else:
        return None

def hitung_batas_kesalahan(proporsi_sampel, nilai_kritis, ukuran_sampel):
    return nilai_kritis * math.sqrt(
        (proporsi_sampel * (1 - proporsi_sampel)) / ukuran_sampel
    )

def hitung_interval_kepercayaan(proporsi_sampel, tingkat_signifikansi, ukuran_sampel):
    if proporsi_sampel < 0 or proporsi_sampel > 1:
        return "Error: Nilai proporsi harus antara 0 dan 1."

    nilai_kritis = cari_nilai_kritis(tingkat_signifikansi)

    if nilai_kritis is None:
        return "Error: Tingkat signifikansi hanya boleh 0.10 atau 0.05."

    batas_kesalahan = hitung_batas_kesalahan(
        proporsi_sampel,
        nilai_kritis,
        ukuran_sampel
    )

    interval_minimum = proporsi_sampel - batas_kesalahan
    interval_maksimum = proporsi_sampel + batas_kesalahan

    return interval_minimum, interval_maksimum

# Program Utama
nilai_proporsi = float(input("Masukkan nilai proporsi sampel: "))
alpha_input = float(input("Masukkan tingkat signifikansi (0.10 atau 0.05): "))
jumlah_data = int(input("Masukkan ukuran sampel: "))

hasil_perhitungan = hitung_interval_kepercayaan(
    nilai_proporsi,
    alpha_input,
    jumlah_data
)

if isinstance(hasil_perhitungan, str):
    print(hasil_perhitungan)
else:
    batas_awal, batas_akhir = hasil_perhitungan
    print(
        f"Interval Kepercayaan = ({batas_awal:.4f}, {batas_akhir:.4f})"
    )

...
Masukkan nilai proporsi sampel: 0.9
Masukkan tingkat signifikansi (0.10 atau 0.05): 0.1
Masukkan ukuran sampel: 400
Interval Kepercayaan = (0.8753, 0.9247)
```

- Uji 2

```

[s]
✓ 10s
import math

def cari_nilai_kritis(tingkat_signifikansi):
    if tingkat_signifikansi == 0.10:
        return 1.645
    elif tingkat_signifikansi == 0.05:
        return 1.96
    else:
        return None

def hitung_batas_kesalahan(proporsi_sampel, nilai_kritis, ukuran_sampel):
    return nilai_kritis * math.sqrt(
        (proporsi_sampel * (1 - proporsi_sampel)) / ukuran_sampel
    )

def hitung_interval_kepercayaan(proporsi_sampel, tingkat_signifikansi, ukuran_sampel):
    if proporsi_sampel < 0 or proporsi_sampel > 1:
        return "Error: Nilai proporsi harus antara 0 dan 1."

    nilai_kritis = cari_nilai_kritis(tingkat_signifikansi)

    if nilai_kritis is None:
        return "Error: Tingkat signifikansi hanya boleh 0.10 atau 0.05."

    batas_kesalahan = hitung_batas_kesalahan(
        proporsi_sampel,
        nilai_kritis,
        ukuran_sampel
    )

    interval_minimum = proporsi_sampel - batas_kesalahan
    interval_maksimum = proporsi_sampel + batas_kesalahan

    return interval_minimum, interval_maksimum

# Program Utama
nilai_proporsi = float(input("Masukkan nilai proporsi sampel: "))
alpha_input = float(input("Masukkan tingkat signifikansi (0.10 atau 0.05): "))
jumlah_data = int(input("Masukkan ukuran sampel: "))

hasil_perhitungan = hitung_interval_kepercayaan(
    nilai_proporsi,
    alpha_input,
    jumlah_data
)

if isinstance(hasil_perhitungan, str):
    print(hasil_perhitungan)
else:
    batas_awal, batas_akhir = hasil_perhitungan
    print(
        f"Interval Kepercayaan = ({batas_awal:.4f}, {batas_akhir:.4f})"
    )

```

... Masukkan nilai proporsi sampel: 1.6
 Masukkan tingkat signifikansi (0.10 atau 0.05): 0.05
 Masukkan ukuran sampel: 3000
 Error: Nilai proporsi harus antara 0 dan 1.

- Uji 3

```

(8) 15s
import math

def cari_nilai_kritis(tingkat_signifikansi):
    if tingkat_signifikansi == 0.10:
        return 1.645
    elif tingkat_signifikansi == 0.05:
        return 1.96
    else:
        return None

def hitung_batas_kesalahan(proporsi_sampel, nilai_kritis, ukuran_sampel):
    return nilai_kritis * math.sqrt(
        (proporsi_sampel * (1 - proporsi_sampel)) / ukuran_sampel
    )

def hitung_interval_kepercayaan(proporsi_sampel, tingkat_signifikansi, ukuran_sampel):

    if proporsi_sampel < 0 or proporsi_sampel > 1:
        return "Error: Nilai proporsi harus antara 0 dan 1."

    nilai_kritis = cari_nilai_kritis(tingkat_signifikansi)

    if nilai_kritis is None:
        return "Error: Tingkat signifikansi hanya boleh 0.10 atau 0.05."

    batas_kesalahan = hitung_batas_kesalahan(
        proporsi_sampel,
        nilai_kritis,
        ukuran_sampel
    )

    interval_minimum = proporsi_sampel - batas_kesalahan
    interval_maksimum = proporsi_sampel + batas_kesalahan

    return interval_minimum, interval_maksimum

# Program Utama
nilai_proporsi = float(input("Masukkan nilai proporsi sampel: "))
alpha_input = float(input("Masukkan tingkat signifikansi (0.10 atau 0.05): "))
jumlah_data = int(input("Masukkan ukuran sampel: "))

hasil_perhitungan = hitung_interval_kepercayaan(
    nilai_proporsi,
    alpha_input,
    jumlah_data
)

if isinstance(hasil_perhitungan, str):
    print(hasil_perhitungan)
else:
    batas_awal, batas_akhir = hasil_perhitungan
    print(
        f"Interval Kepercayaan = ({batas_awal:.4f}, {batas_akhir:.4f})"
    )

... Masukkan nilai proporsi sampel: 0.7
Masukkan tingkat signifikansi (0.10 atau 0.05): 0.03
Masukkan ukuran sampel: 500
Error: Tingkat signifikansi hanya boleh 0.10 atau 0.05.

```

4.2 Source Code R

1. Program 1: Menghitung Jumlah Kata dan Jumlah Kalimat

#Program 1 - Menghitung jumlah kalimat dan jumlah kata dari sebuah teks

#Menerima input teks dari pengguna dan menyimpannya ke variabel

teks <- readline("Masukkan teks: ")

#Menerima input teks dari pengguna dan menyimpannya ke variabel

Teks <- trimws(teks)

#Menghapus semua karakter selain titik (.), lalu menghitung jumlah titik yang tersisa sebagai jumlah kalimat

```
jumlah_kalimat <- nchar(gsub("[^.]", "", teks))
```

#Memisahkan teks menjadi kata-kata berdasarkan spasi, kemudian menghitung jumlah katanya

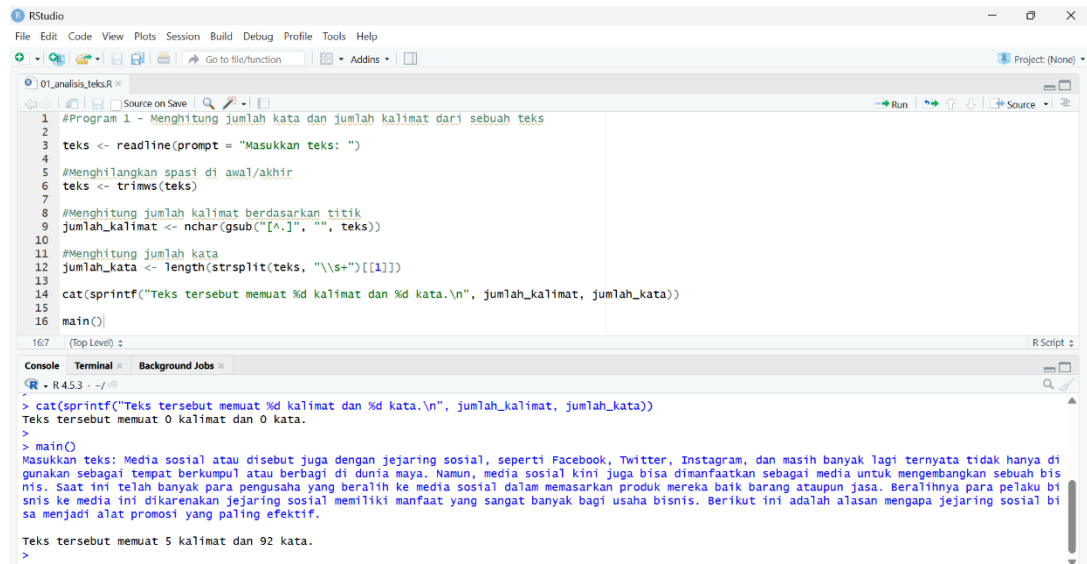
```
jumlah_kata <- length(strsplit(teks, "\\s+"))[[1]]
```

#Menampilkan hasil jumlah kalimat dan jumlah kata

```
cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))
```

Berikut adalah screenshot hasil skenario uji program 1 pada R:

- Uji 1



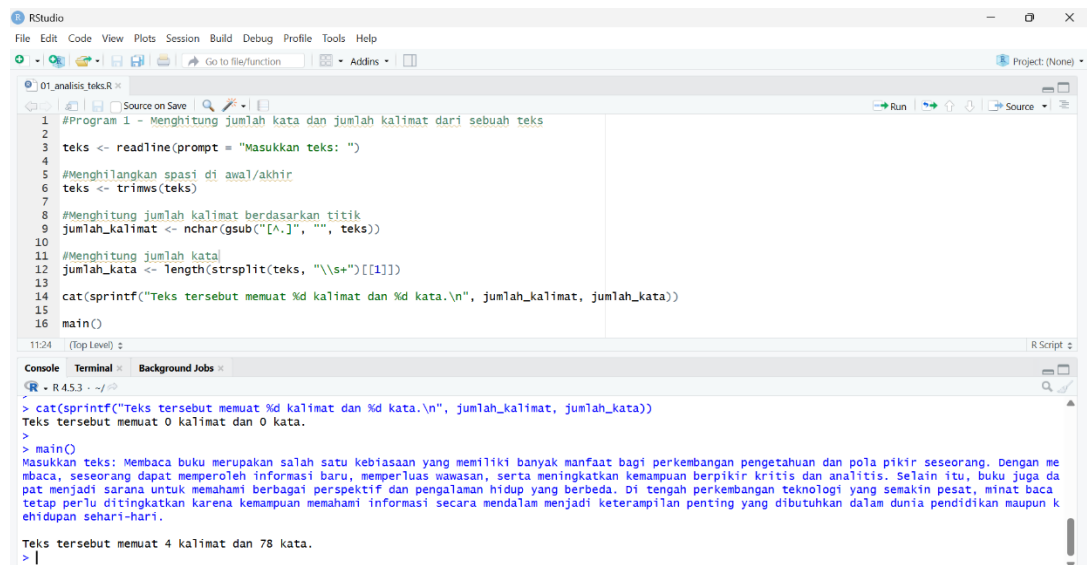
```
#Program 1 - Menghitung jumlah kata dan jumlah kalimat dari sebuah teks
1
2
3 teks <- readline(prompt = "Masukkan teks: ")
4
5 #Menghilangkan spasi di awal/akhir
6 teks <- trimws(teks)
7
8 #Menghitung jumlah kalimat berdasarkan titik
9 jumlah_kalimat <- nchar(gsub("[^.]", "", teks))
10
11 #Menghitung jumlah kata
12 jumlah_kata <- length(strsplit(teks, "\\s+"))[[1]]
13
14 cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))
15
16 main()

16:7 (Top Level) z

Console Terminal Background Jobs
R - R453 - ~/...
> cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))
Teks tersebut memuat 0 kalimat dan 0 kata.
>
> main()
Masukkan teks: Media sosial atau disebut juga dengan jejaring sosial, seperti Facebook, Twitter, Instagram, dan masih banyak lagi ternyata tidak hanya di gunakan sebagai tempat berkumpul atau berbagi di dunia maya. Namun, media sosial kini juga bisa dimanfaatkan sebagai media untuk mengembangkan sebuah bis nis. Saat ini telah banyak para pengusaha yang beralih ke media sosial dalam memasarkan produk mereka baik barang ataupun jasa. Beralihnya para pelaku bi snis ke media ini dikarenakan jejaring sosial memiliki manfaat yang sangat banyak bagi usaha bisnis. Berikut ini adalah alasan mengapa jejaring sosial bi sa menjadi alat promosi yang paling efektif.

Teks tersebut memuat 5 kalimat dan 92 kata.
>
```

- Uji 2



```
#Program 1 - Menghitung jumlah kata dan jumlah kalimat dari sebuah teks
1
2
3 teks <- readline(prompt = "Masukkan teks: ")
4
5 #Menghilangkan spasi di awal/akhir
6 teks <- trimws(teks)
7
8 #Menghitung jumlah kalimat berdasarkan titik
9 jumlah_kalimat <- nchar(gsub("[^.]", "", teks))
10
11 #Menghitung jumlah kata
12 jumlah_kata <- length(strsplit(teks, "\\s+"))[[1]]
13
14 cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))
15
16 main()

11:24 (Top Level) z

Console Terminal Background Jobs
R - R453 - ~/...
> cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))
Teks tersebut memuat 0 kalimat dan 0 kata.
>
> main()
Masukkan teks: Membaca buku merupakan salah satu kebiasaan yang memiliki banyak manfaat bagi perkembangan pengetahuan dan pola pikir seseorang. Dengan me mbaca, seseorang dapat memperoleh informasi baru, memperluas wawasan, serta meningkatkan kemampuan berpikir kritis dan analitis. Selain itu, buku juga da pat menjadi sarana untuk memahami berbagai perspektif dan pengalaman hidup yang berbeda. Di tengah perkembangan teknologi yang semakin pesat, minat baca tetap perlu ditingkatkan karena kemampuan memahami informasi secara mendalam menjadi keterampilan penting yang dibutuhkan dalam dunia pendidikan maupun k ehidupan sehari-hari.

Teks tersebut memuat 4 kalimat dan 78 kata.
> |
```


- Uji 3

```

1 #Program 1 - Menghitung jumlah kata dan jumlah kalimat dari sebuah teks
2
3 teks <- readline(prompt = "Masukkan teks: ")
4
5 #Menghilangkan spasi di awal/akhir
6 teks <- trimws(teks)
7
8 #Menghitung jumlah kalimat berdasarkan titik
9 jumlah_kalimat <- nchar(gsub("[.]", "", teks))
10
11 #Menghitung jumlah kata
12 jumlah_kata <- length(strsplit(teks, "\\s+")[1])
13
14 cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))
15
16 main()

```

Console output:

```

> cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))
Teks tersebut memuat 0 kalimat dan 0 kata.
>
> main()
Masukkan teks: Pendidikan memiliki peran yang sangat penting dalam membentuk kualitas sumber daya manusia dan kemajuan suatu bangsa. Melalui pendidikan, seseorang tidak hanya memperoleh pengetahuan akademik, tetapi juga nilai-nilai karakter seperti tanggung jawab, disiplin, dan kerja sama. Pendidikan yang berkualitas dapat membuka peluang yang lebih luas bagi individu untuk mengembangkan potensi diri dan mencapai tujuan hidupnya. Oleh karena itu, akses terhadap pendidikan yang merata dan berkualitas perlu terus didukung agar setiap orang memiliki kesempatan yang sama untuk berkembang dan berkontribusi bagi masyarakat.
Teks tersebut memuat 4 kalimat dan 80 kata.
>

```

2. Program 2: Membuat String Kalimat

#Membuat objek string

K1 <- "Saya tak 'kan menyerah."

K2 <- 'Ia berkata, "Aku menyayangimu"'

K3 <- "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""

K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."

#Menampilkan hasil

print(K1)

print(K2)

print(K3)

print(K4)

Berikut adalah screenshot hasil skenario uji program 2 pada R:

```

1 #Membuat objek string
2 K1<- "Saya tak 'kan menyerah."
3 K2<- 'Ia berkata, "Aku menyayangimu"'
4 K3<- "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
5 K4<- "Surat keputusan itu bernomor 62/UN.34/19/2023."
6 #Menampilkan hasil
7 print(K1)
8 print(K2)
9 print(K3)
10 print(K4)

```

Environment pane:

Object	Value
K1	"Saya tak 'kan menyerah."
K2	'Ia berkata, "Aku menyayangimu"'
K3	"\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4	"Surat keputusan itu bernomor 62/UN.34/19/2023."

Console output:

```

> K4<- "Surat keputusan itu bernomor 62/UN.34/19/2023."
> #Menampilkan hasil
> print(K1)
[1] "Saya tak 'kan menyerah."
> print(K2)
[1] "Ia berkata, \"Aku menyayangimu\""
> print(K3)
[1] "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
> print(K4)
[1] "Surat keputusan itu bernomor 62/UN.34/19/2023."
>

```

3. Program 3: Menentukan Akar Persamaan Kuadrat

```
cat("==== PROGRAM MENCARI AKAR PERSAMAAN KUADRAT ====\\n")
#Menampilkan judul program pada layar. \\n digunakan untuk pindah ke baris baru.
a <- 1
b <- -5
c <- 6
diskriminan <- b^2 - 4*a*c
if (a == 0) {
  cat("Error: Nilai a tidak boleh 0.\\n")
} else if (diskriminan < 0) {
  cat("Persamaan memiliki akar imajiner.\\n")
} else {
  x1 <- (-b + sqrt(diskriminan)) / (2*a)
  x2 <- (-b - sqrt(diskriminan)) / (2*a)
  cat("Koefisien a =", a, "\\n")
  cat("Koefisien b =", b, "\\n")
  cat("Koefisien c =", c, "\\n")
  cat("Diskriminan =", diskriminan, "\\n")
  cat("Akar pertama =", x1, "\\n")
  cat("Akar kedua =", x2, "\\n")
}
```

Berikut adalah screenshot hasil skenario uji program 3 pada R:

- Uji 1

```
cat("==== PROGRAM MENCARI AKAR PERSAMAAN KUADRAT ====\\n")

## ===== PROGRAM MENCARI AKAR PERSAMAAN KUADRAT =====

a <- 1
b <- -5
c <- 6

diskriminan <- b^2 - 4*a*c

if (a == 0) {

  cat("Error: Nilai a tidak boleh 0.\\n")

} else if (diskriminan < 0) {

  cat("Persamaan memiliki akar imajiner.\\n")

} else {

  x1 <- (-b + sqrt(diskriminan)) / (2*a)
  x2 <- (-b - sqrt(diskriminan)) / (2*a)

  cat("Koefisien a =", a, "\\n")
  cat("Koefisien b =", b, "\\n")
  cat("Koefisien c =", c, "\\n")
  cat("Diskriminan =", diskriminan, "\\n")
  cat("Akar pertama =", x1, "\\n")
  cat("Akar kedua =", x2, "\\n")

}

## Koefisien a = 1
## Koefisien b = -5
## Koefisien c = 6
## Diskriminan = 1
## Akar pertama = 3
## Akar kedua = 2
```

- Uji 2

```
a<-2
b<--2
c<-4
diskriminan <- b^2 - 4*a*c

if (a == 0) {

  cat("Error: Nilai a tidak boleh 0.\\n")

} else if (diskriminan < 0) {

  cat("Persamaan memiliki akar imajiner.\\n")

} else {

  x1 <- (-b + sqrt(diskriminan)) / (2*a)
  x2 <- (-b - sqrt(diskriminan)) / (2*a)

  cat("Koefisien a =", a, "\\n")
  cat("Koefisien b =", b, "\\n")
  cat("Koefisien c =", c, "\\n")
  cat("Diskriminan =", diskriminan, "\\n")
  cat("Akar pertama =", x1, "\\n")
  cat("Akar kedua =", x2, "\\n")

}

## Persamaan memiliki akar imajiner.
```

- Uji 3

```

a<-0
b<-3
c<--2
diskriminan <- b^2 - 4*a*c

if (a == 0) {

  cat("Error: Nilai a tidak boleh 0.\n")

} else if (diskriminan < 0) {

  cat("Persamaan memiliki akar imajiner.\n")

} else {

  x1 <- (-b + sqrt(diskriminan)) / (2*a)
  x2 <- (-b - sqrt(diskriminan)) / (2*a)

  cat("Koefisien a =", a, "\n")
  cat("Koefisien b =", b, "\n")
  cat("Koefisien c =", c, "\n")
  cat("Diskriminan =", diskriminan, "\n")
  cat("Akar pertama =", x1, "\n")
  cat("Akar kedua =", x2, "\n")
}

## Error: Nilai a tidak boleh 0.

```

4. Program 4: Menampilkan Tanggal Lahir ASN Berdasarkan NIP

```

#Input NIP ASN
NIP <- readline(prompt = "Masukkan NIP ASN: ")
if (is.na(as.numeric(NIP))) {
  stop("Error: NIP Harus Berupa Angka")
}

#Mengambil tahun, bulan, dan tanggal
Tahun <- substr(NIP, 1, 4)
Bulan <- substr(NIP, 5, 6)
Tanggal <- substr(NIP, 7, 8)

#Percabangan nama bulan
if (Bulan == "01") {
  namaBulan <- "Januari"
} else if (Bulan == "02") {
  namaBulan <- "Februari"
} else if (Bulan == "03") {
  namaBulan <- "Maret"
} else if (Bulan == "04") {

```

```
namaBulan <- "April"
} else if (Bulan == "05") {
  namaBulan <- "Mei"
} else if (Bulan == "06") {
  namaBulan <- "Juni"
} else if (Bulan == "07") {
  namaBulan <- "Juli"
} else if (Bulan == "08") {
  namaBulan <- "Agustus"
} else if (Bulan == "09") {
  namaBulan <- "September"
} else if (Bulan == "10") {
  namaBulan <- "Oktober"
} else if (Bulan == "11") {
  namaBulan <- "November"
} else if (Bulan == "12") {
  namaBulan <- "Desember"
} else {
  stop("Bulan tidak valid")
}
```

#Output

```
print(paste("Tanggal Lahir ASN :", Tanggal, namaBulan, Tahun))
```

Berikut adalah screenshot hasil skenario uji program 4 pada R:

- Uji 1

```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Source: t_uji_1_sosial_4(fix_banget).R
27 namaBulan <- JULI
28 } else if (bulan == "08") {
29   namaBulan <- "Agustus"
30 } else if (bulan == "09") {
31   namaBulan <- "September"
32 } else if (bulan == "10") {
33   namaBulan <- "Oktober"
34 } else if (bulan == "11") {
35   namaBulan <- "November"
36 } else if (bulan == "12") {
37   namaBulan <- "Desember"
38 } else {
39   namaBulan <- "Bulan Tidak valid"
40 }
41
42 # Output
43 print(paste("Tanggal Lahir ASN :", Tanggal, namaBulan, Tahun))
44
961 (Top Level) z

Console Terminal Background Jobs
R - R4.5.1 - ~/R
+ namaBulan <- "10" {
+   namaBulan <- "Oktober"
+ } else if (bulan == "11") {
+   namaBulan <- "November"
+ } else if (bulan == "12") {
+   namaBulan <- "Desember"
+ } else {
+   namaBulan <- "Bulan Tidak valid"
+ }
+ # Output
+ print(paste("Tanggal Lahir ASN :", Tanggal, namaBulan, Tahun))
[1] "Tanggal Lahir ASN : 21 Januari 1993"
>
>
```

Environment	History	Connections	Tutorial
JK_KelasB	chr [1:5] "L" "p" "p" "L" "p"		
JK_KelasB_Baru	Factor w/ 2 levels "L","p": 1 2 2 1 2		
logika1	"banyak"		
logika3	FALSE		
logika12	FALSE		
mhs	chr [1:3] "budi" "bangbang" "cahyo"		
n	30		
namaBulan	"Januari"		
NIP	"ptpastibangkrrut2019031010"		
p	num [1:5] 8 8 8 8 8		
PI	fnt [1:12] 1 1 1 2 2 2 1 1 2 ...		
pola	chr [1:12] "pola" "pola" "pola" "polos"...		
populasi	fnt [1:30] 1 2 3 4 5 6 7 8 9 10 ...		
q	fnt [1:5] 6 7 8 9 10		
r	num [1:5] 14 15 16 17 18		
r_tabel	0.361006907733233		
s	num [1:5] -2 -1 0 1 2		
sample_acak	fnt [1:15] 18 29 25 5 16 12 20 14 21 23...		
status	Factor w/ 2 levels "Lulus","Tidak lulus"...		
t	num [1:5] 48 56 64 72 80		
t_tabel	2.04840714179524		
Tahun	"2006"		
Tanggal	"09"		

- Uji 2

```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Source: t_uji_2_sosial_4(fix_banget).R
72 namaBulan <- JULI
73 } else if (bulan == "08") {
74   namaBulan <- "Agustus"
75 } else if (bulan == "09") {
76   namaBulan <- "September"
77 } else if (bulan == "10") {
78   namaBulan <- "Oktober"
79 } else if (bulan == "11") {
80   namaBulan <- "November"
81 } else if (bulan == "12") {
82   namaBulan <- "Desember"
83 } else {
84   stop("Bulan tidak valid")
85 }
86
87 # Output
88 print(paste("Tanggal Lahir ASN :", Tanggal, namaBulan, Tahun))
89
90
961 (Top Level) z

Console Terminal Background Jobs
R - R4.5.1 - ~/R
+ namaBulan <- "September"
+ } else if (bulan == "10") {
+   namaBulan <- "Oktober"
+ } else if (bulan == "11") {
+   namaBulan <- "November"
+ } else if (bulan == "12") {
+   namaBulan <- "Desember"
+ } else {
+   stop("Bulan tidak valid")
+ }
+ # Output
+ print(paste("Tanggal Lahir ASN :", Tanggal, namaBulan, Tahun))
Error: Bulan tidak valid
>
>
```

Environment	History	Connections	Tutorial
JK_KelasB	chr [1:5] "L" "p" "p" "L" "p"		
JK_KelasB_Baru	Factor w/ 2 levels "L","p": 1 2 2 1 2		
logika1	"banyak"		
logika3	FALSE		
logika12	FALSE		
mhs	chr [1:3] "budi" "bangbang" "cahyo"		
n	30		
namaBulan	"Januari"		
NIP	"ptpastibangkrrut2019031010"		
p	num [1:5] 8 8 8 8 8		
PI	fnt [1:12] 1 1 1 2 2 2 1 1 2 ...		
pola	chr [1:12] "pola" "pola" "pola" "polos"...		
populasi	fnt [1:30] 1 2 3 4 5 6 7 8 9 10 ...		
q	fnt [1:5] 6 7 8 9 10		
r	num [1:5] 14 15 16 17 18		
r_tabel	0.361006907733233		
s	num [1:5] -2 -1 0 1 2		
sample_acak	fnt [1:15] 18 29 25 5 16 12 20 14 21 23...		
status	Factor w/ 2 levels "Lulus","Tidak lulus"...		
t	num [1:5] 48 56 64 72 80		
t_tabel	2.04840714179524		
Tahun	"2006"		
Tanggal	"09"		

- Uji 3

```
RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Source: t_uji_3_sosial_4(fix_banget).R
91 #Uji Ketiga
92 # Input NIP ASN
93 NIP <- readline(prompt = "Masukkan NIP ASN: ")
94 if (is.na(as.numeric(NIP))) {
95   stop("Error: NIP Harus Berupa Angka")
96 }
97
98 # Mengambil tahun, bulan, dan tanggal
99 Tahun <- substr(NIP, 1, 4)
100 Bulan <- substr(NIP, 5, 6)
101 Tanggal <- substr(NIP, 7, 8)
102
103 # percabangan nama bulan
104 if (bulan == "01") {
105   namaBulan <- "Januari"
106 } else if (bulan == "02") {
107   namaBulan <- "Februari"
108 } else if (bulan == "03") {
109   namaBulan <- "Maret"
110 } else {
111   stop("Error: Bulan tidak valid")
112 }
113
114 # Output
115 print(paste("Tanggal Lahir ASN :", Tanggal, namaBulan, Tahun))
116
961 (Top Level) z

Console Terminal Background Jobs
R - R4.5.1 - ~/R
+ #Uji Ketiga
+ # Input NIP ASN
+ NIP <- readline(prompt = "Masukkan NIP ASN: ")
+ Masukkan NIP ASN: ptpastibangkrrut2019031010
+ stop("Error: NIP Harus Berupa Angka")
Error: Error: NIP Harus Berupa Angka
>
>
```

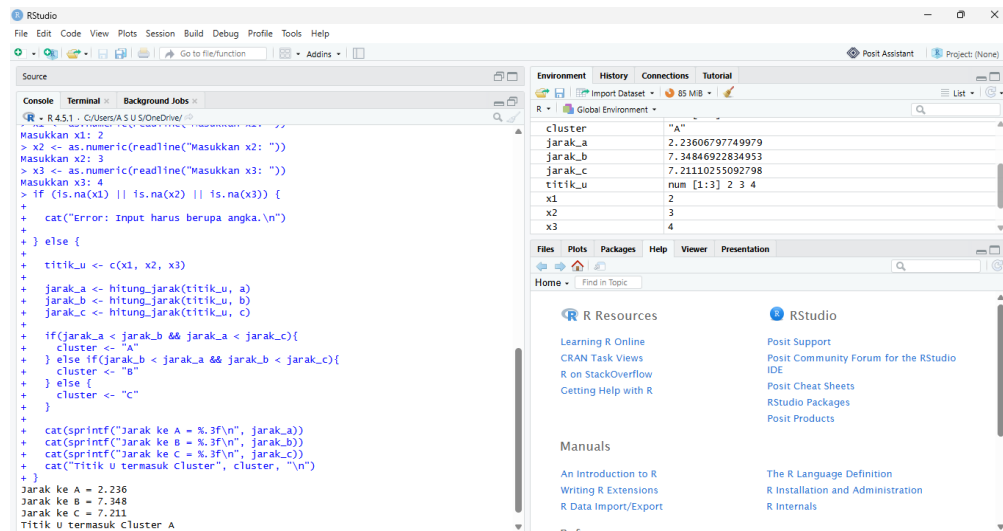
Environment	History	Connections	Tutorial
JK_KelasB	chr [1:5] "L" "p" "p" "L" "p"		
JK_KelasB_Baru	Factor w/ 2 levels "L","p": 1 2 2 1 2		
logika1	"banyak"		
logika3	FALSE		
logika12	FALSE		
mhs	chr [1:3] "budi" "bangbang" "cahyo"		
n	30		
namaBulan	"Januari"		
NIP	"ptpastibangkrrut2019031010"		
p	num [1:5] 8 8 8 8 8		
PI	fnt [1:12] 1 1 1 2 2 2 1 1 2 ...		
pola	chr [1:12] "pola" "pola" "pola" "polos"...		
populasi	fnt [1:30] 1 2 3 4 5 6 7 8 9 10 ...		
q	fnt [1:5] 6 7 8 9 10		
r	num [1:5] 14 15 16 17 18		
r_tabel	0.361006907733233		
s	num [1:5] -2 -1 0 1 2		
sample_acak	fnt [1:15] 18 29 25 5 16 12 20 14 21 23...		
status	Factor w/ 2 levels "Lulus","Tidak lulus"...		
t	num [1:5] 48 56 64 72 80		
t_tabel	2.04840714179524		
Tahun	"2006"		
Tanggal	"09"		

5. Program 5: Klasifikasi Cluster Tiga Dimensi

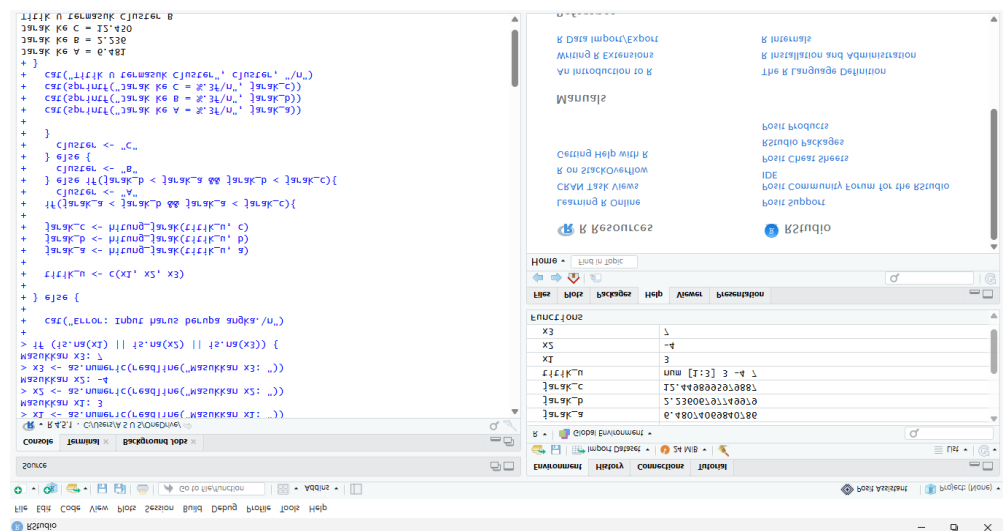
```
a <- c(2, 1, 3)
b <- c(1, -4, 6)
c <- c(-2, 3, -2)
hitung_jarak <- function(x,y) {
  sqrt( (x[1] - y[1])^2 + (x[2] - y[2])^2 + (x[3] - y[3])^2 )
}
x1 <- as.numeric(readline("Masukkan x1: "))
x2 <- as.numeric(readline("Masukkan x2: "))
x3 <- as.numeric(readline("Masukkan x3: "))
if (is.na(x1) || is.na(x2) || is.na(x3)) {
  cat("Error: Input harus berupa angka.\n")
} else {
  titik_u <- c(x1, x2, x3)
  jarak_a <- hitung_jarak(titik_u, a)
  jarak_b <- hitung_jarak(titik_u, b)
  jarak_c <- hitung_jarak(titik_u, c)
  if(jarak_a < jarak_b && jarak_a < jarak_c){
    cluster <- "A"
  } else if(jarak_b < jarak_a && jarak_b < jarak_c){
    cluster <- "B"
  } else {
    cluster <- "C"
  }
  cat(sprintf("Jarak ke A = %.3f\n", jarak_a))
  cat(sprintf("Jarak ke B = %.3f\n", jarak_b))
  cat(sprintf("Jarak ke C = %.3f\n", jarak_c))
  cat("Titik U termasuk Cluster", cluster, "\n")
}
```

Berikut adalah screenshot hasil skenario uji program 5 pada R:

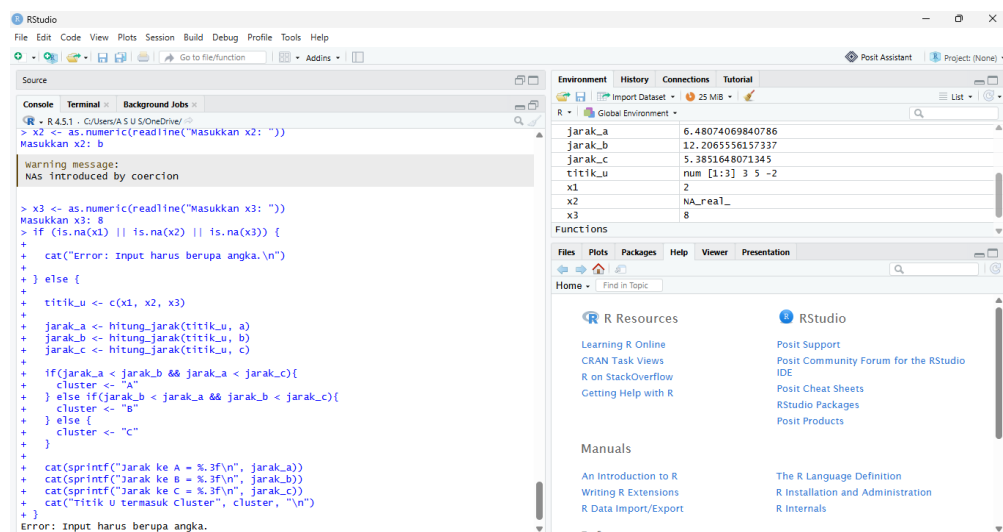
- Uji 1



- Uji 2



- Uji 3



6. Program 6: Interval Konfidensi Proporsi

#Menghitung interval konfidensi proporsi

```
hitung_interval_konfidensi <- function(phat, n, alpha) {  
  #Validasi input  
  if (is.na(phat) || is.na(n) || is.na(alpha)) {  
    cat("ERROR: Input harus berupa  
    return(NULL)  
  }  
  if (phat < 0 || phat > 1) {  
    cat("ERROR: Proporsi sampel (phat) harus antara 0 dan 1.\n")  
    return(NULL)  
  }  
  if (n <= 0 || n != round(n)) {  
    cat("ERROR: Ukuran sampel (n) harus bilangan bulat positif.\n")  
    return(NULL)  
  }  
  if (alpha == 0.10) {  
    z <- 1.645  
  } else if (alpha == 0.05) {  
    z <- 1.96  
  } else {  
    cat("ERROR: Alpha hanya boleh 0.05 atau 0.10.\n")  
    return(NULL)  
  }  
  #Perhitungan  
  margin_error <- z * sqrt(phat * (1 - phat) / n)  
  lower <- phat - margin_error  
  upper <- phat + margin_error  
  return(list(  
    phat = phat,  
    n = n,  
    alpha = alpha,  
    confidence_level = (1 - alpha) * 100,  
    z = z,
```

```

        lower_bound = lower,
        upper_bound = upper
    ))
}

#Program utama

phat <- as.numeric(readline("Masukkan proporsi sampel (0 - 1): "))
n <- as.numeric(readline("Masukkan ukuran sampel (n): "))
alpha <- as.numeric(readline("Masukkan alpha (0.05 atau 0.10): "))
hasil <- hitung_interval_konfidensi(phat, n, alpha)
if (!is.null(hasil)) {
    cat(sprintf("Interval Konfidensi    : (%.4f, %.4f)\n",
        hasil$lower_bound, hasil$upper_bound))
} else {
    cat("\nProgram dihentikan karena input tidak valid.\n")
}

```

Berikut adalah screenshot hasil skenario uji program 5 pada R:

- Uji 1

```

RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help

Source
Console Terminal Background Jobs

> #Program 6 - Interval proporsi
> #Menghitung interval konfidensi proporsi
> hitung_interval_konfidensi <- function(phat, n, alpha) {
+   #Validasi input
+   if (is.na(phat) || is.na(n) || is.na(alpha)) {
+     cat("ERROR: Input harus berupa angka.\n")
+     return(NULL)
+   }
+   if (phat < 0 || phat > 1) {
+     cat("ERROR: Proporsi sampel (phat) harus antara 0 dan 1.\n")
+     return(NULL)
+   }
+   if (n <= 0 || n != round(n)) {
+     cat("ERROR: Ukuran sampel (n) harus bilangan bulat positif.\n")
+     return(NULL)
+   }
+   if (alpha == 0.10) {
+     z <- 1.645
+   } else if (alpha == 0.05) {
+     z <- 1.96
+   } else {
+     cat("ERROR: Alpha hanya boleh 0.05 atau 0.10.\n")
+     return(NULL)
+   }
+   #Perhitungan
+   margin_error <- z * sqrt(phat * (1 - phat) / n)
+   lower <- phat - margin_error
+   upper <- phat + margin_error
+   return(list(
+     phat = phat,
+     n = n,
+     alpha = alpha,
+     confidence_level = (1 - alpha) * 100,
+     z = z,
+     lower_bound = lower,
+     upper_bound = upper
+   ))
+ }
> #Program utama
> phat <- as.numeric(readline("Masukkan proporsi sampel (0 - 1): "))
Masukkan proporsi sampel (0 - 1): 0.9
> n <- as.numeric(readline("Masukkan ukuran sampel (n): "))
Masukkan ukuran sampel (n): 400
> alpha <- as.numeric(readline("Masukkan alpha (0.05 atau 0.10): "))
Masukkan alpha (0.05 atau 0.10): 0.10
> hasil <- hitung_interval_konfidensi(phat, n, alpha)
> if (!is.null(hasil)) {
+   cat(sprintf("Interval Konfidensi    : (%.4f, %.4f)\n",
+     hasil$lower_bound, hasil$upper_bound))
+ } else {
+   cat("\nProgram dihentikan karena input tidak valid.\n")
+ }
Interval Konfidensi    : (0.8753, 0.9247)

```

- Uji 2

```
> #Program utama
> phat <- as.numeric(readline("Masukkan proporsi sampel (0 - 1): "))
Masukkan proporsi sampel (0 - 1): 1.6
> n <- as.numeric(readline("Masukkan ukuran sampel (n): "))
Masukkan ukuran sampel (n): 3000
> alpha <- as.numeric(readline("Masukkan alpha (0.05 atau 0.10): "))
Masukkan alpha (0.05 atau 0.10): 0.05
> hasil <- hitung_interval_konfidensi(phat, n, alpha)
ERROR: Proporsi sampel (phat) harus antara 0 dan 1.
> if (!is.null(hasil)) {
+   cat(sprintf("Interval Konfidensi      : (%.4f, %.4f)\n",
+             hasil$lower_bound, hasil$upper_bound))
+ } else {
+   cat("\nProgram dihentikan karena input tidak valid.\n")
+ }

Program dihentikan karena input tidak valid.
```

- Uji 3

```
> #Program utama
> phat <- as.numeric(readline("Masukkan proporsi sampel (0 - 1): "))
Masukkan proporsi sampel (0 - 1): 0.7
> n <- as.numeric(readline("Masukkan ukuran sampel (n): "))
Masukkan ukuran sampel (n): 500
> alpha <- as.numeric(readline("Masukkan alpha (0.05 atau 0.10): "))
Masukkan alpha (0.05 atau 0.10): 0.03
> hasil <- hitung_interval_konfidensi(phat, n, alpha)
ERROR: Alpha hanya boleh 0.05 atau 0.10.
> if (!is.null(hasil)) {
+   cat(sprintf("Interval Konfidensi      : (%.4f, %.4f)\n",
+             hasil$lower_bound, hasil$upper_bound))
+ } else {
+   cat("\nProgram dihentikan karena input tidak valid.\n")
+ }

Program dihentikan karena input tidak valid.
> |
```

BAB V

PENJELASAN LINE-BY-LINE CODE

5.1 Program 1: Menghitung Jumlah Kata dan Jumlah Kalimat

1. Kode program pada Python

Kode Program Python	Penjelasan
<code>teks = input("Masukkan teks: ").strip()</code>	Menerima input teks dari pengguna dan menghapus spasi di awal maupun akhir teks
<code>jumlah_kalimat = teks.count(".")</code>	Menghitung jumlah tanda titik (.) dalam teks sebagai jumlah kalimat
<code>jumlah_kata = len(teks.split())</code>	Memisahkan teks menjadi kata-kata berdasarkan spasi, lalu menghitung jumlah katanya
<code>print(f"Teks tersebut memuat {jumlah_kalimat} kalimat dan {jumlah_kata} kata.")</code>	Menampilkan hasil jumlah kalimat dan jumlah kata

2. Kode Program pada R

Kode Program R	Penjelasan
<code>teks <- readline("Masukkan teks: ")</code>	Menerima input teks dari pengguna dan menyimpannya ke variabel
<code>teks <- trimws(teks)</code>	Menghapus spasi berlebih di awal/akhir.
<code>jumlah_kalimat <- nchar(gsub("[^.]", "", teks))</code>	Menghapus semua karakter selain titik (.), lalu menghitung jumlah titik yang tersisa sebagai jumlah kalimat
<code>jumlah_kata <- length(strsplit(teks, "\\s+"))[[1]]</code>	Memisahkan teks menjadi kata-kata berdasarkan spasi, kemudian menghitung jumlah katanya
<code>cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))</code>	Menampilkan hasil jumlah kalimat dan jumlah kata

5.2 Program 2: Membuat String Kalimat

1. Kode program pada Python

Kode Program Python	Penjelasan
<code>teks = input("Masukkan teks: ").strip()</code>	Menerima input teks dari pengguna dan menghapus spasi di awal maupun akhir teks
<code>jumlah_kalimat = teks.count(".")</code>	Menghitung jumlah tanda titik (.) dalam teks sebagai jumlah kalimat
<code>jumlah_kata = len(teks.split())</code>	Memisahkan teks menjadi kata-kata berdasarkan spasi, lalu menghitung jumlah katanya
<code>print(f"Teks tersebut memuat {jumlah_kalimat} kalimat dan {jumlah_kata} kata.")</code>	Menampilkan hasil jumlah kalimat dan jumlah kata

2. Kode Program pada R

Kode Program R	Penjelasan
<code>K1 <- "Saya tak 'kan menyerah."</code> <code>K2 <- 'Ia berkata, "Aku menyayangimu"'</code> <code>K3 <- Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""</code> <code>K4 <- "Surat keputusan itu bernomor 62/UN.34/19/2023."</code>	Membuat objek string dari input pengguna
<code>print(K1)</code> <code>print(K2)</code> <code>print(K3)</code> <code>print(K4)</code>	Menampilkan teks yang sudah dimasukkan pengguna

5.3 Program 3: Menentukan Akar Persamaan Kuadrat

.1 Kode program pada Python

Kode Program Python	Penjelasan
<code>import math</code>	Mengimpor modul math untuk menggunakan fungsi matematika, seperti akar kuadrat (sqrt).

def hitung_akar(data_koefisien):	Mendefinisikan fungsi bernama hitung_akar yang menerima parameter berupa data koefisien persamaan kuadrat.
nilai_a, nilai_b, nilai_c = data_koefisien	Mengambil nilai koefisien a, b, dan c dari list data_koefisien lalu menyimpannya ke masing-masing variabel.
nilai_diskriminan = (nilai_b ** 2) - (4 * nilai_a * nilai_c)	Menghitung nilai diskriminan menggunakan rumus $D = b^2 - 4ac$.
if nilai_diskriminan >= 0:	Memeriksa apakah diskriminan bernilai nol atau positif sehingga akar persamaan dapat dihitung sebagai bilangan real.
akar_1 = (-nilai_b + math.sqrt(nilai_diskriminan)) / (2 * nilai_a)	Menghitung akar pertama persamaan kuadrat menggunakan rumus kuadrat.
akar_2 = (-nilai_b - math.sqrt(nilai_diskriminan)) / (2 * nilai_a)	Menghitung akar kedua persamaan kuadrat menggunakan rumus kuadrat.
print(f"Akar pertama = {akar_1:.3f}")	Menampilkan nilai akar pertama dengan format tiga angka di belakang koma.
print(f"Akar kedua = {akar_2:.3f}")	Menampilkan nilai akar kedua dengan format tiga angka di belakang koma.
else:	Menjalankan perintah alternatif jika diskriminan bernilai negatif.
print("Persamaan memiliki akar imajiner")	Menampilkan pesan bahwa persamaan memiliki akar imajiner (tidak memiliki akar real).
nilai_a = float(input("Masukkan koefisien a : "))	Membaca input koefisien a dari pengguna dan mengubahnya menjadi tipe data float.
nilai_b = float(input("Masukkan koefisien b : "))	Membaca input koefisien b dari pengguna dan mengubahnya menjadi tipe data float.
nilai_c = float(input("Masukkan koefisien c : "))	Membaca input koefisien c dari pengguna dan mengubahnya menjadi tipe data float.

<code>data_koefisien = [nilai_a, nilai_b, nilai_c]</code>	Menyimpan ketiga koefisien ke dalam sebuah list bernama <code>data_koefisien</code> .
<code>hitung_akar(data_koefisien)</code>	Memanggil fungsi <code>hitung_akar</code> dengan parameter list koefisien untuk menghitung dan menampilkan akar persamaan kuadrat.

.2 Kode Program pada R

Kode Program R	Penjelasan
<code>a <- 1</code>	Menyimpan nilai 1 ke dalam variabel <code>a</code> sebagai koefisien x^2 .
<code>b <- -5</code>	Menyimpan nilai -5 ke dalam variabel <code>b</code> sebagai koefisien x .
<code>c <- 6</code>	Menyimpan nilai 6 ke dalam variabel <code>c</code> sebagai konstanta persamaan.
<code>Diskriminan <- b^2 - 4*a*c</code>	Menghitung nilai diskriminan menggunakan rumus $D = b^2 - 4ac$.
<code>if (a == 0) {</code>	Memeriksa apakah nilai <code>a</code> sama dengan 0. Jika ya, persamaan bukan persamaan kuadrat.
<code>cat("Error: Nilai a tidak boleh 0.\n")</code>	Menampilkan pesan kesalahan karena nilai <code>a</code> tidak boleh 0.
<code>} else if (diskriminan < 0) {</code>	Memeriksa apakah diskriminan bernilai negatif.
<code>cat("Persamaan memiliki akar imajiner.\n")</code>	Menampilkan pesan bahwa persamaan memiliki akar imajiner (tidak memiliki akar real).
<code>} else {</code>	Menjalankan blok perintah jika nilai <code>a</code> valid dan diskriminan tidak negatif.
<code>x1 <- (-b + sqrt(diskriminan)) / (2*a)</code>	Menghitung akar pertama persamaan kuadrat menggunakan rumus kuadrat.
<code>x2 <- (-b - sqrt(diskriminan)) / (2*a)</code>	Menghitung akar kedua persamaan kuadrat menggunakan rumus kuadrat.
<code>cat("Koefisien a =", a, "\n")</code>	Menampilkan nilai koefisien <code>a</code> .

cat("Koefisien b =", b, "\n")	Menampilkan nilai koefisien b.
cat("Koefisien c =", c, "\n")	Menampilkan nilai koefisien c.
cat("Diskriminan =", diskriminan, "\n")	Menampilkan nilai diskriminan yang telah dihitung.
cat("Akar pertama =", x1, "\n")	Menampilkan hasil akar pertama persamaan kuadrat.
cat("Akar kedua =", x2, "\n")	Menampilkan hasil akar kedua persamaan kuadrat.
}	Menandai akhir dari blok percabangan if-else.

5.4 Program 4: Menampilkan Tanggal Lahir ASN Berdasarkan NIP

1. Kode program pada Python

Kode Program Python	Penjelasan
NIP = input("Masukkan NIP ASN: ")	Memasukkan NIP ASN yang akan dicek tanggal lahirnya.
if not NIP.isdigit(): raise ValueError("NIP Harus Berupa Angka")	Memunculkan error jika NIP yang diinput bukan berupa angka.
Tahun = NIP[0:4]	Mengambil digit ke 1 sampai 4 dari NIP sebagai tahun lahir ASN.
Bulan = NIP[4:6]	Mengambil digit ke 5 dan 6 dari NIP sebagai bulan lahir ASN.
Tanggal = NIP[6:8]	Mengambil digit ke 7 dan 8 dari NIP sebagai tanggal lahir ASN.
if Bulan == "01":	Memeriksa kondisi apakah bulan adalah bulan 01.
nama_bulan = "Januari"	Menunjukkan bahwa nama bulan adalah Januari jika bulan 01.
elif Bulan == "02":	Memeriksa apakah bulan adalah bulan 02 jika kondisi sebelumnya salah.
nama_bulan = "Februari"	Menentukan bahwa nama bulan adalah Februari jika bulan 02.

elif Bulan == "03":	Memeriksa apakah bulan adalah bulan 03 jika kondisi sebelumnya salah.
nama_bulan = "Maret"	Menentukan bahwa nama bulan adalah Maret jika bulan 03.
elif Bulan == "04":	Memeriksa apakah bulan adalah bulan 04 jika kondisi sebelumnya salah.
nama_bulan = "April"	Menentukan bahwa nama bulan adalah April jika bulan 04.
elif Bulan == "05":	Memeriksa apakah bulan adalah bulan 05 jika kondisi sebelumnya salah.
nama_bulan = "Mei"	Menentukan bahwa nama bulan adalah Mei jika bulan 05.
elif Bulan == "06":	Memeriksa apakah bulan adalah bulan 06 jika kondisi sebelumnya salah.
nama_bulan = "Juni"	Menentukan bahwa nama bulan adalah Juni jika bulan 06.
elif Bulan == "07":	Memeriksa apakah bulan adalah bulan 07 jika kondisi sebelumnya salah.
nama_bulan = "Juli"	Menunjukkan bahwa nama bulan adalah Juli jika bulan 07.
elif Bulan == "08":	Memeriksa apakah bulan adalah bulan 08 jika kondisi sebelumnya salah.
nama_bulan = "Agustus"	Menunjukkan bahwa nama bulan adalah Agustus jika bulan 08.
elif Bulan == "09":	Memeriksa apakah bulan adalah bulan 09 jika kondisi sebelumnya salah.
nama_bulan = "September"	Menunjukkan bahwa nama bulan adalah September jika bulan 09.
elif Bulan == "10":	Memeriksa apakah bulan adalah bulan 10 jika kondisi sebelumnya salah.
nama_bulan = "Oktober"	Menunjukkan bahwa nama bulan adalah Oktober jika bulan 10.

elif Bulan == "11":	Memeriksa apakah bulan adalah bulan 11 jika kondisi sebelumnya salah.
nama_bulan = "November"	Menunjukkan bahwa nama bulan adalah November jika bulan 11.
elif Bulan == "12":	Memeriksa apakah bulan adalah bulan 12 jika kondisi sebelumnya salah.
nama_bulan = "Desember"	Menunjukkan bahwa nama bulan adalah Desember jika bulan 12.
else:	Menjalankan perintah jika semua kondisi salah.
nama_bulan = "Bulan Tidak Valid"	Menentukan bahwa bulan tidak valid jika semua kondisi salah.
print("Tanggal Lahir ASN:", Tanggal, nama_bulan, Tahun)	Menampilkan tanggal lahir ASN yang terdiri dari tanggal, nama bulan, dan tahun.

2. Kode Program pada R

Kode Program R	Penjelasan
NIP <- readline(prompt = "Masukkan NIP ASN: ")	Menerima input NIP ASN dari pengguna.
if (is.na(as.numeric(NIP))) { stop("Error: NIP Harus Berupa Angka") }	Memunculkan error jika NIP yang diinput bukan berupa angka.
Tahun <- substr(NIP, 1, 4)	Mengambil digit pertama hingga keempat sebagai tahun lahir ASN.
Bulan <- substr(NIP, 5, 6)	Mengambil digit ke 5 dan 6 dari NIP sebagai bulan lahir ASN.
Tanggal <- substr(NIP, 7, 8)	Mengambil digit ke 7 dan ke 8 dari NIP sebagai tanggal lahir ASN.
if (Bulan == "01") {	Memeriksa kondisi pertama apakah bulan adalah 01.
namaBulan <- "Januari"	Menunjukkan bahwa nama bulan adalah Januari jika bulan 01.
} else if (Bulan == "02") {	Memeriksa kondisi berikutnya jika kondisi sebelumnya salah.

namaBulan <- "Februari"	Menunjukkan bahwa nama bulan adalah Februari jika bulan 02.
} else if (Bulan == "03") {	Memeriksa kondisi berikutnya jika kondisi sebelumnya salah.
namaBulan <- "Maret"	Menunjukkan bahwa nama bulan adalah Maret jika bulan 03.
} else if (Bulan == "04") {	Memeriksa kondisi berikutnya jika kondisi sebelumnya salah.
namaBulan <- "April"	Menunjukkan bahwa nama bulan adalah April jika bulan 04.
} else if (Bulan == "05") {	Memeriksa kondisi berikutnya jika kondisi sebelumnya salah.
namaBulan <- "Mei"	Menunjukkan bahwa nama bulan adalah Mei jika bulan 05.
} else if (Bulan == "06") {	Memeriksa kondisi berikutnya jika kondisi sebelumnya salah.
namaBulan <- "Juni"	Menunjukkan bahwa nama bulan adalah Juni jika bulan 06.
} else if (Bulan == "07") {	Memeriksa kondisi berikutnya jika kondisi sebelumnya salah.
namaBulan <- "Juli"	Menunjukkan bahwa nama bulan adalah Juli jika bulan 07.
} else if (Bulan == "08") {	Memeriksa kondisi berikutnya jika kondisi sebelumnya salah.
namaBulan <- "Agustus"	Menunjukkan bahwa nama bulan adalah Agustus jika bulan 08.
} else if (Bulan == "09") {	Memeriksa kondisi berikutnya jika kondisi sebelumnya salah.
namaBulan <- "September"	Menunjukkan bahwa nama bulan adalah September jika bulan 09.
} else if (Bulan == "10") {	Memeriksa kondisi berikutnya jika kondisi sebelumnya salah.

namaBulan <- "Oktober"	Menunjukkan bahwa nama bulan adalah Oktober jika bulan 10.
} else if (Bulan == "11") {	Memeriksa kondisi berikutnya jika kondisi sebelumnya salah.
namaBulan <- "November"	Menunjukkan bahwa nama bulan adalah November jika bulan 11.
} else if (Bulan == "12") {	Memeriksa kondisi berikutnya jika kondisi sebelumnya salah.
namaBulan <- "Desember"	Menunjukkan bahwa nama bulan adalah Desember jika bulan 12.
} else {	Menjalankan perintah jika semua kondisi salah.
namaBulan <- "Bulan Tidak Valid"	Menunjukkan bahwa bulan tidak valid jika semua kondisi salah.
}	Menandai akhir dari blok percabangan if-else.
print(paste("Tanggal Lahir ASN :", Tanggal, namaBulan, Tahun))	Menampilkan tanggal lahir ASN yang terdiri dari tanggal, nama bulan, dan tahun.

5.5 Program 5: Klasifikasi Cluster Tiga Dimensi

1. Kode Program pada Python

Kode Program Python	Penjelasan
a <- c(2, 1, 3)	Menyimpan koordinat pusat cluster a.
b <- c(1, -4, 6)	Menyimpan koordinat pusat cluster b.
c <- c(-2, 3, -2)	Menyimpan koordinat pusat cluster c.
hitung_jarak <- function(x,y) {	Mendefinisikan fungsi bernama hitung_jarak dengan parameter x dan y.
sqrt((x[1] - y[1])^2 + (x[2] - y[2])^2 + (x[3] - y[3])^2.)}	Rumus untuk menghitung jarak Euclidean 3 dimensi.
x1 <- as.numeric(readline("Masukkan x1: "))	Membaca input x1 dari pengguna lalu mengubahnya menjadi numerik.

x2 <- as.numeric(readline("Masukkan x2:"))	Membaca input x2 dari pengguna lalu mengubahnya menjadi numerik.
x3 <- as.numeric(readline("Masukkan x3:"))	Membaca input x3 dari pengguna lalu mengubahnya menjadi numerik.
if (is.na(x1) is.na(x2) is.na(x3)) {	Fungsi cabang untuk memeriksa jika input yang dimasukkan bukan angka.
cat("Error: Input harus berupa angka.\n")	Menampilkan "error : input harus berupa angka".
else { titik_u <- c(x1, x2, x3)	Menggabungkan ketiga koordinat menjadi sebuah vektor yang mewakili titik u.
jarak_a <- hitung_jarak(titik_u, a)	Input untuk menetapkan nilai jarak u ke a.
jarak_b <- hitung_jarak(titik_u, b)	Input untuk menetapkan nilai jarak u ke b.
jarak_c <- hitung_jarak(titik_u, c)	Input untuk menetapkan nilai jarak u ke c.
if(jarak_a <- jarak_b && jarak_a < jarak_c){	Memeriksa apakah jarak ke cluster a paling kecil dibandingkan cluster lainnya.
cluster <- "A"	Menetapkan hasil klasifikasi sebagai cluster a.
} else if(jarak_b < jarak_a && jarak_b < jarak_c){	Memeriksa apakah jarak ke cluster b paling kecil dibandingkan cluster lainnya.
cluster <- "B"	Menetapkan hasil klasifikasi sebagai cluster b.
} else {	Kemungkinan kondisi selain dua kondisi sebelumnya.
cluster <- "C"	Menetapkan hasil klasifikasi sebagai cluster c.
}	Menandai akhir dari blok percabangan if-else.
cat(sprintf("Jarak ke A = %.3f\n", jarak_a))	Menampilkan panjang jarak u ke cluster a, dibatasi 3 digit di belakang koma.
cat(sprintf("Jarak ke B = %.3f\n", jarak_b))	Menampilkan panjang jarak u ke cluster b, dibatasi 3 digit di belakang koma.

<code>cat(sprintf("Jarak ke C = %.3f\n", jarak_c))</code>	Menampilkan panjang jarak u ke cluster c, dibatasi 3 digit di belakang koma.
<code>cat("Titik U termasuk Cluster", cluster, "\n")</code>	Menampilkan hasil akhir klasifikasi cluster berdasarkan jarak terkecil.

2. Kode Program pada R

Kode Program R	Penjelasan
<code>a <- c(2, 1, 3)</code>	Menyimpan koordinat pusat cluster a.
<code>b <- c(1, -4, 6)</code>	Menyimpan koordinat pusat cluster b.
<code>c <- c(-2, 3, -2)</code>	Menyimpan koordinat pusat cluster c.
<code>hitung_jarak <- function(x,y)</code> <code>{</code>	Mendefinisikan fungsi bernama <code>hitung_jarak</code> dengan parameter <code>x</code> dan <code>y</code> .
<code>sqrt((x[1] - y[1])^2 + (x[2] - y[2])^2 +</code> <code>(x[3] - y[3])^2.)}</code>	Rumus untuk menghitung jarak Euclidean 3 dimensi.
<code>x1 <- as.numeric(readline("Masukkan x1:</code> <code>"))</code>	Membaca input <code>x1</code> dari pengguna lalu mengubahnya menjadi numerik.
<code>x2 <- as.numeric(readline("Masukkan x2:</code> <code>"))</code>	Membaca input <code>x2</code> dari pengguna lalu mengubahnya menjadi numerik.
<code>x3 <- as.numeric(readline("Masukkan x3:</code> <code>"))</code>	Membaca input <code>x3</code> dari pengguna lalu mengubahnya menjadi numerik.
<code>if (is.na(x1) is.na(x2) is.na(x3))</code> <code>{</code>	Fungsi cabang untuk memeriksa jika input yang dimasukkan bukan angka.
<code>cat("Error: Input harus berupa angka.\n")</code>	Menampilkan "error : input harus berupa angka".
<code>else {</code> <code>titik_u <- c(x1, x2, x3)</code>	Menggabungkan ketiga koordinat menjadi sebuah vektor yang mewakili titik u.
<code>jarak_a <- hitung_jarak(titik_u, a)</code>	Input untuk menetapkan nilai jarak u ke a.
<code>jarak_b <- hitung_jarak(titik_u, b)</code>	Input untuk menetapkan nilai jarak u ke b.
<code>jarak_c <- hitung_jarak(titik_u, c)</code>	Input untuk menetapkan nilai jarak u ke c.
<code>if(jarak_a < jarak_b && jarak_a <</code> <code>jarak_c){</code>	Memeriksa apakah jarak ke cluster a paling kecil dibandingkan cluster lainnya.

<code>cluster <- "A"</code>	Menetapkan hasil klasifikasi sebagai cluster a.
<code>} else if(jarak_b < jarak_a && jarak_b < jarak_c){</code>	Memeriksa apakah jarak ke cluster b paling kecil dibandingkan cluster lainnya.
<code>cluster <- "B"</code>	Menetapkan hasil klasifikasi sebagai cluster b.
<code>} else {</code>	Kemungkinan kondisi selain dua kondisi sebelumnya.
<code>cluster <- "C"</code>	Menetapkan hasil klasifikasi sebagai cluster c.
<code>}</code>	Menandai akhir dari blok percabangan if-else.
<code>cat(sprintf("Jarak ke A = %.3f\n", jarak_a))</code>	Menampilkan panjang jarak u ke cluster a, dibatasi 3 digit di belakang koma.
<code>cat(sprintf("Jarak ke B = %.3f\n", jarak_b))</code>	Menampilkan panjang jarak u ke cluster b, dibatasi 3 digit di belakang koma.
<code>cat(sprintf("Jarak ke C = %.3f\n", jarak_c))</code>	Menampilkan panjang jarak u ke cluster c, dibatasi 3 digit di belakang koma.
<code>cat("Titik U termasuk Cluster", cluster, "\n")</code>	Menampilkan hasil akhir klasifikasi cluster berdasarkan jarak terkecil.

5.6 Program 6: Interval Konfidensi Proporsi

1. Kode Program pada Python

Kode Program Python	Penjelasan
<code>import math</code>	Mengimpor modul math untuk menggunakan fungsi matematika seperti akar kuadrat (sqrt).
<code>def cari_nilai_kritis(tingkat_signifikansi):</code>	Mendefinisikan fungsi untuk menentukan nilai kritis (Z) berdasarkan tingkat signifikansi.
<code>if tingkat_signifikansi == 0.10:</code>	Memeriksa apakah tingkat signifikansi yang dimasukkan adalah 0,10.

return 1.645	Mengembalikan nilai kritis $Z = 1,645$ untuk $\alpha = 0,10$.
elif tingkat_signifikansi == 0.05:	Memeriksa apakah tingkat signifikansi yang dimasukkan adalah 0,05.
return 1.96	Mengembalikan nilai kritis $Z = 1,96$ untuk $\alpha = 0,05$.
else:	Menjalankan perintah jika nilai α tidak sesuai ketentuan.
return None	Mengembalikan nilai kosong sebagai tanda input tidak valid.
def hitung_batas_kesalahan(proporsi_sampel, nilai_kritis, ukuran_sampel):	Mendefinisikan fungsi untuk menghitung margin of error (batas kesalahan).
return nilai_kritis * math.sqrt((proporsi_sampel * (1 -proporsi_sampel)) / ukuran_sampel)	Menghitung batas kesalahan menggunakan rumus interval kepercayaan proporsi.
def hitung_interval_kepercayaan (proporsi_sampel, tingkat_signifikansi, ukuran_sampel):	Mendefinisikan fungsi utama untuk menghitung interval kepercayaan.
if proporsi_sampel < 0 or proporsi_sampel >1:	Memeriksa apakah nilai proporsi berada di luar rentang 0 sampai 1.
return "Error: Nilai proporsi harus antara 0 dan 1."	Mengembalikan pesan kesalahan jika proporsi tidak valid.
nilai_kritis = cari_nilai_kritis (tingkat_signifikansi)	Memanggil fungsi untuk mendapatkan nilai kritis sesuai α .
if nilai_kritis is None:	Memeriksa apakah nilai kritis berhasil diperoleh.
return "Error: Tingkat signifikansi hanya boleh 0.10 atau 0.05."	Mengembalikan pesan kesalahan jika α tidak sesuai.
batas_kesalahan = hitung_batas_kesalahan(	Memanggil fungsi untuk menghitung margin of error.

proporsi_sampel,	Menyimpan nilai proporsi sampel ($\hat{p}$), misalnya 0,6 atau 60%. Menyimpan nilai Z sesuai tingkat signifikansi, misalnya 1,645 ($\alpha = 0,10$) atau 1,96 ($\alpha = 0,05$). Digunakan untuk menentukan tingkat kepercayaan interval.
nilai_kritis,	Digunakan dalam perhitungan standar error.
ukuran_sampel)	Menyimpan jumlah data atau responden (n). Digunakan dalam penyebut rumus standar error.
interval_minimum = proporsi_sampel - batas_kesalahan	Menghitung batas bawah interval kepercayaan.
interval_maksimum = proporsi_sampel + batas_kesalahan	Menghitung batas atas interval kepercayaan.
return interval_minimum, interval_maksimum	Mengembalikan hasil interval kepercayaan dalam bentuk tuple.
nilai_proporsi = float(input("Masukkan nilai proporsi sampel: "))	Membaca input proporsi sampel dari pengguna dan mengubahnya menjadi tipe data float.
alpha_input = float(input("Masukkan tingkat signifikansi (0.10 atau 0.05): "))	Membaca input tingkat signifikansi dari pengguna dan mengubahnya menjadi float.
jumlah_data = int(input("Masukkan ukuran sampel: "))	Membaca input ukuran sampel dan mengubahnya menjadi integer.
hasil_perhitungan = hitung_interval_kepercayaan(nilai_proporsi,alpha_input, jumlah_data)	Memanggil fungsi utama untuk menghitung interval kepercayaan.
if isinstance(hasil_perhitungan, str):	Memeriksa apakah hasil berupa pesan error (string). --> #isinstance() berfungsi sebagai percabangan untuk membedakan apakah hasil yang diterima adalah pesan kesalahan

	(string) atau hasil perhitungan interval kepercayaan (tuple).
<code>print(hasil_perhitungan)</code>	Menampilkan pesan error jika terjadi kesalahan input.
<code>else:</code>	Menjalankan perintah jika perhitungan berhasil dilakukan.
<code>batas_awal, batas_akhir = hasil_perhitungan</code>	Mengambil batas bawah dan batas atas dari hasil perhitungan.
<code>print(f"Interval Kepercayaan = ({batas_awal:.4f}, {batas_akhir:.4f})")</code>	Menampilkan interval kepercayaan dengan 4 angka di belakang koma.

2. Kode Program pada R

Kode Program R	Penjelasan
<code>hitung_interval_konfidensi <- function(phat, n, alpha) {</code>	Mendefinisikan fungsi bernama <code>hitung_interval_konfidensi</code> yang menerima tiga parameter yaitu proporsi sampel (<code>phat</code>), ukuran sampel (<code>n</code>), dan tingkat signifikansi (<code>alpha</code>).
<code>if (is.na(phat) is.na(n) is.na(alpha)) {</code>	<code>is.na()</code> adalah fungsi bawaan di R yang digunakan untuk mengecek apakah suatu nilai adalah NA (Not Available) atau tidak.
<code>cat("ERROR: Input harus berupa angka.\n")</code>	Menampilkan pesan kesalahan jika input tidak valid.
<code>return(NULL) }</code>	Menghentikan fungsi dan mengembalikan nilai kosong sebagai tanda terjadi kesalahan.
<code>if (phat < 0 phat > 1) {</code>	Kondisi jika proporsi kurang dari 0 atau lebih dari 1
<code>cat("ERROR: Proporsi sampel (phat) harus antara 0 dan 1.\n")</code>	Menampilkan pesan kesalahan jika proporsi di luar rentang yang diperbolehkan.
<code>return(NULL) }</code>	Menghentikan fungsi karena data tidak memenuhi syarat.

<code>if (n <= 0 n != round(n)) {</code>	Kondisi jika ukuran sampel bukan bilangan bulat positif
<code>cat("ERROR: Ukuran sampel (n) harus bilangan bulat positif.\n")</code>	Menampilkan pesan kesalahan jika ukuran sampel tidak valid.
<code>return(NULL) }</code>	Mengakhiri fungsi ketika ukuran sampel salah.
<code>if (alpha == 0.10) {</code>	Memeriksa apakah tingkat signifikansi yang dipilih adalah 10%.
<code>z <- 1.645</code>	Menetapkan nilai Z sebesar 1,645 untuk tingkat kepercayaan 90%
<code>} else if (alpha == 0.05) {</code>	Memeriksa apakah tingkat signifikansi yang dipilih adalah 5%
<code>z <- 1.96</code>	Menetapkan nilai Z sebesar 1,96 untuk tingkat kepercayaan 95%.
<code>} else {</code>	Menjalankan blok ini jika nilai alpha tidak sesuai ketentuan
<code>cat("ERROR: Alpha hanya boleh 0.05 atau 0.10.\n")</code>	Menampilkan pesan kesalahan jika alpha selain 0,05 atau 0,10.
<code>return(NULL)</code> <code>}</code>	Menghentikan fungsi karena alpha tidak valid.
<code>margin_error <- z * sqrt(phat * (1 - phat) / n)</code>	Menghitung margin of error menggunakan rumus interval kepercayaan proporsi.
<code>lower <- phat - margin_error</code>	Menghitung batas bawah interval kepercayaan.
<code>upper <- phat + margin_error</code>	Menghitung batas atas interval kepercayaan.
<code>return(list()) }</code>	Mengembalikan hasil perhitungan dalam bentuk list agar dapat digunakan di program utama.
<code>phat = phat,</code>	Menyimpan nilai proporsi sampel ke dalam hasil keluaran.
<code>n = n,</code>	Menyimpan ukuran sampel ke dalam hasil keluaran.

<code>alpha = alpha,</code>	Menyimpan nilai alpha yang digunakan.
<code>confidence_level = (1 - alpha) * 100,</code>	Menghitung tingkat kepercayaan dalam bentuk persen.
<code>z = z,</code>	Menyimpan nilai Z yang digunakan dalam perhitungan.
<code>lower_bound = lower,</code>	Menyimpan batas bawah interval kepercayaan.
<code>upper_bound = upper</code>	Menyimpan batas atas interval kepercayaan.
<code>phat <- as.numeric(readline ("Masukkan proporsi sampel (0 - 1): "))</code>	Membaca input proporsi sampel dari pengguna dan mengubahnya menjadi tipe numerik.
<code>n <- as.numeric(readline("Masukkan ukuran sampel (n): "))</code>	Membaca ukuran sampel dari pengguna dan mengubahnya menjadi numerik.
<code>alpha <- as.numeric(readline ("Masukkan alpha (0.05 atau 0.10): "))</code>	Membaca nilai alpha dari pengguna dan mengubahnya menjadi numerik.
<code>hasil <- hitung_interval_konfidensi(phat, n, alpha)</code>	Memanggil fungsi perhitungan interval kepercayaan menggunakan input yang diberikan pengguna.
<code>if (!is.null(hasil)) {</code>	Memeriksa apakah fungsi berhasil menghasilkan output.
<code>cat(sprintf("Interval Konfidensi : (%.4f, %.4f)\n", hasil\$lower_bound, hasil\$upper_bound))</code>	Menampilkan hasil interval kepercayaan dengan format empat angka di belakang koma.
<code> } else { }</code>	Menjalankan bagian ini jika fungsi mengembalikan nilai NULL.
<code>cat("\nProgram dihentikan karena input tidak valid.\n")</code>	Menampilkan pesan bahwa program tidak dapat melanjutkan karena terdapat kesalahan input.

BAB VI

PENGUJIAN PROGRAM

6.1 Tabel Pengujian

1. Program 1: Menghitung Jumlah Kata dan Jumlah Kalimat

- Pengujian pada Python dan R:

No	Input	Output yang Diharapkan	Output	Status	Catatan
1	Media sosial atau disebut juga dengan jejaring sosial, seperti Facebook, Twitter, Instagram, dan masih banyak lagi ternyata tidak hanya digunakan sebagai tempat berkumpul atau berbagi di dunia maya. Namun, media sosial kini juga bisa dimanfaatkan sebagai media untuk mengembangkan sebuah bisnis. Saat ini telah banyak para pengusaha yang	Teks tersebut memuat 5 kalimat dan 92 kata.	Teks tersebut memuat 5 kalimat dan 92 kata.	Berhasil	Program berhasil menghitung jumlah kalimat dan jumlah kata dalam teks tersebut.

	beralih ke media sosial dalam memasarkan produk mereka baik barang ataupun jasa. Beralihnya para pelaku bisnis ke media ini dikarenakan jejaring sosial memiliki manfaat yang sangat banyak bagi usaha bisnis. Berikut ini adalah alasan mengapa jejaring sosial bisa menjadi alat promosi yang paling efektif.				
2	Membaca buku merupakan salah satu kebiasaan yang memiliki banyak manfaat bagi perkembangan pengetahuan dan pola pikir seseorang. Dengan	Teks tersebut memuat 4 kalimat dan 78 kata.	Teks tersebut memuat 4 kalimat dan 78 kata.	Berhasil	Program berhasil menghitung jumlah kalimat dan jumlah kata dalam teks tersebut.

	membaca, seseorang dapat memperoleh informasi baru, memperluas wawasan, serta meningkatkan kemampuan berpikir kritis dan analitis. Selain itu, buku juga dapat menjadi sarana untuk memahami berbagai perspektif dan pengalaman hidup yang berbeda. Di tengah perkembangan teknologi yang semakin pesat, minat baca tetap perlu ditingkatkan karena kemampuan memahami informasi secara mendalam menjadi keterampilan				
--	--	--	--	--	--

	penting yang dibutuhkan dalam dunia pendidikan maupun kehidupan sehari-hari.				
3	Pendidikan memiliki peran yang sangat penting dalam membentuk kualitas sumber daya manusia dan kemajuan suatu bangsa. Melalui pendidikan, seseorang tidak hanya memperoleh pengetahuan akademik, tetapi juga nilai-nilai karakter seperti tanggung jawab, disiplin, dan kerja sama. Pendidikan yang berkualitas dapat membuka peluang yang lebih luas bagi	Teks tersebut memuat 4 kalimat dan 80 kata.	Teks tersebut memuat 4 kalimat dan 80 kata.	Berhasil	Program berhasil menghitung jumlah kalimat dan jumlah kata dalam teks tersebut.

	individu untuk mengembangkan potensi diri dan mencapai tujuan hidupnya. Oleh karena itu, akses terhadap pendidikan yang merata dan berkualitas perlu terus didukung agar setiap orang memiliki kesempatan yang sama untuk berkembang dan berkontribusi bagi masyarakat.				
--	---	--	--	--	--

2. Program 3: Menentukan Akar Persamaan Kuadrat

- Pengujian pada Python:

No	Input	Output yang Diharapkan	Output	Status	Catatan
1	a = 1 b = -5 c = 6 diskriminan = 1	Akar pertama = 3.000 Akar kedua = 2.000	Akar pertama = 3.000, Akar kedua = 2.000	Berhasil	Dua akar real berhasil dihitung dengan format 3 desimal.
2	a = 2 b = -2 c = 4 diskriminan = -28	Persamaan memiliki akar imajiner	Persamaan memiliki akar imajiner	Berhasil	Diskriminan negatif terdeteksi dengan benar.

3	a = 0 b = 3 c = -2 diskriminan = 9	Error: Nilai a tidak boleh 0	ZeroDivisionError (pembagian dengan nol)	Gagal	Belum terdapat validasi untuk a = 0.
---	---	------------------------------	--	-------	--------------------------------------

- Pengujian pada R:

No	Input	Output yang Diharapkan	Output	Status	Catatan
1	a = 1 b = -5 c = 6 diskriminan = 1	Akar pertama = 3.000 Akar kedua = 2.000	Akar pertama = 3.000, Akar kedua = 2.000	Berhasil	Dua akar real berhasil dihitung dengan format 3 desimal.
2	a = 2 b = -2 c = 4 diskriminan = -28	Persamaan memiliki akar imajiner	Persamaan memiliki akar imajiner	Berhasil	Diskriminan negatif terdeteksi dengan benar.
3	a = 0 b = 3 c = -2 diskriminan = 9	Error: Nilai a tidak boleh 0	Error: Nilai a tidak boleh 0	Gagal	Belum terdapat validasi untuk a = 0.

3. Program 4: Menampilkan Tanggal Lahir ASN Berdasarkan NIP

- Pengujian pada Python:

No	Input	Output yang Diharapkan	Output	Status	Catatan
1	199301212019031010	21 Januari 1993	21 Januari 1993	Berhasil	NIP berhasil dihitung.
2	200614092019031010	9 Bulan Tidak Valid 2006	9 Bulan Tidak Valid 2006	Berhasil	Bulan tidak valid

					berhasil terdeteksi.
3	Ptpastibangkrut2019031010	ValueError: NIP Harus Berupa Angka	ValueError: NIP Harus Berupa Angka	Gagal	Input tidak valid berhasil terdeteksi.

- Pengujian pada R:

No	Input	Output yang Diharapkan	Output	Status	Catatan
1	199301212019031010	21 Januari 1993	21 Januari 1993	Berhasil	NIP berhasil dihitung.
2	200614092019031010	9 Bulan Tidak Valid 2006	9 Bulan Tidak Valid 2006	Berhasil	Bulan tidak valid berhasil terdeteksi.
3	Ptpastibangkrut2019031010	Error: NIP Harus Berupa Angka	Error: NIP Harus Berupa Angka	Gagal	Input tidak valid berhasil terdeteksi.

4. Program 5: Klasifikasi Cluster Tiga Dimensi

- Pengujian pada Python:

No	Input	Output yang Diharapkan	Output	Status	Catatan
1	Titik U = 2, 3, 4	Jarak ke A = 2.236 Jarak ke B = 7.348 Jarak ke C = 7.211 Titik U termasuk Cluster A	Jarak ke A = 2.236 Jarak ke B = 7.348 Jarak ke C = 7.211 Titik U termasuk Cluster A	Berhasil	Karena Cluster A memiliki jarak terkecil atau yang paling dekat dengan titik U, maka titik

					U termasuk Cluster A.
2	Titik U = 3, -4, 7	Jarak ke A = 6.481 Jarak ke B = 2.236 Jarak ke C = 12.450 Titik U termasuk Cluster B	Persamaan memiliki akar imajiner	Berhasil	Karena Cluster B memiliki jarak terkecil atau yang paling dekat dengan titik U, maka titik U termasuk Cluster B.
3	Titik U = 2, b, 8	Error: Input harus berupa angka	Error: Input harus berupa angka	Berhasil	Fungsi program hanya dapat membaca input berupa angka, jika input bukan berupa angka maka sistem akan menampilkan pesan error.

- Pengujian pada R:

No	Input	Output yang Diharapkan	Output	Status	Catatan
1	Titik U = 2, 3, 4	Jarak ke A = 2.236 Jarak ke B = 7.348 Jarak ke C = 7.211	Jarak ke A = 2.236 Jarak ke B = 7.348 Jarak ke C = 7.211	Berhasil	Karena Cluster A memiliki jarak terkecil atau yang paling dekat

		Titik U termasuk Cluster A	Titik U termasuk Cluster A		dengan titik U, maka titik U termasuk Cluster A.
2	Titik U = 3, -4, 7	Jarak ke A = 6.481 Jarak ke B = 2.236 Jarak ke C = 12.450 Titik U termasuk Cluster B	Persamaan memiliki akar imajiner	Berhasil	Karena Cluster B memiliki jarak terkecil atau yang paling dekat dengan titik U, maka titik U termasuk Cluster B.
3	Titik U = 2, b, 8	Error: Input harus berupa angka	Error: Input harus berupa angka	Berhasil	Fungsi program hanya dapat membaca input berupa angka, jika input bukan berupa angka maka sistem akan menampilkan pesan error.

5. Program 6: Interval Konfidensi Proporsi

- Pengujian pada Python:

No	Input	Output yang Diharapkan	Output	Status	Catatan
1	Proporsi sampel: 0,9	Interval Kepercayaan:	Interval Kepercayaan:	Berhasil	Nilai inputnya valid dan program

	Tingkat signifikansi: 0,1 Ukuran sampel: 400	0,8753 dan 0,9247	0,8753 dan 0,9247		berhasil menghitung interval kepercayaannya.
2	Proporsi sampel: 1,6 Tingkat signifikansi: 0,05 Ukuran sampel: 3000	Error: Nilai proporsi harus antara 0 dan 1	Error: Nilai proporsi harus antara 0 dan 1	Berhasil	Nilai input untuk proporsi harus antara 0 dan 1.
3	Proporsi sampel: 0,9 Tingkat signifikansi: 0,1 Ukuran sampel: 400	Error: Tingkat signifikansi hanya boleh 0,10 atau 0,05	Error: Tingkat signifikansi hanya boleh 0,10 atau 0,05	Berhasil	Nilai input untuk tingkat signifikansi hanya boleh 0,10 atau 0,05.

• Pengujian pada R:

No	Input	Output yang Diharapkan	Output	Status	Catatan
1	Proporsi sampel: 0,9 Ukuran sampel: 400 Alpha: 0,1	Interval Konfidensi: (0,8753, 0,9247)	Interval Konfidensi: (0,8753, 0,9247)	Berhasil	Nilai inputnya valid dan program berhasil menghitung interval konfidensinya.
2	Proporsi sampel: 1,6 Ukuran sampel: 3000	ERROR: Proporsi sampel (phat)	ERROR: Proporsi sampel (phat)	Berhasil	Nilai input untuk proporsi sampel (phat)

	Alpha: 0,05	harus antara 0 dan 1	harus antara 0 dan 1		harus antara 0 dan 1.
3	Proporsi sampel: 0,7 Ukuran sampel: 500 Alpha: 0,03	ERORR: Alpha hanya boleh 0,05 atau 0,10	ERORR: Alpha hanya boleh 0,05 atau 0,10	Berhasil	Nilai input untuk alpha hanya boleh 0,05 atau 0,10.

6.2 Screenshot Hasil Running

1. Hasil Running Program 1: Menghitung Jumlah Kata dan Jumlah Kalimat

- Python

1) Uji 1

```

#Program 1 - Menghitung jumlah kata dan jumlah kalimat dari sebuah teks
teks = input("Masukkan teks: ").strip()

#Hitung jumlah kalimat (berdasarkan titik)
jumlah_kalimat = teks.count(".")

#Hitung jumlah kata
jumlah_kata = len(teks.split())

print(f"Teks tersebut memuat {jumlah_kalimat} kalimat dan {jumlah_kata} kata.")

```

... Masukkan teks: Media sosial atau disebut juga dengan jejaring sosial, seperti Facebook, Twitter, Instagram, dan masih banyak lagi ternyata ti
Teks tersebut memuat 5 kalimat dan 92 kata.

2) Uji 2

```

#Program 1 - Menghitung jumlah kata dan jumlah kalimat dari sebuah teks
teks = input("Masukkan teks: ").strip()

#Hitung jumlah kalimat (berdasarkan titik)
jumlah_kalimat = teks.count(".")

#Hitung jumlah kata
jumlah_kata = len(teks.split())

print(f"Teks tersebut memuat {jumlah_kalimat} kalimat dan {jumlah_kata} kata.")

```

... Masukkan teks: Membaca buku merupakan salah satu kebiasaan yang memiliki banyak manfaat bagi perkembangan pengetahuan dan pola pikir seseorang
Teks tersebut memuat 4 kalimat dan 78 kata.

3) Uji 3

```

#Program 1 - Menghitung jumlah kata dan jumlah kalimat dari sebuah teks
teks = input("Masukkan teks: ").strip()

#Hitung jumlah kalimat (berdasarkan titik)
jumlah_kalimat = teks.count(".")

#Hitung jumlah kata
jumlah_kata = len(teks.split())

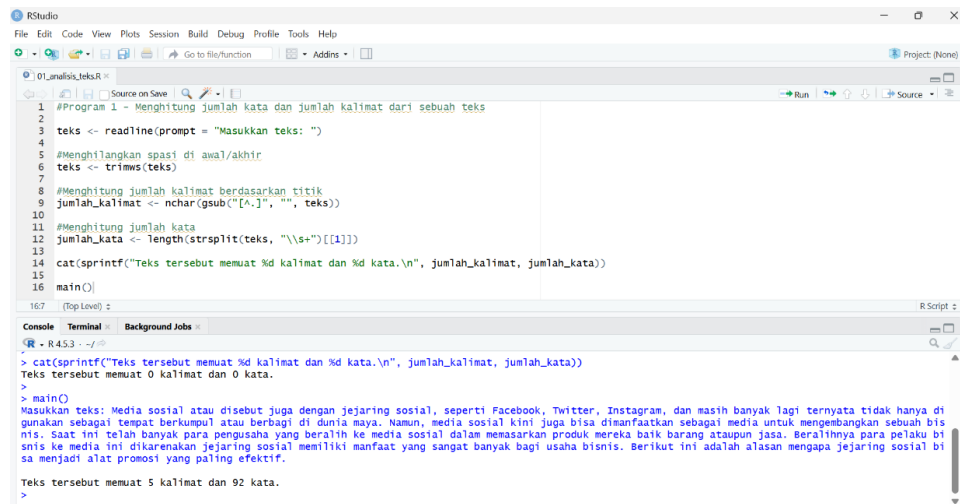
print(f"Teks tersebut memuat {jumlah_kalimat} kalimat dan {jumlah_kata} kata.")

```

... Masukkan teks: Pendidikan memiliki peran yang sangat penting dalam membentuk kualitas sumber daya manusia dan kemajuan suatu bangsa. Melalui
Teks tersebut memuat 4 kalimat dan 80 kata.

- R

1) Uji 1



```

1 #Program 1 - Menghitung jumlah kata dan jumlah kalimat dari sebuah teks
2
3 teks <- readline(prompt = "Masukkan teks: ")
4
5 #Menghilangkan spasi di awal/akhir
6 teks <- trimws(teks)
7
8 #Menghitung jumlah kalimat berdasarkan titik
9 jumlah_kalimat <- nchar(gsub("[.]", "", teks))
10
11 #Menghitung jumlah kata
12 jumlah_kata <- length(strsplit(teks, "\\s+")[1])
13
14 cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))
15
16 main()
17

```

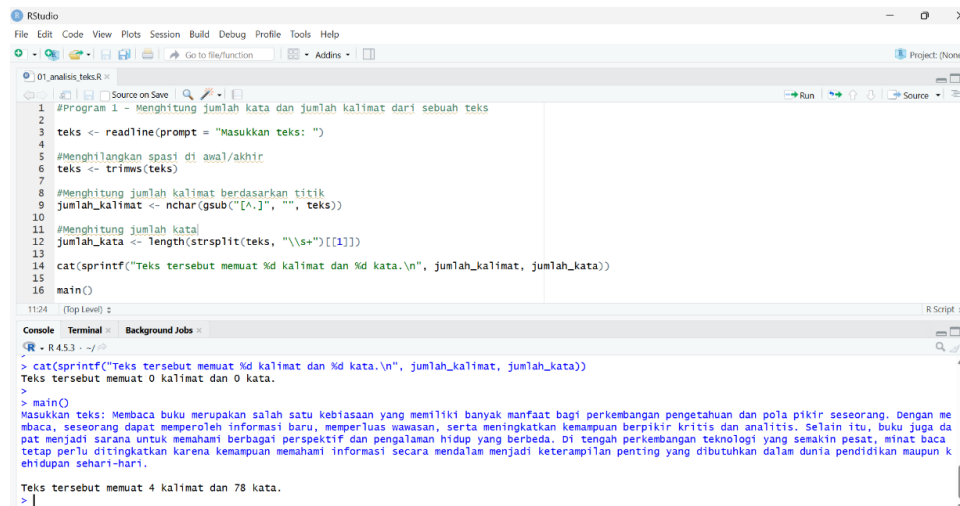
Console Terminal Background Jobs

```

> cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))
Teks tersebut memuat 0 kalimat dan 0 kata.
>
> main()
Masukkan teks: Media sosial atau disebut juga dengan jejaring sosial, seperti Facebook, Twitter, Instagram, dan masih banyak lagi ternyata tidak hanya di
gunakan sebagai tempat berkumpul atau berbagi di dunia maya. Namun, media sosial kini juga bisa dimanfaatkan sebagai media untuk mengembangkan sebuah bis
nis. Saat ini telah banyak para pengusaha yang beralih ke media sosial dalam memasarkan produk mereka baik barang ataupun jasa. Beralihnya para pelaku bi
snis ke media ini dikarenakan jejaring sosial memiliki manfaat yang sangat banyak bagi usaha bisnis. Berikut ini adalah alasan mengapa jejaring sosial bi
sa menjadi alat promosi yang paling efektif.
Teks tersebut memuat 5 kalimat dan 92 kata.
>

```

2) Uji 2



```

1 #Program 1 - Menghitung jumlah kata dan jumlah kalimat dari sebuah teks
2
3 teks <- readline(prompt = "Masukkan teks: ")
4
5 #Menghilangkan spasi di awal/akhir
6 teks <- trimws(teks)
7
8 #Menghitung jumlah kalimat berdasarkan titik
9 jumlah_kalimat <- nchar(gsub("[.]", "", teks))
10
11 #Menghitung jumlah kata
12 jumlah_kata <- length(strsplit(teks, "\\s+")[1])
13
14 cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))
15
16 main()
17

```

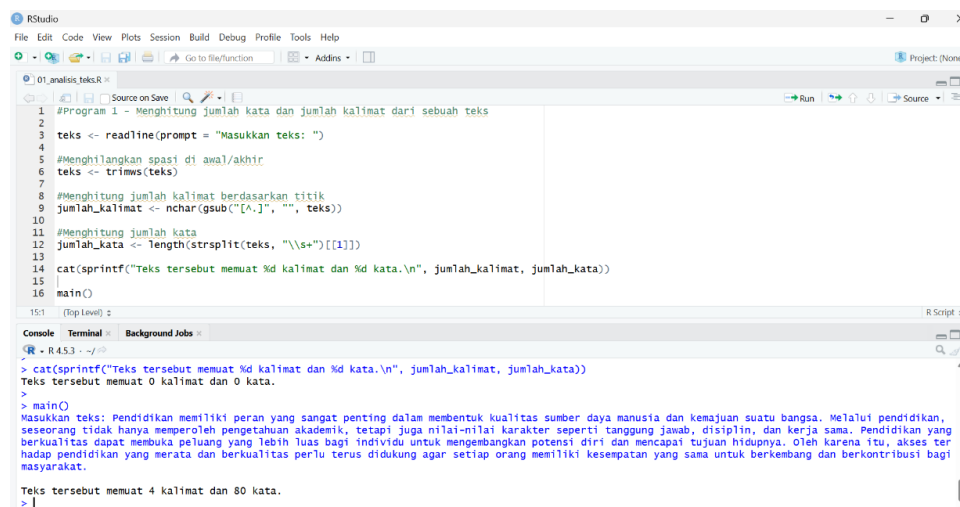
Console Terminal Background Jobs

```

> cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))
Teks tersebut memuat 0 kalimat dan 0 kata.
>
> main()
Masukkan teks: Membaca buku merupakan salah satu kebiasaan yang memiliki banyak manfaat bagi perkembangan pengetahuan dan pola pikir seseorang. Dengan me
mbaca, seseorang dapat memperoleh informasi baru, memperluas wawasan, serta meningkatkan kemampuan berpikir kritis dan analitis. Selain itu, buku juga da
pat menjadi sarana untuk memahami berbagai perspektif dan pengalaman hidup yang berbeda. Di tengah perkembangan teknologi yang semakin pesat, minat baca
tetap perlu ditingkatkan karena kemampuan memahami informasi secara mendalam menjadi keterampilan penting yang dibutuhkan dalam dunia pendidikan maupun k
ehidupan sehari-hari.
Teks tersebut memuat 4 kalimat dan 78 kata.
>

```

3) Uji 3



```

1 #Program 1 - Menghitung jumlah kata dan jumlah kalimat dari sebuah teks
2
3 teks <- readline(prompt = "Masukkan teks: ")
4
5 #Menghilangkan spasi di awal/akhir
6 teks <- trimws(teks)
7
8 #Menghitung jumlah kalimat berdasarkan titik
9 jumlah_kalimat <- nchar(gsub("[.]", "", teks))
10
11 #Menghitung jumlah kata
12 jumlah_kata <- length(strsplit(teks, "\\s+")[1])
13
14 cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))
15
16 main()
17

```

Console Terminal Background Jobs

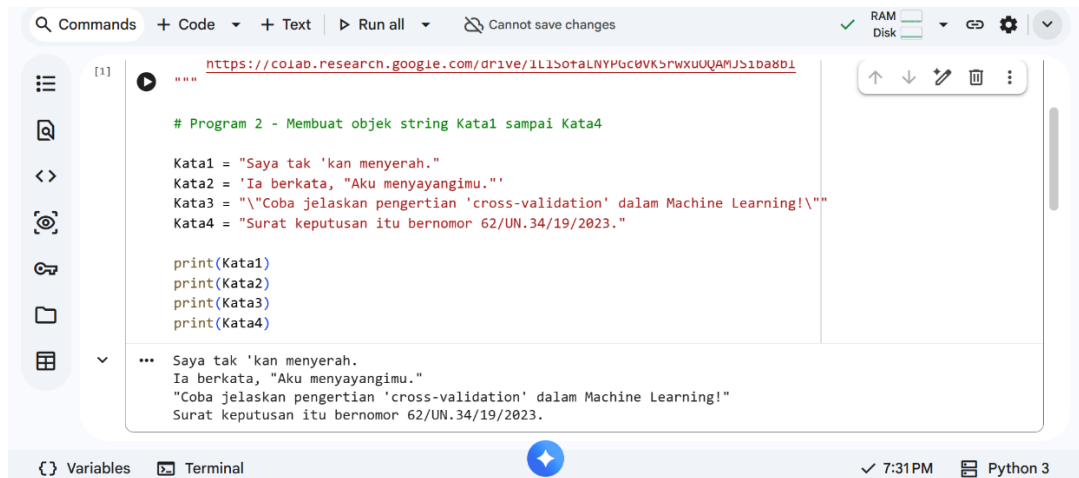
```

> cat(sprintf("Teks tersebut memuat %d kalimat dan %d kata.\n", jumlah_kalimat, jumlah_kata))
Teks tersebut memuat 0 kalimat dan 0 kata.
>
> main()
Masukkan teks: Pendidikan memiliki peran yang sangat penting dalam membentuk kualitas sumber daya manusia dan kemajuan suatu bangsa. Melalui pendidikan,
seseorang tidak hanya memperoleh pengetahuan akademik, tetapi juga nilai-nilai karakter seperti tanggung jawab, disiplin, dan kerja sama. Pendidikan yang
berkualitas dapat membuka peluang yang lebih luas bagi individu untuk mengembangkan potensi diri dan mencapai tujuan hidupnya. Oleh karena itu, akses ter
hadap pendidikan yang merata dan berkualitas perlu terus didukung agar setiap orang memiliki kesempatan yang sama untuk berkembang dan berkontribusi bagi
masyarakat.
Teks tersebut memuat 4 kalimat dan 80 kata.
>

```


2. Hasil Running Program 2: Membuat String Kalimat

- Python



```
https://colab.research.google.com/drive/1L150taLNYPGc0VK5rWxUQAMJ51Ba8D1

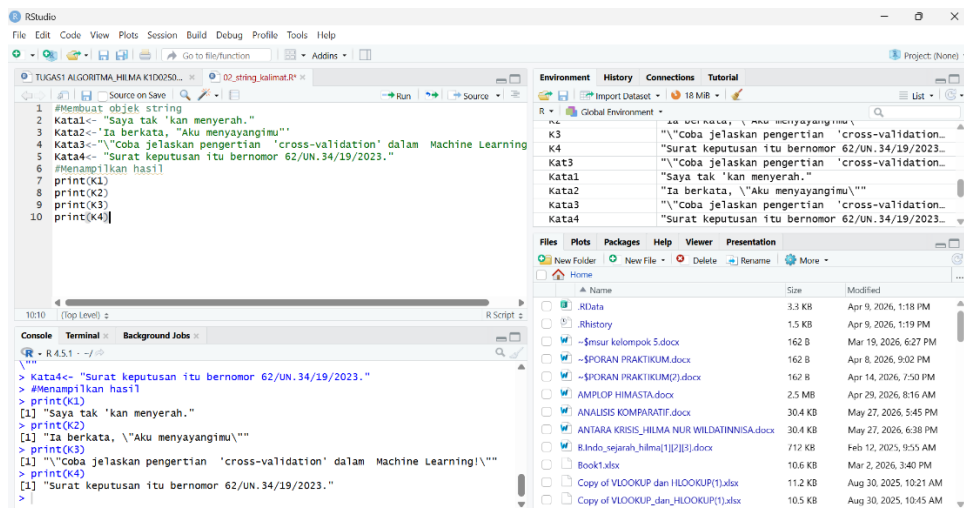
# Program 2 - Membuat objek string Kata1 sampai Kata4

Kata1 = "Saya tak 'kan menyerah."
Kata2 = "Ia berkata, "Aku menyayangimu.""
Kata3 = "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
Kata4 = "Surat keputusan itu bernomor 62/UN.34/19/2023."

print(Kata1)
print(Kata2)
print(Kata3)
print(Kata4)
```

... Saya tak 'kan menyerah.
Ia berkata, "Aku menyayangimu."
"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!"
Surat keputusan itu bernomor 62/UN.34/19/2023.

- R



```
#Membuat objek string
1 kata1<- "Saya tak 'kan menyerah."
2 kata2<- "\"Ia berkata, \"Aku menyayangimu.\""
3 kata3<- "\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
4 kata4<- "Surat keputusan itu bernomor 62/UN.34/19/2023."
5 #Menampilkan hasil
6 print(k1)
7 print(k2)
8 print(k3)
9 print(k4)
10
```

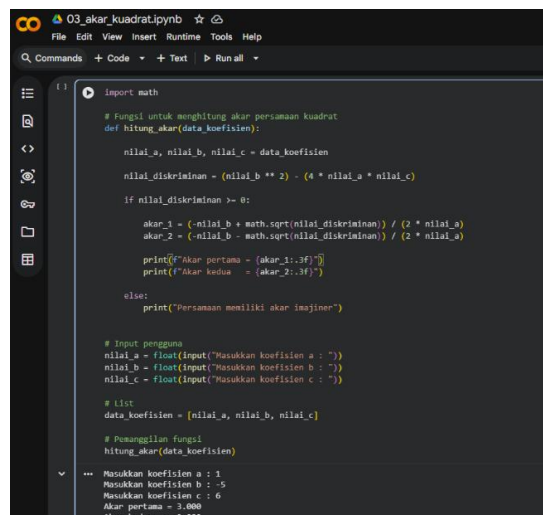
Environment

Object	Value
K3	"\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
K4	"Surat keputusan itu bernomor 62/UN.34/19/2023."
Kat3	"\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
Kata1	"Saya tak 'kan menyerah."
Kata2	"\"Ia berkata, \"Aku menyayangimu.\""
Kata3	"\"Coba jelaskan pengertian 'cross-validation' dalam Machine Learning!\""
Kata4	"Surat keputusan itu bernomor 62/UN.34/19/2023."

3. Hasil Running Program 3: Menentukan Akar Persamaan Kuadrat

- Python

1) Uji 1



```
import math

# Fungsi untuk menghitung akar persamaan kuadrat
def hitung_akar(data_koefisien):
    nilai_a, nilai_b, nilai_c = data_koefisien
    nilai_diskriminan = (nilai_b ** 2) - (4 * nilai_a * nilai_c)

    if nilai_diskriminan >= 0:
        akar_1 = (-nilai_b + math.sqrt(nilai_diskriminan)) / (2 * nilai_a)
        akar_2 = (-nilai_b - math.sqrt(nilai_diskriminan)) / (2 * nilai_a)
        print(f"Akar pertama = {akar_1:.3f}")
        print(f"Akar kedua = {akar_2:.3f}")
    else:
        print("Persamaan memiliki akar imajiner")

# Input pengguna
nilai_a = float(input("Masukkan koefisien a : "))
nilai_b = float(input("Masukkan koefisien b : "))
nilai_c = float(input("Masukkan koefisien c : "))

# List
data_koefisien = [nilai_a, nilai_b, nilai_c]

# Pemanggilan fungsi
hitung_akar(data_koefisien)
```

... Masukkan koefisien a : 1
Masukkan koefisien b : -5
Masukkan koefisien c : 6
Akar pertama = 3.000
Akar kedua = -2.000

2) Uji 2

```
03_akar_kuadrat.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text ▶ Run all +

import math

# Fungsi untuk menghitung akar persamaan kuadrat
def hitung_akar(data_koefisien):

    nilai_a, nilai_b, nilai_c = data_koefisien

    nilai_diskriminan = (nilai_b ** 2) - (4 * nilai_a * nilai_c)

    if nilai_diskriminan >= 0:

        akar_1 = (-nilai_b + math.sqrt(nilai_diskriminan)) / (2 * nilai_a)
        akar_2 = (-nilai_b - math.sqrt(nilai_diskriminan)) / (2 * nilai_a)

        print(f"Akar pertama = {akar_1:.3f}")
        print(f"Akar kedua = {akar_2:.3f}")

    else:
        print("Persamaan memiliki akar imajiner")

# Input pengguna
nilai_a = float(input("Masukkan koefisien a : "))
nilai_b = float(input("Masukkan koefisien b : "))
nilai_c = float(input("Masukkan koefisien c : "))

# List
data_koefisien = [nilai_a, nilai_b, nilai_c]

# Pemanggilan fungsi
hitung_akar(data_koefisien)

Masukkan koefisien a : 2
Masukkan koefisien b : -2
Masukkan koefisien c : 4
Persamaan memiliki akar imajiner
```

3) Uji 3

```
import math

# Fungsi untuk menghitung akar persamaan kuadrat
def hitung_akar(data_koefisien):

    nilai_a, nilai_b, nilai_c = data_koefisien

    nilai_diskriminan = (nilai_b ** 2) - (4 * nilai_a * nilai_c)

    if nilai_diskriminan >= 0:

        akar_1 = (-nilai_b + math.sqrt(nilai_diskriminan)) / (2 * nilai_a)
        akar_2 = (-nilai_b - math.sqrt(nilai_diskriminan)) / (2 * nilai_a)

        print(f"Akar pertama = {akar_1:.3f}")
        print(f"Akar kedua = {akar_2:.3f}")

    else:
        print("Persamaan memiliki akar imajiner")

# Input pengguna
nilai_a = float(input("Masukkan koefisien a : "))
nilai_b = float(input("Masukkan koefisien b : "))
nilai_c = float(input("Masukkan koefisien c : "))

# List
data_koefisien = [nilai_a, nilai_b, nilai_c]

# Pemanggilan fungsi
hitung_akar(data_koefisien)

Masukkan koefisien a : 0
Masukkan koefisien b : 3
Masukkan koefisien c : -2
ZeroDivisionError: float division by zero
/tmp/ipykernel_1745/1217246129.py in <cell line: 0>()
29
30 # Pemanggilan fungsi
----> 31 hitung_akar(data_koefisien)

/tmp/ipykernel_1745/1217246129.py in hitung_akar(data_koefisien)
10     if nilai_diskriminan >= 0:
11
----> 12         akar_1 = (-nilai_b + math.sqrt(nilai_diskriminan)) / (2 * nilai_a)
13         akar_2 = (-nilai_b - math.sqrt(nilai_diskriminan)) / (2 * nilai_a)
14

ZeroDivisionError: float division by zero

Next steps: Explain error
```

- R

1) Uji 1

```
cat("==== PROGRAM MENCARI AKAR PERSAMAAN KUADRAT ====\\n")
## ===== PROGRAM MENCARI AKAR PERSAMAAN KUADRAT =====
a <- 1
b <- -5
c <- 6

diskriminan <- b^2 - 4*a*c

if (a == 0) {
  cat("Error: Nilai a tidak boleh 0.\\n")
} else if (diskriminan < 0) {
  cat("Persamaan memiliki akar imajiner.\\n")
} else {
  x1 <- (-b + sqrt(diskriminan)) / (2*a)
  x2 <- (-b - sqrt(diskriminan)) / (2*a)

  cat("Koefisien a =", a, "\\n")
  cat("Koefisien b =", b, "\\n")
  cat("Koefisien c =", c, "\\n")
  cat("Diskriminan =", diskriminan, "\\n")
  cat("Akar pertama =", x1, "\\n")
  cat("Akar kedua =", x2, "\\n")
}

## Koefisien a = 1
## Koefisien b = -5
## Koefisien c = 6
## Diskriminan = 1
## Akar pertama = 3
## Akar kedua = 2
```

2) Uji 2

```
a<-2
b<- -2
c<-4
diskriminan <- b^2 - 4*a*c

if (a == 0) {
  cat("Error: Nilai a tidak boleh 0.\\n")
} else if (diskriminan < 0) {
  cat("Persamaan memiliki akar imajiner.\\n")
} else {
  x1 <- (-b + sqrt(diskriminan)) / (2*a)
  x2 <- (-b - sqrt(diskriminan)) / (2*a)

  cat("Koefisien a =", a, "\\n")
  cat("Koefisien b =", b, "\\n")
  cat("Koefisien c =", c, "\\n")
  cat("Diskriminan =", diskriminan, "\\n")
  cat("Akar pertama =", x1, "\\n")
  cat("Akar kedua =", x2, "\\n")
}

## Persamaan memiliki akar imajiner.
```

3) Uji 3

```
a<-0
b<-3
c<-2
diskriminan <- b^2 - 4*a*c

if (a == 0) {

  cat("Error: Nilai a tidak boleh 0.\n")

} else if (diskriminan < 0) {

  cat("Persamaan memiliki akar imajiner.\n")

} else {

  x1 <- (-b + sqrt(diskriminan)) / (2*a)
  x2 <- (-b - sqrt(diskriminan)) / (2*a)

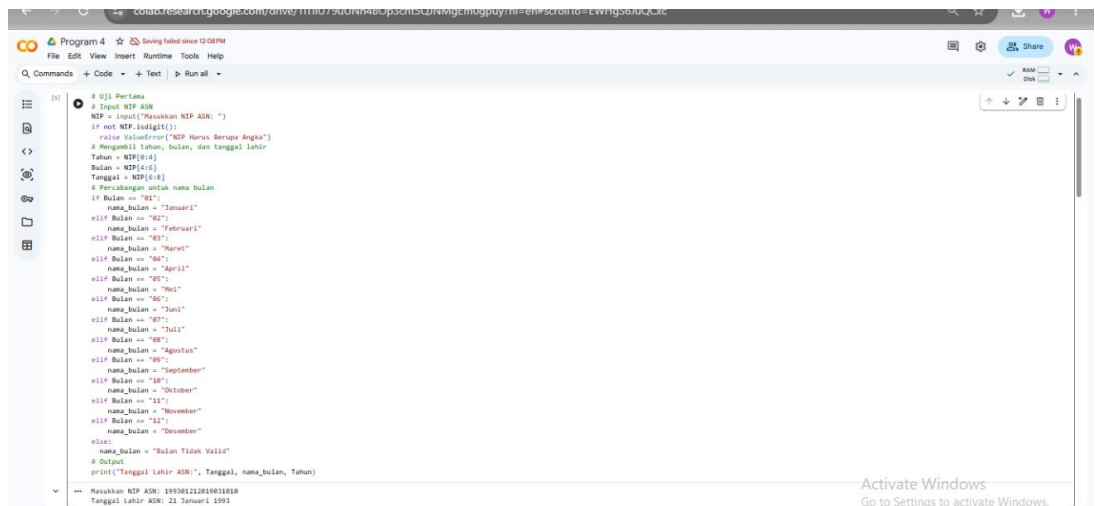
  cat("Koefisien a =", a, "\n")
  cat("Koefisien b =", b, "\n")
  cat("Koefisien c =", c, "\n")
  cat("Diskriminan =", diskriminan, "\n")
  cat("Akar pertama =", x1, "\n")
  cat("Akar kedua =", x2, "\n")
}

## Error: Nilai a tidak boleh 0.
```

4. Hasil Running Program 4: Menampilkan Tanggal Lahir ASN Berdasarkan NIP

- Python

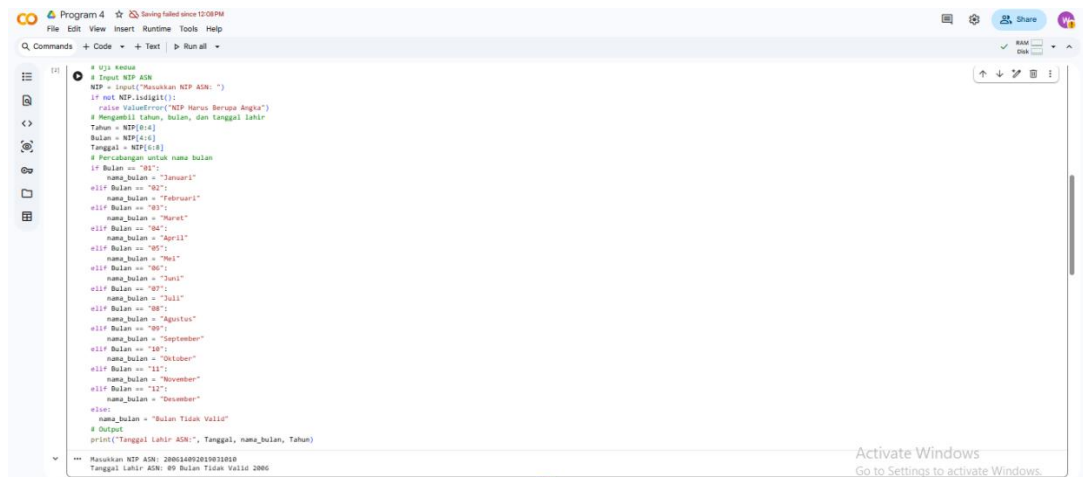
1) Uji 1



```
# Uji Pertama
# Input NIP ASN
NIP = input("Masukkan NIP ASN: ")
if not NIP.isdigit():
    raise ValueError("NIP Harus Berupa Angka")
# Mengambil tahun, bulan, dan tanggal lahir
Tahun = NIP[6:8]
Bulan = NIP[8:10]
Tanggal = NIP[10:12]
# Perbandingan untuk nama bulan
if Bulan == "01":
    nama_bulan = "Januari"
elif Bulan == "02":
    nama_bulan = "Februari"
elif Bulan == "03":
    nama_bulan = "Maret"
elif Bulan == "04":
    nama_bulan = "April"
elif Bulan == "05":
    nama_bulan = "Mei"
elif Bulan == "06":
    nama_bulan = "Juni"
elif Bulan == "07":
    nama_bulan = "Juli"
elif Bulan == "08":
    nama_bulan = "Agustus"
elif Bulan == "09":
    nama_bulan = "September"
elif Bulan == "10":
    nama_bulan = "Oktober"
elif Bulan == "11":
    nama_bulan = "November"
elif Bulan == "12":
    nama_bulan = "Desember"
else:
    nama_bulan = "Bulan Tidak Valid"
# Output
print("Tanggal Lahir ASN:", Tanggal, nama_bulan, Tahun)

==> Masukan NIP ASN: 1990012119010101010
Tanggal Lahir ASN: 21 Januari 1990
```

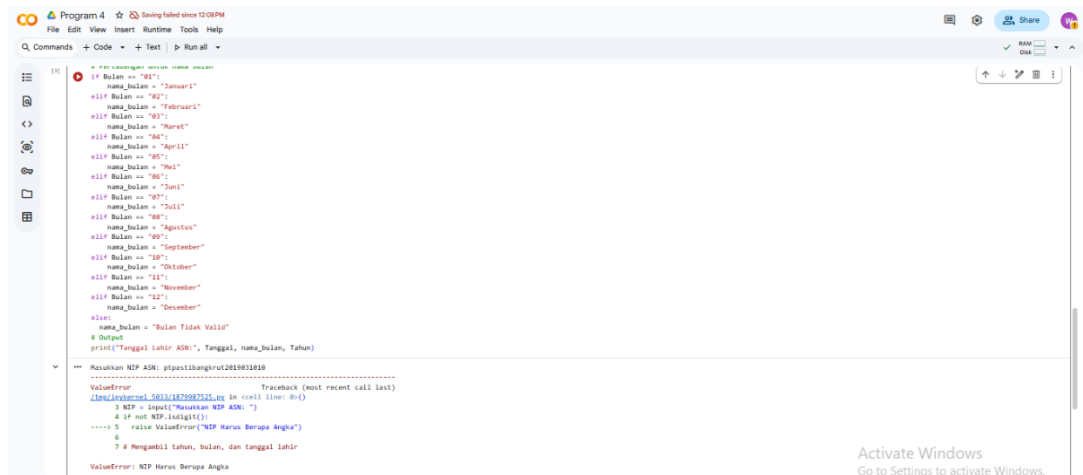
2) Uji 2



```
# Uji ke-2
# Input NIP ASN
NIP = input("Masukkan NIP ASN: ")
if not NIP.isdigit():
    raise ValueError("NIP Harus Berupa Angka")
# Mengambil tahun, bulan, dan tanggal lahir
Tahun = NIP[6:8]
Bulan = NIP[8:10]
Tanggal = NIP[10:]
# Percabangan untuk nama bulan
if Bulan == "01":
    nama_bulan = "Januari"
elif Bulan == "02":
    nama_bulan = "Februari"
elif Bulan == "03":
    nama_bulan = "Maret"
elif Bulan == "04":
    nama_bulan = "April"
elif Bulan == "05":
    nama_bulan = "Mei"
elif Bulan == "06":
    nama_bulan = "Juni"
elif Bulan == "07":
    nama_bulan = "Juli"
elif Bulan == "08":
    nama_bulan = "Agustus"
elif Bulan == "09":
    nama_bulan = "September"
elif Bulan == "10":
    nama_bulan = "Oktober"
elif Bulan == "11":
    nama_bulan = "November"
elif Bulan == "12":
    nama_bulan = "Desember"
else:
    nama_bulan = "Bulan Tidak Valid"
# Output
print("Tanggal Lahir ASN:", Tanggal, nama_bulan, Tahun)
```

*** Masukan NIP ASN: 200610090201001008
Tanggal Lahir ASN: 09 Bulan Tidak Valid 2006

3) Uji 3



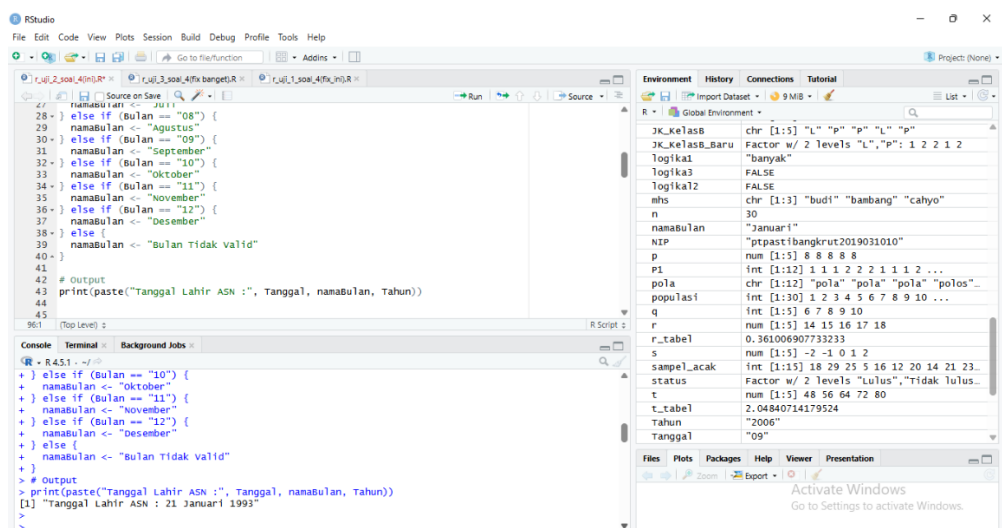
```
# Percabangan untuk nama bulan
if Bulan == "01":
    nama_bulan = "Januari"
elif Bulan == "02":
    nama_bulan = "Februari"
elif Bulan == "03":
    nama_bulan = "Maret"
elif Bulan == "04":
    nama_bulan = "April"
elif Bulan == "05":
    nama_bulan = "Mei"
elif Bulan == "06":
    nama_bulan = "Juni"
elif Bulan == "07":
    nama_bulan = "Juli"
elif Bulan == "08":
    nama_bulan = "Agustus"
elif Bulan == "09":
    nama_bulan = "September"
elif Bulan == "10":
    nama_bulan = "Oktober"
elif Bulan == "11":
    nama_bulan = "November"
elif Bulan == "12":
    nama_bulan = "Desember"
else:
    nama_bulan = "Bulan Tidak Valid"
# Output
print("Tanggal Lahir ASN:", Tanggal, nama_bulan, Tahun)
```

*** Masukan NIP ASN: ptpastibangkrt2019031010

ValueError Traceback (most recent call last)
 /opt/conda/lib/python3.8/site-packages/ipykernel_launcher.py in <cell line: 0>()
 3 NIP = input("Masukkan NIP ASN: ")
 4 if not NIP.isdigit():
----> 5 raise ValueError("NIP Harus Berupa Angka")
 6
 7 # Mengambil tahun, bulan, dan tanggal lahir
ValueError: NIP Harus Berupa Angka

- R

1) Uji 1



```
# RStudio
File Edit Code View Plots Session Build Debug Profile Tools Help
Go to file/function
Source
Run
Source
96:1 (Top Level) 2
R - R4.5.1 - ~/
+ } else if (Bulan == "10") {
+   namaBulan <- "Oktober"
+ } else if (Bulan == "11") {
+   namaBulan <- "November"
+ } else if (Bulan == "12") {
+   namaBulan <- "Desember"
+ } else {
+   namaBulan <- "Bulan Tidak valid"
+ }
+ # output
+ print(paste("Tanggal Lahir ASN :", Tanggal, namaBulan, Tahun))
[1] "tanggal Lahir ASN : 21 Januari 1993"
```

Environment History Connections Tutorial
R - Global Environment
JK_kelas8 chr [1:5] "L" "p" "p" "L" "p"
JK_kelas8_baru Factor w/ 2 levels "L", "p": 1 2 2 1 2
logika1 "banyak"
logika3 FALSE
logika12 FALSE
mhs chr [1:3] "budi" "bambang" "cahyo"
n 30
namaBulan "Januari"
NIP "ptpastibangkrt2019031010"
num [1:5] 8 8 8 8 8
p fnt [1:12] 1 1 1 2 2 2 1 1 1 2 ...
pola chr [1:12] "pola" "pola" "pola" "polos"
populasi fnt [1:30] 1 2 3 4 5 6 7 8 9 10 ...
q fnt [1:5] 6 7 8 9 10
r num [1:5] 14 15 16 17 18
r_tabel 0.361006907733233
s num [1:5] -2 -1 0 1 2
sampel_acak fnt [1:15] 18 29 25 5 16 12 20 14 21 23 ...
status Factor w/ 2 levels "lulus", "Tidak lulus"
t num [1:5] 48 56 64 72 80
t_tabel 2.04840714179524
tahun "2006"
tanggal "09"

2) Uji 2

```

73 } else if (Bulan == "08") {
74   namaBulan <- "Agustus"
75 } else if (Bulan == "09") {
76   namaBulan <- "September"
77 } else if (Bulan == "10") {
78   namaBulan <- "Oktober"
79 } else if (Bulan == "11") {
80   namaBulan <- "November"
81 } else if (Bulan == "12") {
82   namaBulan <- "Desember"
83 } else {
84   stop("Bulan tidak valid")
85 }
86
87 # output
88 print(paste("Tanggal Lahir ASN :", Tanggal, namaBulan, Tahun))
89
90
91

```

Environment

JK_Kelas8	chr	[1:5]	"L" "p" "p" "L" "p"
JK_Kelas8_Baru	Factor w/ 2 levels	"L", "p": 1 2 2 1 2	
logika1			"banyak"
logika3			FALSE
logika2			FALSE
mhs	chr	[1:3]	"budi" "banbang" "cahyo"
n			30
namaBulan			"Januari"
NIP			"ptpastibangkrut2019031010"
p	num	[1:5]	8 8 8 8 8
p1	int	[1:12]	1 1 1 2 2 2 1 1 1 2 ...
pola	chr	[1:12]	"pola" "pola" "pola" "polos"...
populasi	int	[1:30]	1 2 3 4 5 6 7 8 9 10 ...
q	int	[1:5]	6 7 8 9 10
r	num	[1:5]	14 15 16 17 18
r_tabel			0.361006907733233
s	num	[1:5]	-2 -1 0 1 2
sampel_acak	int	[1:15]	18 29 25 5 16 12 20 14 21 23...
status	Factor w/ 2 levels	"Lulus", "Tidak lulus"	
t	num	[1:5]	48 56 64 72 80
t_tabel			2.04840714179524
tahun			"2006"
tanggal			"09"

3) Uji 3

```

91 #Uji Ketiga
92 # Input NIP ASN
93 NIP <- readline(prompt = "Masukkan NIP ASN: ")
94 if (is.na(as.numeric(NIP))) {
95   stop("Error: NIP Harus Berupa Angka")
96 }
97
98 # Mengambil tahun, bulan, dan tanggal
99 Tahun <- substr(NIP, 1, 4)
100 bulan <- substr(NIP, 5, 6)
101 tanggal <- substr(NIP, 7, 8)
102
103 # Percabangan nama bulan
104 if (bulan == "01") {
105   namaBulan <- "Januari"
106 } else if (bulan == "02") {
107   namaBulan <- "Februari"
108 } else if (bulan == "03") {

```

Environment

JK_Kelas8	chr	[1:5]	"L" "p" "p" "L" "p"
JK_Kelas8_Baru	Factor w/ 2 levels	"L", "p": 1 2 2 1 2	
logika1			"banyak"
logika3			FALSE
logika2			FALSE
mhs	chr	[1:3]	"budi" "banbang" "cahyo"
n			30
namaBulan			"Januari"
NIP			"ptpastibangkrut2019031010"
p	num	[1:5]	8 8 8 8 8
p1	int	[1:12]	1 1 1 2 2 2 1 1 1 2 ...
pola	chr	[1:12]	"pola" "pola" "pola" "polos"...
populasi	int	[1:30]	1 2 3 4 5 6 7 8 9 10 ...
q	int	[1:5]	6 7 8 9 10
r	num	[1:5]	14 15 16 17 18
r_tabel			0.361006907733233
s	num	[1:5]	-2 -1 0 1 2
sampel_acak	int	[1:15]	18 29 25 5 16 12 20 14 21 23...
status	Factor w/ 2 levels	"Lulus", "Tidak lulus"	
t	num	[1:5]	48 56 64 72 80
t_tabel			2.04840714179524
tahun			"2006"
tanggal			"09"

5. Hasil Running Program 5: Klasifikasi Cluster Tiga Dimensi

- Python

1) Uji 1

```

def hitung_jarak(titik_u, c):
    if jarak_a < jarak_b and jarak_a < jarak_c:
        cluster = "A"
    elif jarak_b < jarak_a and jarak_b < jarak_c:
        cluster = "B"
    else:
        cluster = "C"

    print(f"Jarak ke Cluster A = {jarak_a:.3f}")
    print(f"Jarak ke Cluster B = {jarak_b:.3f}")
    print(f"Jarak ke Cluster C = {jarak_c:.3f}")
    print(f"Titik U termasuk Cluster {cluster}")

except ValueError:
    print("Input harus berupa angka.")

```

Daftar isi

- Bagian

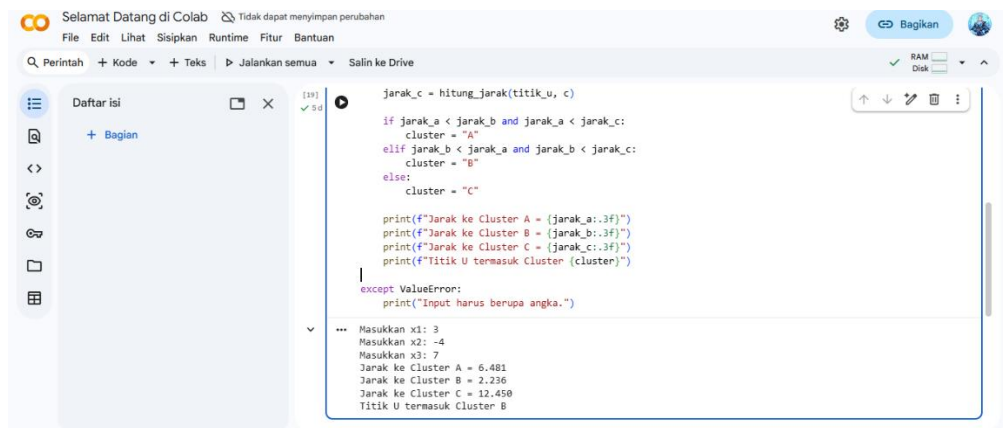
Terminal

```

Masukkan x1: 2
Masukkan x2: 3
Masukkan x3: 4
Jarak ke Cluster A = 2.236
Jarak ke Cluster B = 7.348
Jarak ke Cluster C = 7.211
Titik U termasuk Cluster A

```

2) Uji 2



Selamat Datang di Colab

File Edit Lihat Sisipkan Runtime Fitur Bantuan

Q Perintah + Kode + Teks ▶ Jalankan semua Salin ke Drive

Daftar isi

+ Bagian

```
jarak_c = hitung_jarak(titik_u, c)

if jarak_a < jarak_b and jarak_a < jarak_c:
    cluster = "A"
elif jarak_b < jarak_a and jarak_b < jarak_c:
    cluster = "B"
else:
    cluster = "C"

print(f"Jarak ke Cluster A = {jarak_a:.3f}")
print(f"Jarak ke Cluster B = {jarak_b:.3f}")
print(f"Jarak ke Cluster C = {jarak_c:.3f}")
print(f"Titik U termasuk Cluster {cluster}")

except ValueError:
    print("Input harus berupa angka.")
```

Masukkan x1: 3
Masukkan x2: -4
Masukkan x3: 7
Jarak ke Cluster A = 6.481
Jarak ke Cluster B = 2.236
Jarak ke Cluster C = 12.450
Titik U termasuk Cluster B

3) Uji 3



Q Perintah + Kode + Teks ▶ Jalankan semua Tidak dapat menyimpan perubahan

Daftar isi

+ Bagian

```
x1 = float(input("Masukkan x1: "))
x2 = float(input("Masukkan x2: "))
x3 = float(input("Masukkan x3: "))

titik_u = (x1, x2, x3)

jarak_a = hitung_jarak(titik_u, a)
jarak_b = hitung_jarak(titik_u, b)
jarak_c = hitung_jarak(titik_u, c)

if jarak_a < jarak_b and jarak_a < jarak_c:
    cluster = "A"
elif jarak_b < jarak_a and jarak_b < jarak_c:
    cluster = "B"
else:
    cluster = "C"

print(f"Jarak ke Cluster A = {jarak_a:.3f}")
print(f"Jarak ke Cluster B = {jarak_b:.3f}")
print(f"Jarak ke Cluster C = {jarak_c:.3f}")
print(f"Titik U termasuk Cluster {cluster}")

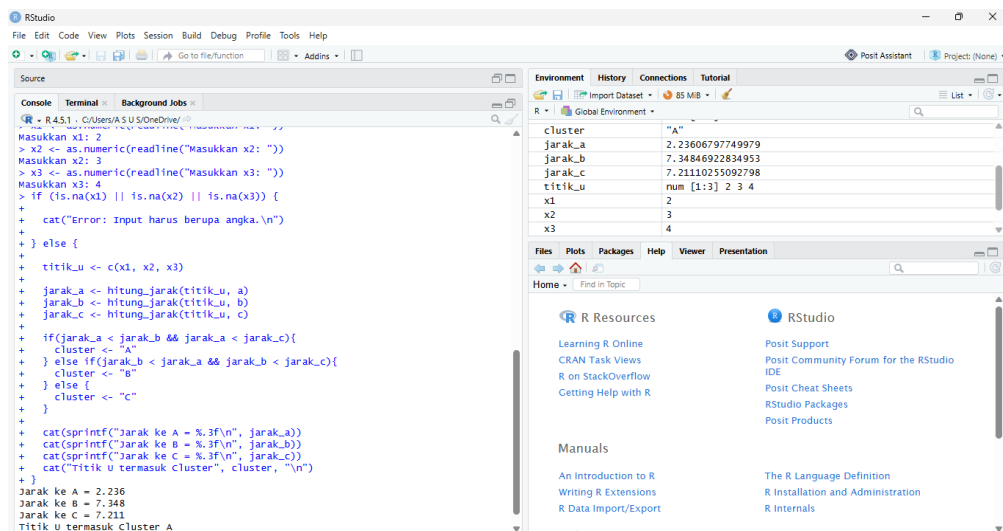
except ValueError:
    print("Input harus berupa angka.")
```

Masukkan x1: 2
Masukkan x2: b
Input harus berupa angka.

Terminal

- R

1) Uji 1



RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Source

```
x1 = as.numeric(readline("Masukkan x1: "))
x2 = as.numeric(readline("Masukkan x2: "))
x3 = as.numeric(readline("Masukkan x3: "))

if (is.na(x1) || is.na(x2) || is.na(x3)) {
  cat("Error: Input harus berupa angka.\n")
} else {
  titik_u = c(x1, x2, x3)

  jarak_a = hitung_jarak(titik_u, a)
  jarak_b = hitung_jarak(titik_u, b)
  jarak_c = hitung_jarak(titik_u, c)

  if (jarak_a < jarak_b && jarak_a < jarak_c) {
    cluster = "A"
  } else if (jarak_b < jarak_a && jarak_b < jarak_c) {
    cluster = "B"
  } else {
    cluster = "C"
  }

  cat(sprintf("Jarak ke A = %.3f\n", jarak_a))
  cat(sprintf("Jarak ke B = %.3f\n", jarak_b))
  cat(sprintf("Jarak ke C = %.3f\n", jarak_c))
  cat("Titik U termasuk Cluster", cluster, "\n")
}
```

Console

```
Masukkan x1: 2
Masukkan x2: 3
Masukkan x3: 4
Jarak ke A = 2.236
Jarak ke B = 7.348
Jarak ke C = 7.211
Titik U termasuk Cluster A
```

Environment

Object	Value
cluster	"A"
jarak_a	2.23606797749979
jarak_b	7.34846922834953
jarak_c	7.21110255092798
titik_u	num [1:3] 2 3 4
x1	2
x2	3
x3	4

Files Plots Packages Help Viewer Presentation

Home Find in Topic

R Resources

- Learning R Online
- CRAN Task Views
- R on StackOverflow
- Getting Help with R

Manuals

- An Introduction to R
- Writing R Extensions
- R Data Import/Export

RStudio

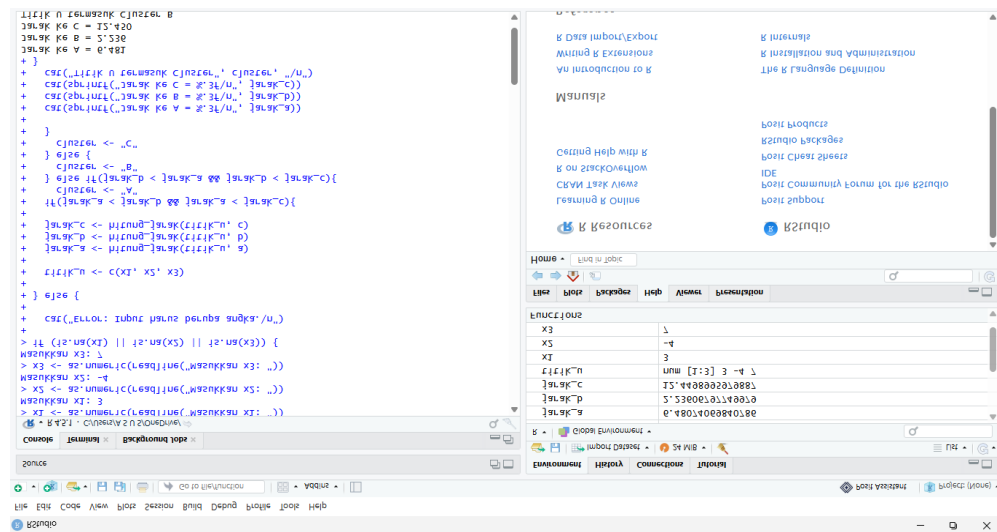
- Posit Support
- Posit Community Forum for the RStudio IDE
- Posit Cheat Sheets
- RStudio Packages
- Posit Products

The R Language Definition

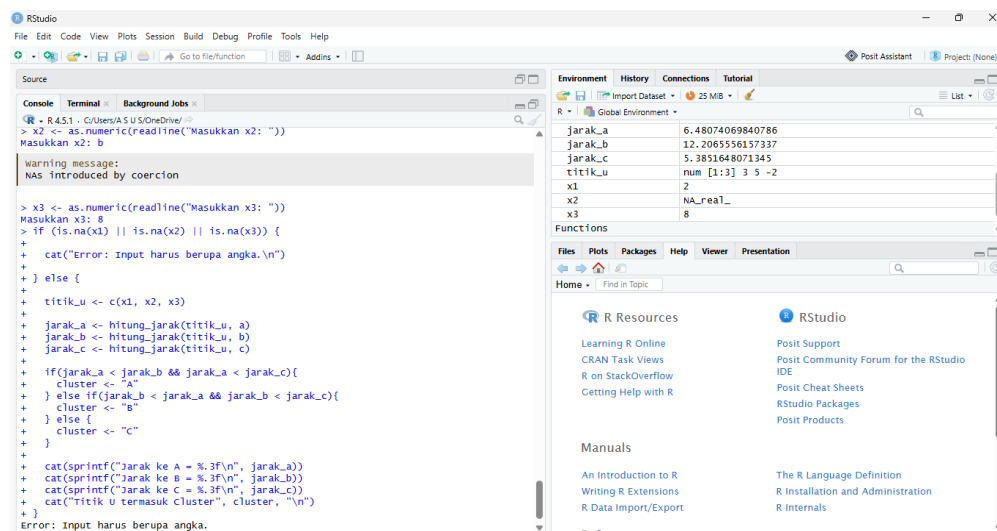
R Installation and Administration

R Internals

2) Uji 2



3) Uji 3



6. Hasil Running Program 6: Interval Konfidensi Proporsi

- Python

1) Uji 1

```
[4]
✓ 16s
import math

def cari_nilai_kritis(tingkat_signifikansi):
    if tingkat_signifikansi == 0.10:
        return 1.645
    elif tingkat_signifikansi == 0.05:
        return 1.96
    else:
        return None

def hitung_batas_kesalahan(proporsi_sampel, nilai_kritis, ukuran_sampel):
    return nilai_kritis * math.sqrt(
        (proporsi_sampel * (1 - proporsi_sampel)) / ukuran_sampel
    )

def hitung_interval_kepercayaan(proporsi_sampel, tingkat_signifikansi, ukuran_sampel):

    if proporsi_sampel < 0 or proporsi_sampel > 1:
        return "Error: Nilai proporsi harus antara 0 dan 1."

    nilai_kritis = cari_nilai_kritis(tingkat_signifikansi)

    if nilai_kritis is None:
        return "Error: Tingkat signifikansi hanya boleh 0.10 atau 0.05."

    batas_kesalahan = hitung_batas_kesalahan(
        proporsi_sampel,
        nilai_kritis,
        ukuran_sampel
    )

    interval_minimum = proporsi_sampel - batas_kesalahan
    interval_maksimum = proporsi_sampel + batas_kesalahan

    return interval_minimum, interval_maksimum

# Program Utama
nilai_proporsi = float(input("Masukkan nilai proporsi sampel: "))
alpha_input = float(input("Masukkan tingkat signifikansi (0.10 atau 0.05): "))
jumlah_data = int(input("Masukkan ukuran sampel: "))

hasil_perhitungan = hitung_interval_kepercayaan(
    nilai_proporsi,
    alpha_input,
    jumlah_data
)

if isinstance(hasil_perhitungan, str):
    print(hasil_perhitungan)
else:
    batas_awal, batas_akhir = hasil_perhitungan
    print(
        f"Interval Kepercayaan = ({batas_awal:.4f}, {batas_akhir:.4f})"
    )

*** Masukkan nilai proporsi sampel: 0.9
Masukkan tingkat signifikansi (0.10 atau 0.05): 0.1
Masukkan ukuran sampel: 400
Interval Kepercayaan = (0.8753, 0.9247)
```

2) Uji 2

```
[5]
✓ 10s ▶ import math

def cari_nilai_kritis(tingkat_signifikansi):
    if tingkat_signifikansi == 0.10:
        return 1.645
    elif tingkat_signifikansi == 0.05:
        return 1.96
    else:
        return None

def hitung_batas_kesalahan(proporsi_sampel, nilai_kritis, ukuran_sampel):
    return nilai_kritis * math.sqrt(
        (proporsi_sampel * (1 - proporsi_sampel)) / ukuran_sampel
    )

def hitung_interval_kepercayaan(proporsi_sampel, tingkat_signifikansi, ukuran_sampel):

    if proporsi_sampel < 0 or proporsi_sampel > 1:
        return "Error: Nilai proporsi harus antara 0 dan 1."

    nilai_kritis = cari_nilai_kritis(tingkat_signifikansi)

    if nilai_kritis is None:
        return "Error: Tingkat signifikansi hanya boleh 0.10 atau 0.05."

    batas_kesalahan = hitung_batas_kesalahan(
        proporsi_sampel,
        nilai_kritis,
        ukuran_sampel
    )

    interval_minimum = proporsi_sampel - batas_kesalahan
    interval_maksimum = proporsi_sampel + batas_kesalahan

    return interval_minimum, interval_maksimum

# Program Utama
nilai_proporsi = float(input("Masukkan nilai proporsi sampel: "))
alpha_input = float(input("Masukkan tingkat signifikansi (0.10 atau 0.05): "))
jumlah_data = int(input("Masukkan ukuran sampel: "))

hasil_perhitungan = hitung_interval_kepercayaan(
    nilai_proporsi,
    alpha_input,
    jumlah_data
)

if isinstance(hasil_perhitungan, str):
    print(hasil_perhitungan)
else:
    batas_awal, batas_akhir = hasil_perhitungan
    print(
        f"Interval Kepercayaan = ({batas_awal:.4f}, {batas_akhir:.4f})"
    )

▼ ... Masukkan nilai proporsi sampel: 1.6
    Masukkan tingkat signifikansi (0.10 atau 0.05): 0.05
    Masukkan ukuran sampel: 3000
    Error: Nilai proporsi harus antara 0 dan 1.
```

3) Uji 3

```
[#] 15s
import math

def cari_nilai_kritis(tingkat_signifikansi):
    if tingkat_signifikansi == 0.10:
        return 1.645
    elif tingkat_signifikansi == 0.05:
        return 1.96
    else:
        return None

def hitung_batas_kesalahan(proporsi_sampel, nilai_kritis, ukuran_sampel):
    return nilai_kritis * math.sqrt(
        (proporsi_sampel * (1 - proporsi_sampel)) / ukuran_sampel
    )

def hitung_interval_kepercayaan(proporsi_sampel, tingkat_signifikansi, ukuran_sampel):

    if proporsi_sampel < 0 or proporsi_sampel > 1:
        return "Error: Nilai proporsi harus antara 0 dan 1."

    nilai_kritis = cari_nilai_kritis(tingkat_signifikansi)

    if nilai_kritis is None:
        return "Error: Tingkat signifikansi hanya boleh 0.10 atau 0.05."

    batas_kesalahan = hitung_batas_kesalahan(
        proporsi_sampel,
        nilai_kritis,
        ukuran_sampel
    )

    interval_minimum = proporsi_sampel - batas_kesalahan
    interval_maksimum = proporsi_sampel + batas_kesalahan

    return interval_minimum, interval_maksimum

# Program Utama
nilai_proporsi = float(input("Masukkan nilai proporsi sampel: "))
alpha_input = float(input("Masukkan tingkat signifikansi (0.10 atau 0.05): "))
jumlah_data = int(input("Masukkan ukuran sampel: "))

hasil_perhitungan = hitung_interval_kepercayaan(
    nilai_proporsi,
    alpha_input,
    jumlah_data
)

if isinstance(hasil_perhitungan, str):
    print(hasil_perhitungan)
else:
    batas_awal, batas_akhir = hasil_perhitungan
    print(
        f"Interval Kepercayaan = ({batas_awal:.4f}, {batas_akhir:.4f})"
    )

*** Masukkan nilai proporsi sampel: 0.7
Masukkan tingkat signifikansi (0.10 atau 0.05): 0.03
Masukkan ukuran sampel: 500
Error: Tingkat signifikansi hanya boleh 0.10 atau 0.05.
```

- R

1) Uji 1

```

RStudio

File Edit Code View Plots Session Build Debug Profile Tools Help

Source
Console Terminal Background Jobs

> #Program 6 - Interval proporsi
> #Menghitung interval konfidensi proporsi
> hitung_interval_konfidensi <- function(phat, n, alpha) {
+   #Validasi input
+   if (is.na(phat) || is.na(n) || is.na(alpha)) {
+     cat("ERROR: Input harus berupa angka.\n")
+     return(NULL)
+   }
+   if (phat < 0 || phat > 1) {
+     cat("ERROR: Proporsi sampel (phat) harus antara 0 dan 1.\n")
+     return(NULL)
+   }
+   if (n <= 0 || n != round(n)) {
+     cat("ERROR: Ukuran sampel (n) harus bilangan bulat positif.\n")
+     return(NULL)
+   }
+   if (alpha == 0.10) {
+     z <- 1.645
+   } else if (alpha == 0.05) {
+     z <- 1.96
+   } else {
+     cat("ERROR: Alpha hanya boleh 0.05 atau 0.10.\n")
+     return(NULL)
+   }
+   #Perhitungan
+   margin_error <- z * sqrt(phat * (1 - phat) / n)
+   lower <- phat - margin_error
+   upper <- phat + margin_error
+   return(list(
+     phat = phat,
+     n = n,
+     alpha = alpha,
+     confidence_level = (1 - alpha) * 100,
+     z = z,
+     lower_bound = lower,
+     upper_bound = upper
+   ))
+ }
> #Program utama
> phat <- as.numeric(readline("Masukkan proporsi sampel (0 - 1): "))
Masukkan proporsi sampel (0 - 1): 0.9
> n <- as.numeric(readline("Masukkan ukuran sampel (n): "))
Masukkan ukuran sampel (n): 400
> alpha <- as.numeric(readline("Masukkan alpha (0.05 atau 0.10): "))
Masukkan alpha (0.05 atau 0.10): 0.10
> hasil <- hitung_interval_konfidensi(phat, n, alpha)
> if (is.null(hasil)) {
+   cat(sprintf("Interval Konfidensi : (%.4f, %.4f)\n",
+     hasil$lower_bound, hasil$upper_bound))
+ } else {
+   cat("\nProgram dihentikan karena input tidak valid.\n")
+ }
Interval Konfidensi : (0.8749, 0.9247)

```

2) Uji 2

```

> #Program utama
> phat <- as.numeric(readline("Masukkan proporsi sampel (0 - 1): "))
Masukkan proporsi sampel (0 - 1): 1.6
> n <- as.numeric(readline("Masukkan ukuran sampel (n): "))
Masukkan ukuran sampel (n): 3000
> alpha <- as.numeric(readline("Masukkan alpha (0.05 atau 0.10): "))
Masukkan alpha (0.05 atau 0.10): 0.05
> hasil <- hitung_interval_konfidensi(phat, n, alpha)
ERROR: Proporsi sampel (phat) harus antara 0 dan 1.
> if (is.null(hasil)) {
+   cat(sprintf("Interval Konfidensi : (%.4f, %.4f)\n",
+     hasil$lower_bound, hasil$upper_bound))
+ } else {
+   cat("\nProgram dihentikan karena input tidak valid.\n")
+ }

Program dihentikan karena input tidak valid.

```

3) Uji 3

```

> #Program utama
> phat <- as.numeric(readline("Masukkan proporsi sampel (0 - 1): "))
Masukkan proporsi sampel (0 - 1): 0.7
> n <- as.numeric(readline("Masukkan ukuran sampel (n): "))
Masukkan ukuran sampel (n): 500
> alpha <- as.numeric(readline("Masukkan alpha (0.05 atau 0.10): "))
Masukkan alpha (0.05 atau 0.10): 0.03
> hasil <- hitung_interval_konfidensi(phat, n, alpha)
ERROR: Alpha hanya boleh 0.05 atau 0.10.
> if (is.null(hasil)) {
+   cat(sprintf("Interval Konfidensi : (%.4f, %.4f)\n",
+     hasil$lower_bound, hasil$upper_bound))
+ } else {
+   cat("\nProgram dihentikan karena input tidak valid.\n")
+ }

Program dihentikan karena input tidak valid.
>

```

6.3 Interpretasi Output

1. Program 1: Menghitung Jumlah Kata dan Jumlah Kalimat

Program 1 adalah program analisis teks yang menghitung jumlah kalimat dan jumlah kata dari sebuah teks yang diinputkan, di mana R menghitung kalimat berdasarkan jumlah titik menggunakan `gsub` dan kata menggunakan `strsplit`, sedangkan Python menggunakan `teks.count(".")` untuk kalimat dan `len(teks.split())` untuk kata.

Pada uji pertama dengan teks bertema media sosial, kedua bahasa menghasilkan output yang identik yaitu 5 kalimat dan 92 kata, membuktikan bahwa logika penghitungan berjalan dengan benar dan konsisten di kedua bahasa untuk teks yang relatif panjang.

Pada uji kedua dengan teks bertema membaca buku, kedua bahasa kembali menghasilkan output yang identik yaitu 4 kalimat dan 78 kata, menunjukkan konsistensi program dalam memproses teks dengan panjang yang berbeda.

Pada uji ketiga dengan teks bertema pendidikan, kedua bahasa juga menghasilkan output yang sama yaitu 4 kalimat dan 80 kata, membuktikan bahwa program stabil dan akurat untuk berbagai jenis teks.

2. Program 2: Membuat String Kalimat

Kata 1 (tanda kutip tunggal dalam string), kedua bahasa menampilkan output yang sama. Penggunaan tanda kutip tunggal ('kan) di dalam string tidak menyebabkan masalah karena string dibungkus dengan tanda kutip ganda.

Kata 2 (tanda kutip ganda di dalam string), Python menampilkan kutipan ganda secara langsung karena string dibungkus dengan kutip tunggal ('...'), sedangkan R menampilkan backslash ("\") karena R secara default menampilkan escape character saat mencetak string berisi kutip ganda.

Kata 3 (kombinasi kutip tunggal dan ganda), Python berhasil menampilkan kutip ganda di awal dan akhir kalimat dengan bersih, sedangkan R menampilkan ("\") sebagai representasi kutip ganda yang di-escape, sehingga tampilannya lebih verbose.

Kata 4 (string dengan karakter khusus, yaitu titik dan garis miring), kedua bahasa menampilkan output yang identik tanpa masalah, karena karakter (.) dan (/) tidak memerlukan escape.

3. Program 3: Menentukan Akar Persamaan Kuadrat

Pada keberhasilan uji pertama, kedua bahasa menghasilkan output yang benar (akar pertama = 3, akar kedua = 2). Perbedaannya hanya pada format tampilan Python yang

menampilkan 3 desimal (3.000 dan 2.000), sedangkan R menampilkan bilangan bulat. Fungsi bekerja dengan benar untuk kasus diskriminan positif.

Pada keberhasilan uji kedua, kedua bahasa berhasil mendeteksi bahwa diskriminan negatif dan menampilkan pesan "Persamaan memiliki akar imajiner". Logika kondisional untuk diskriminan negatif berjalan dengan benar di kedua program.

Pada kegagalan uji ketiga, R berhasil memvalidasi bahwa nilai a tidak boleh 0 dan menampilkan pesan error yang sesuai, sedangkan Python mengalami ZeroDivisionError karena tidak memiliki validasi untuk kondisi $a = 0$, sehingga terjadi pembagian dengan nol pada rumus $2 * \text{nilai}_a$.

4. Program 4: Menampilkan Tanggal Lahir ASN Berdasarkan NIP

Pada keberhasilan uji pertama, kedua bahasa berhasil mengekstrak dan menampilkan tanggal lahir 21 Januari 1993 dengan benar. Program mampu memisahkan komponen tahun (`NIP[0:4]`), bulan (`NIP[4:6]`), dan tanggal (`NIP[6:8]`) dari string NIP, lalu mengkonversi kode bulan "01" menjadi nama bulan "Januari" dengan tepat.

Pada keberhasilan uji kedua, kedua bahasa berhasil mendeteksi bahwa kode bulan "40" tidak termasuk dalam rentang bulan yang valid (01–12), sehingga menampilkan output "9 Bulan Tidak Valid 2006". Logika else pada percabangan if-elif berfungsi dengan benar untuk menangani bulan di luar jangkauan.

Pada kegagalan uji ketiga, R menampilkan pesan "Error: NIP Harus Berupa Angka" sebab validasi berhasil mendeteksi input non-numerik, sedangkan Python menampilkan "ValueError: NIP Harus Berupa Angka" sebab validasi juga berhasil mendeteksi input non-numerik

5. Program 5: Klasifikasi Cluster Tiga Dimensi

Pada keberhasilan uji pertama, kedua bahasa menghasilkan output identik. Jarak ke Cluster A paling kecil, program dengan benar mengklasifikasikan titik U ke Cluster A. Logika kondisional `if jarak_a < jarak_b and jarak_a < jarak_c` berjalan sempurna di kedua bahasa.

Pada keberhasilan uji kedua, kedua bahasa menghasilkan output identic. Jarak ke Cluster B paling kecil sehingga klasifikasi ke Cluster B sudah benar. Program berhasil menangani koordinat negatif tanpa masalah.

Pada uji ketiga, Python menampilkan "Input harus berupa angka." karena except ValueError menangkap error saat `float(input())` menerima huruf "b", sedangkan R menampilkan "Error: Input harus berupa angka." namun prosesnya berbeda. R

mengeluarkan warning "NAs introduced by coercion" terlebih dahulu sebelum validasi `is.na()` mendeteksi input tidak valid.

6. Program 6: Interval Konfidensi Proporsi

Pada keberhasilan uji pertama, kedua bahasan menampilkan output yang sama. Hal ini menunjukkan bahwa perhitungan dengan nilai $z=1,645$ untuk $\alpha 0,10$ berjalan dengan benar dan menghasilkan nilai yang identik di kedua bahasa.

Pada uji kedua ditujukan untuk menguji validasi nilai proporsi yang melebihi batas valid. Python menampilkan "Error: Nilai proporsi harus antara 0 dan 1" dan R menampilkan "Program dihentikan karena input tidak valid", sehingga kedua bahasa berhasil mendeteksi bahwa nilai proporsi 1,6 berada di luar rentang $[0,1]$ meskipun pesan errornya sedikit berbeda redaksinya.

Pada uji ketiga ditujukan untuk menguji validasi tingkat signifikansi yang tidak valid. Python menampilkan "Error: Tingkat signifikansi hanya boleh 0,10 atau 0,05" dan R menampilkan "Program dihentikan karena input tidak valid", membuktikan bahwa kedua bahasa berhasil menolak nilai $\alpha 0,03$ yang tidak termasuk dalam pilihan yang diizinkan.

BAB VII

PENUTUP

7.1 Kesimpulan

Berdasarkan hasil pelaksanaan project Algoritma dan Pemrograman, dapat disimpulkan bahwa seluruh tujuan yang telah ditetapkan telah berhasil dicapai. Analisis terhadap enam studi kasus telah dilakukan dengan mengidentifikasi kebutuhan input, proses, output, serta batasan masalah pada masing-masing program sebagai dasar dalam menentukan solusi yang tepat. Selanjutnya, setiap permasalahan dirancang dalam bentuk algoritma, pseudocode, dan flowchart yang sistematis sehingga alur penyelesaian dapat dipahami dengan jelas sebelum diimplementasikan.

Implementasi algoritma ke dalam bahasa pemrograman Python dan R menunjukkan bahwa kedua bahasa tersebut mampu digunakan untuk menyelesaikan permasalahan komputasional dengan logika yang setara. Dalam proses implementasi, berbagai konsep dasar pemrograman, seperti variabel, tipe data, operator, manipulasi string, percabangan, fungsi, dan struktur data, telah diterapkan sesuai dengan karakteristik masing-masing studi kasus. Selain itu, setiap program telah melalui proses pengujian menggunakan beberapa skenario untuk memastikan bahwa hasil yang diperoleh sesuai dengan kebutuhan serta mampu menangani kondisi-kondisi tertentu yang telah ditentukan.

Melalui keseluruhan rangkaian kegiatan tersebut, project ini tidak hanya menghasilkan enam program yang berfungsi sesuai dengan ketentuan, tetapi juga meningkatkan kemampuan berpikir komputasional, berpikir logis, dan pemecahan masalah secara sistematis. Pengalaman dalam menganalisis masalah, merancang algoritma, mengimplementasikan solusi, serta mengevaluasi hasil program menjadi bekal yang bermanfaat dalam pengembangan kompetensi pemrograman dan penyelesaian permasalahan komputasional di masa mendatang.

7.2 Kendala yang Dihadapi

Selama proses pengerjaan project, terdapat beberapa kendala yang dihadapi oleh kelompok, antara lain:

1. Perbedaan sintaks dan fungsi antara bahasa pemrograman Python dan R sehingga diperlukan penyesuaian agar kedua program menghasilkan keluaran yang setara.
2. Kesulitan dalam merancang algoritma yang mampu menangani berbagai kondisi khusus, seperti validasi input yang tidak sesuai dengan ketentuan soal.

3. Proses mencari, menganalisis, dan memperbaiki kesalahan (bug) memerlukan waktu yang cukup lama karena beberapa kesalahan logika baru terlihat setelah dilakukan pengujian terhadap berbagai skenario.
4. Penyusunan laporan membutuhkan ketelitian agar analisis, pseudocode, flowchart, implementasi program, serta penjelasan setiap baris kode memiliki keterkaitan dan konsistensi.
5. Koordinasi antaranggota kelompok dalam pembagian tugas dan penyatuan hasil pekerjaan menjadi tantangan tersendiri, terutama agar format penulisan dan kualitas isi laporan tetap seragam.

7.3 Saran Pengembangan Program

Program yang telah dibuat pada project ini masih dapat dikembangkan agar menjadi lebih baik. Beberapa saran pengembangan yang dapat dilakukan, antara lain:

1. Menambahkan validasi input yang lebih lengkap sehingga program dapat menangani berbagai bentuk masukan yang tidak sesuai tanpa menghasilkan kesalahan saat dijalankan.
2. Mengembangkan program agar mampu menangani lebih banyak variasi kasus dan kondisi khusus, sehingga hasil yang diberikan menjadi lebih akurat dan sesuai dengan berbagai kemungkinan masukan dari pengguna.
3. Menyederhanakan dan menyusun kode program menjadi lebih modular dengan memanfaatkan fungsi secara lebih optimal. Hal ini akan mempermudah proses pembacaan, pemeliharaan, dan pengembangan program di kemudian hari.
4. Menambahkan lebih banyak skenario pengujian, baik untuk kondisi normal maupun kondisi ekstrem, sehingga keandalan dan ketepatan hasil program dapat dievaluasi dengan lebih baik.
5. Mengembangkan program dengan menerapkan materi pemrograman yang dipelajari pada semester berikutnya, sehingga program dapat memiliki fitur yang lebih lengkap serta mampu menyelesaikan permasalahan yang lebih kompleks.

Dengan adanya pengembangan tersebut, program diharapkan tidak hanya memenuhi kebutuhan tugas perkuliahan, tetapi juga menjadi sarana untuk meningkatkan kemampuan dalam merancang algoritma dan mengimplementasikan solusi komputasional secara lebih efektif.

LAMPIRAN

1. Github

<https://github.com/nisrinayaaprilla/Kelompok-H-Algoritma-Pemrograman/tree/main>

2. Video Demo

<https://drive.google.com/drive/folders/1Ez14NXdxROXXEeVn-ycxiOsTk2AfbqzU>

3. Penggunaan AI

- Program 1

<https://claude.ai/share/e697f77f-0bde-4c40-b885-033cfa102193>

- Program 2

https://chatgpt.com/s/t_6a2a4b5d8b4c81919197e2dc27ac4098

https://chatgpt.com/s/t_6a2e621b9a5c8191a9b759eae719187b

https://chatgpt.com/s/t_6a33678de6b48191ac03620afd0965bf

- Program 3

<https://chatgpt.com/share/6a33600f-da48-83ec-87ec-fabee629ea51>

<https://chatgpt.com/share/6a3362f9-1044-83ec-abbf-990d49bead17>

- Program 4

<https://chatgpt.com/share/6a290670-7190-83ec-b77d-c30613f6d1e3>

- Program 5

<https://chatgpt.com/share/6a283175-a538-83ec-891b-7a2011b3971f>

- Program 6

https://grok.com/share/c2hhcmQtMg_3cc340f6-bac7-4e89-9d77-550319756c21

<https://chatgpt.com/share/6a329b4c-a3e8-83ec-bcb6-f9292b45dccc>

https://chatgpt.com/s/t_6a3368cd29708191987f984565b9ba72